
Japanese Generative AI Technical Survey Special

SP-efficient-llm-2026

Efficient Intelligence——LLM を速く、軽く、安くする技術史 / Thematic survey as of
2026-09-21

Efficient Intelligence

LLM を速く、軽く、安くする技術史。訓練の配分、疎な計算、記憶の節約、低ビット、逐次復号の緩和、運用の束ね、安価計算への振り分けを、層の重なりとして読む。

[Bottleneck Contract](#) / [Conditional Compute](#) / [Attention & Memory](#) / [Precision](#) / [Decoding](#) /
[Serving](#) / [Routing](#) / [2026 Capstones](#) / [Measurement Literacy](#)

Editorial cutoff

Open history, evidence observed through 2026-09-21 UTC

Repository

[eariver/japanese-generative-ai-survey](#)

Build

[LuaLaTeX](#) / [jlreq](#) / [LuaTeX-jā](#)

一次情報・論文・OSS・X 上の技術反応を分離し、検証可能な Evidence chain として編成する Thematic Longform Technical

Survey。

本巻の問いと読み方

本巻の問いは、大規模言語モデル（LLM）の効率化が、単一軸の最適化から、学習・構造・記憶・復号・運用・振り分けにまたがる層状の設計問題へと、どのように進化してきたのかである。対象は基礎的な規模則から疎な混合専門家、注意機構と記憶保持、低ビット化と局所実行、逐次復号の高速化と推論予算の管理、サービング運用、安価計算への振り分け、そして二〇二六年の四つの実装点へと至る。本巻は製品の順位付けでも、特定モデルの推奨でも、条件の異なる数値の横断比較でもない。どの隘路を、どの作業負荷と装置条件のもとで、何で削り、何を代償にしたかを追う技術史である。

重要なのは、ここで「効率」を一語で済ませないことである。学習に要する計算量を減らしたのか、配備後の応答を速くしたのか、同時接続の捌き量を増やしたのか、ひとつの仕事を終えるまでの総費用を下げたのか。これらを区別せずに手法だけを並べれば、あらゆる比較は意味を失う。第 1 節で結ぶ用語と隘路の契約が、本巻全体の共通言語になる。

確認範囲と主張境界

本巻の技術的主張は、受理された証拠の範囲に縛られる。一次資料の本文まで読み込めた箇所もあれば、要旨の範囲に限定される箇所もある。概要水準の主張は概要水準として扱い、本文で裏付けられた言語では記述しない。提供元による速度・価格・精度の数値は、測定条件付きの主張として扱い、独立検証のない数値は事実として引用しない。異なる条件で測られた性能値を直接並べて優劣を論じることがはしない。測定条件や文脈長や同時実行数が異なれば、律速要因そのものが変わるからである。

未解決の事項は未解決のまま残す。独立した再現が未了の主張、公開の較正手順が見当たらない評価、単独の製品姿が確認できない後継の存在、本文未読の制約、版や対応状況の未確定は、記事を読みやすくするために閉じない。境界の明示は弱さではなく、本巻の読みの作法である。

本巻の限界の要約

本巻の限界をあらかじめ要約する。特化型の独立再現は薄く、公開較正手順は未解決であり、提供元主張は提供元主張のまま扱う。第四の実装点の技術報告は要旨限定であり、本文言語では記述しない。提供元比率は提供元測定のまま隔離する。独立再現が限定的な実装、要約範囲の脆弱性分析、節単位の消費が残る方法論、一部未取得の実装・文書・仕様本体、図別の未確定、取得済みだが未消費の本体、誤った正準探索子の履歴を補遺で修復した経緯、配置依存の観察は、いずれも本文中で境界として残す。これらを隠して記事を整えないことが、本巻の編集方針である。

この巻で比較するもの

本巻が横断的に比較するのは、モデルの絶対順位ではなく、どの設計問題が第一級の競争軸になったかである。具体的には、学習の資源配分、疎な条件付き計算、注意と記憶保持の設計、数値精度と局所実行、逐次復号と推論予算、運用の束ねと受け渡し、安価計算への振り分けと特化判断を追う。二〇二六年の四つの実装点は、順位付けではなく機構の積み重ねとして読み解く。最終節では、数字を読むための物差しそのものを点検する。

効率史の時代区分

本巻の通史を三期に分ける。第一期は、密な規模拡大の時代である。損失の規模則を指針に、模型とデータと計算をそろえて増やすことが性能の向上に直結した。大きく作り、後から小さく使う発想が共有された。第二期は、条件付き計算と記憶節約の時代である。疎な混合専門家が容量と計算を切り離し、鍵と値の共有化と疎化が保持の設計を変え、重みの低ビット化が手元実行の道を開いた。第三期は、運用と一体の設計の時代である。逐次復号の先読みと推論予算の管理、運用の束ねと受け渡し、安価計算への振り分けが、模型内部の工

夫と対等の重みを持ち始めた。三期は断絶ではなく積み重ねであり、後の層は前の層を置き換えるのではなく、前提として使う。規模則の配分は組成の設計に、疎な活性は常駐の設計に、記憶の節約は運用の設計に接続する。時代区分は、層の重なりを時間の言葉で言い直したものである。

時代	代表的手法	中心の問い
第一期・密な規模拡大	規模則と配分の見直し	大きく作るか賢く配分するか
第二期・条件と記憶の節約	疎な混合専門家と共有・疎化・低ビット	どこを動かし何を保持するか
第三期・運用一体の設計	先読みと予算管理と束ね・振り分け	どう捌きどう選ぶか

記号と表記の約束

本巻では、総パラメータを総量、有効に働く部分を作動量と呼ぶ。鍵と値の中間記憶を KV キャッシュ、前処理段階をプリフィル、逐次生成段階をデコードと呼ぶ。遅延は応答の待ち、処理能力は単位時間の捌き量を指す。トークン単価は単位の安さ、仕事単価は課題完遂の費用を指す。提供元による数値は提供元主張、独立検証のない数値は未確認として扱い、本文で断定的に引用しない。技術的事実を確定する典拠は一次資料・公式文書・実装記録・公開手順書を基本とし、受容記録やコミュニティ資料を引用する場合は配置依存の観察・再現の手がかりに限定し、技術的事実の根拠には用いない。

引用の約束

本巻の引用は、主張の出所を示すためのものであり、権威の飾りではない。一次資料の引用は、本文まで読んだ範囲でのみ構造と条件を語り、要旨だけ読んだ範囲では見出しの事実留める。公式文書と手順書の引用は、告知と配置の範囲に留め、他構成への転用をしない。実装記録の引用は、存在と手順の証拠に留め、正確性の裏書きにはしない。提供元の数値の引用は、測定条件付きの主張として隔離し、事実としては引用しない。受容記録の引用は、配置依存の観察として扱い、技術的事実の根拠にはしない。引用の後に残る未解決は、境界として明示する。引用は主張を閉じるためではなく、読み手が確かめに戻れるようにするためにある。

読者への道案内

時間の限られた読者は、第 1 節の契約表と各節末の境界ボックスだけを追っても、本巻の主張の骨格をつかめる。各節の用語は第 1 節の区別表に準拠する。総量と有効量、重みとキャッシュ、トークン単価と仕事単価の三つの分離が、各節で繰り返し使われる合言葉である。実装の詳細よりも設計思想の違いに関心のある読者は、第 2 節の系譜表、第 3 節の節約位置表、第 8 節の比較表を中心に読むとよい。運用と測定の実務に関心のある読者は、第 6 節の無駄と処方表、第 7 節の設計表、第 9 節の作法表が近道になる。巻末の技術要点表は、各節の主張と境界を一次資料の裏付けとともに圧縮したものであり、本文の詳述に戻る際の索引として使える。巻末の用語集は、本巻で使う技術用語の定義を集めたものであり、節をまたいで同じ言葉が同じ意味で使われていることを確かめるために置いた。

節	問い	要点
第 1 節	何を減らすのか	削減対象の名指しと律速の見分け方
第 2 節	どこを動かすのか	条件付き計算の系譜と割当・配置の区別
第 3 節	何を保持するのか	注意と記憶の節約位置と置き場所の設計
第 4 節	何ビットで持つのか	対象と時点の分離と器・手法の区別
第 5 節	どう待たずに進めるのか	受容率の物差しと予算表の運用
第 6 節	どう捌くのか	記憶管理と束ね方と受け渡しの経済学
第 7 節	毎回呼ぶ必要があるか	閾値の運用と適合の監視と特化の居場所
第 8 節	2026 年の到達点は何か	順位付けなき機構の対照と証拠の深さ
第 9 節	数字をどう読むのか	条件をそろえる作法と総費用の視点
結び	何を持ち帰るのか	層の要点と指針と診断の実践

目次

1	「効率」とは何を減らすことなのか	6
1.1	学習の物差しとしての規模則とその見直し	6
1.2	推論の詰まりを分けて見る	7
1.3	総量と有効量の分離が効く場面	7
1.4	測定条件が変われば律速が変わる	8
1.5	重みとキャッシュの見積もりを分ける実践	9
2	Scaling から Conditional Compute へ	10
2.1	振り分けの設計問題	11
2.2	細分化と共有化、そして稼働量の経済学	11
2.3	学習を支える土台	11
2.4	学習の割当としてのデータ組成	12
2.5	載せるための技術と削る技術の区別	12
2.6	補助損失と負荷分散の設計感覚	13
2.7	常駐と待避の予告	13
3	Attention と Memory を減らす	14
3.1	疎化と局所化	15
3.2	回帰と線形注意の系譜	15
3.3	保持を削る・使い回す・深さを絞る	15
3.4	共有化の系譜をもう一度たどる	16
3.5	疎化の二つの顔	16
3.6	記憶の三つの置き場所	17
3.7	潜在共有という第三の道	18
3.8	窓の再生と層の不均一性	18
4	Bit を減らし、巨大モデルを手元で動かす	19
4.1	同時量子化の二つの道	20
4.2	器と削り方を分ける	20
4.3	浮動小数点の低ビット化	20
4.4	適応と原生低ビット	20
4.5	総量と作動量の落差	21
4.6	外れ値と重要度の二つの見方	21
4.7	形式の標準化がもたらしたもの	21
4.8	低ビットの三つの時間軸	22
4.9	較正データの集め方と落とし穴	23
4.10	削り方の選び方	23
4.11	手元実行の可否をどう見極めるか	24
5	1 token ずつ待たない	25
5.1	先読みの質を上げる	25
5.2	足し算の効率	26
5.3	引き算の効率	26
5.4	受容率という単一の物差し	26
5.5	学習の改善としての先読み	27

5.6	予算表で考える	27
5.7	多頭草案から特徴量草案へ	28
5.8	運用に載せる先読み	28
6	Serving で消える無駄	29
6.1	重なりの使い回し	30
6.2	段階の分離	30
6.3	核と通信と基盤	30
6.4	待ち行列の約束を守る	31
6.5	受け渡しの経済学	31
6.6	文書と実装の対応	31
6.7	同時実行と約束の両立	32
6.8	置き場所の選び方	32
6.9	切り幅の調整つまみ	33
6.10	優先度と割り込みの設計	33
7	「大きなモデルを毎回呼ぶ」必要はあるか	34
7.1	学習型の振り分けとその脆さ	34
7.2	生成しない判断という特化策	35
7.3	契約面の読み方	35
7.4	閾値の調整が設計の核心である	35
7.5	確信度との付き合い方	36
7.6	前段のさばき役としての居場所	36
7.7	料金表の読み方	37
7.8	分布が動く運用へ	38
8	2026 年の実装点	39
8.1	大規模疎活性化と記憶圧縮を重ねた設計	40
8.2	残差と最適化と疎注意を共同設計した構成	40
8.3	線形注意系の実行可能性	40
8.4	要旨範囲に限定される四つ目の実装点	40
8.5	作動量の設計思想の違い	41
8.6	注意と記憶の選択の違い	42
8.7	証拠の深さの違いと読み方	42
8.8	DeepSeek 系 Flash の読み方	43
8.9	Qwen 系 Flash-Next の読み方	44
8.10	Kimi 系 Linear の読み方	44
8.11	GLM 系 Flash の読み方	45
9	数字をどう読むか	46
9.1	実課題の系譜を読む	47
9.2	長さや報告の構え	47
9.3	物差しの持ち帰り	49
9.4	混入への備えを読む	49
10	結び——層として読む効率	50
10.1	失敗の診断学	51
10.2	層の読み方の訓練	51
10.3	立場別の指針	52

1 「効率」とは何を減らすことなのか

BOTTLENECK CONTRACT

効率が良いという言葉は、大規模言語モデルの文脈では、それだけでは何も約束しない。学習に要する計算量を減らしたのか、配備後の推論の応答を速くしたのか、同じ費用でさばける要求数を増やしたのか。これらを区別せずに手法だけを並べれば、後のあらゆる比較は意味を失う。本稿が最初に結ぶのは、その混乱を避けるための契約、すなわちどこが詰まるのかを見定める見取り図である。[1, 2, 3]

出発点は学習と推論の分離である。学習は一度きりの巨大な投資であり、モデル規模と学習データ量と投下する計算量の配分が問われる。損失をどこまで下げるかが中心であり、使った資源は一度支払えば済む。これに対して推論は、配備後に繰り返し発生し続ける継続費用であり、使われ方によって効き方が変わる。同じ速さであっても、ひとつの応答が返るまでの待ち時間である遅延と、単位時間あたりに処理できる要求量であるスループットは別の軸である。対話の応答性を高めることと、同時接続の捌き量を増やすことは、同じ工夫では解けない場合が多い。さらに、トークンあたりの単価を下げたことと、ひとつの仕事を終えるまでの総費用を下げたことも等しくない。短い生成を安く大量に回せたとしても、長い文脈の読解や複数段階の推論を要する仕事の費用が下がらなければ、利用者にとっての効率は改善しない。

計算か記憶容量か帯域か通信か、総パラメータか有効に働く部分か、固定資産としての重みか文脈に応じて膨らむ生成時の中間記憶か。総量の大きさと実際に働く有効部分の大きさを混同せず、固定資産と変動費を分けて考える習慣が、以降の節約策を見極める土台になる。この契約なくして高速化や軽量化を語れば、遅延の改善か捌き量の改善か、固定費の削減か変動費の削減かが曖昧になり、読み手は判断を誤る。削減対象が違えば選ぶべき節約策も評価の物差しも変わるという当然の事実から出発したい。何を減らすのかを名指しすることこそ、本特集の効率論の出発点であり、以降の手法を読み解く際の共通言語なのである。

1.1 学習の物差しとしての規模則とその見直し

学習側の物差しとして長く参照されてきたのは、言語モデルの損失がモデル規模とデータ量と計算量に対してべき乗則に従って滑らかに減少するという見方である [1]。ここで重要なのは係数の具体値ではなく、大きくすれば損失が下がるという関係の形そのものである。この形が、濃密なモデルを前提とした規模拡大の議論の土台になり、研究開発の資源配分を正当化してきた。まず大きなモデルを作り、後から小さく使うという発想も、この方向性の共有に支えられている。しかしこの関係は、あくまで学習時の損失に関する見取り図であり、配備後の推論費用の内訳を解消するものではない。規模だけを信奉すれば、保持すべき重みは増え、それを読み出す帯域の負担も増し、複数装置にまたがる通信の重さも増す。演算器の処理能力と、記憶容量と、読み出しの帯域と、装置間の通信は、それぞれ異なる局面で律速になる。損失の低下と引き換えに配備の重さが増すという緊張関係がここに生まれる。本稿が依拠する範囲では、係数の具体値や実験条件の細部には立ち入らず、べき乗則という関係の形を取り出すにとどめる。

学習の資源配分を問い直したのは、従来大型モデルは投じた規模に見合う学習トークンを使い切れておらず、過小学習の状態にあったという指摘である [2]。器だけを大きくし、中身の学習が追いついていない状態では、投下した計算量が見合いの損失低下に結びつかない。そこでモデル規模と学習トークン数をほぼ均等に伸ばす配分が代表的な処方箋として示され、パラメータあたり約二〇トークンという目安が広く知られるようになった。肝心なのは、この数値を普遍の定数として扱わないことである。本稿が依拠する範囲では、それは特定の条件のもとで導かれた見出しとしての提案であり、条件や構成が変われば望ましい配分も変わり得るという留保つきで受け止める。むしろ要点は、効率の問いがモデルの大きさだけでなく、データの割り当て方へと移った点にある。同じ計算量を使うなら、巨大な器を不十分なデータで鍛えるよりも、規模とデータの釣り合いを取り直す方が損失の低減に資

するという視点転換である [2]。

規模則の再適合にも触れておく。モデルの構成が変われば、損失の振る舞いも変わり得る。密なモデルで導かれた目安を、疎なモデルや線形記憶のモデルにそのまま持ち込むことはできない。構成別の規模則の当てはめ直しは、配分の処方箋を構成ごとに作り直す作業であり、目安の数値を普遍化しないための手続きである。本稿が依拠する範囲では、再適合の数式や曲線の細部には立ち入らず、目安は構成依存であるという留保だけを受け取る。巨大モデルの系列で再適合が試みられていることは、配分の問いが一度きりの処方ではなく、構成の変化に追従する反復作業であることを示す。目安を使うときは、どの構成で導かれた目安なのかを確かめる。この確認が、規模則の誤用を防ぐ。総量と有効量を区別し、トークンあたりの単価ではなく、ひとつの仕事を終えるまでの費用で考える習慣がここから要請される。

1.2 推論の詰まりを分けて見る

推論側の詰まりを可視化したのは、生成過程を一括処理から反復単位の細粒度な管理へと変えた発想である [3]。生成は、入力全体を一気に読み込んで内部状態を構築する前処理であるプリフィル段階と、トークンをひとつずつ逐次的に生み出す復号であるデコード段階に分かれる。前者は並列に処理しやすく演算が律速になりやすいのに対し、後者は直前の結果に依存して逐次に進むため、重みの読み出しや蓄積された中間記憶の保持と読み出しが律速になりやすい。従来のように要求をひとまとまりとして最後まで占有させる方式では、長い生成が短い要求を待たせ、遅延とスループットの両方が損なわれる。そこで導入されたのが、反復のたびに待ち行列を見直し、段階の異なる要求をきめ細かく詰め合わせる継続的なスケジューリングの考え方である。分散した装置群の上でも反復単位の管理を徹底する発想は、その後のスケジューラ設計の起点になる考え方であり、段階の分離が運用の柔軟性を生むことを示した。

ここで効く区別が、重みと中間記憶の違いである。重みはモデルに固有の固定資産であり記憶容量を占有する。一方、逐次生成に伴って蓄積されるキーとバリューの中間記憶である KV キャッシュは、文脈の長さや同時処理数に応じて膨らむ変動費であり、容量と帯域の両面から圧迫する。計算能力と記憶容量と帯域と通信のどれが詰まっているのかを名指しし、遅延と

スループットのどちらを改善したいのかを明確にすること。それが推論の効率化の前提であり、手法の評価軸を定める作業にほかならない。本稿では、反復単位でプリフィルとデコードを分けて扱うという設計思想の水準にとどめ、具体的な処理手順の細部や数値化された高速化の幅には踏み込まない。

1.3 総量と有効量の分離が効く場面

総パラメータ量と有効パラメータ量の区別は、学習の議論と推論の議論を切り分けるための基本的な物差しである。総パラメータ量はモデルが保持する重みの全体規模を表し、保存に必要な容量や読み込みの対象となる量を決める。一方、有効パラメータ量は一つのトークンを処理する際に実際に計算へ寄与する部分の規模を表し、一トークンあたりの計算量の見通しを与える。この二つを分けずに「大きいモデル」「小さいモデル」と呼ぶと、保存や転送の重さの話と計算の重さの話が混ざり、どの対策が効くのかが見えなくなる。総量を問題にするのか有効量を問題にするのかを先に宣言すること自体が、本節の用語の使い方の核心である。

この分離がとくに効くのは、重みの大きさとキャッシュの大きさを別々に考えたい場面である。重みは入力によらず固定された量として存在し、モデルの総量によって決まる。これに対してキーとバリューのキャッシュは、処理する文脈の長さや同時に抱える要求の数に応じて増減する。短い応答を一つだけ返す場合には重みの読み出しが目立ち、長い文脈を保ったまま多くの要求を並行してさばく場合にはキャッシュの置き場と読み書きが目立つようになる。同じモデルを使っても、利用の形が変われば重みとキャッシュのどちらが先に苦しくなるかは変わる。だからこそ、重みの量とキャッシュの量を一つの「メモリ使用量」という言葉に丸めず、総量に由来する分と利用条件に由来する分に分けて呼ぶことが重要になる。

トークンあたりの費用と課題あたりの費用の区別も同じ発想である。一トークンを処理する費用が下がっても、課題を解くために必要なトークンが増えれば、課題全体の費用は下がらない。たとえば中間的な推論過程を長く書き出す手法や、複数回の試行から良いものを選ぶ手法では、一回ごとの生成は軽く見えても、課題を解き終えるまでに消費するトークンの総量は増える。逆に、一トークンあたりの費用が多少上がっても、必要なトークンが十分に減れば課題全体では安く

なる場合がある。効率の主張を読むときは、分母がトークンなのか課題なのかを必ず確かめる必要がある。分母の取り違えは、軽量化の効果の過大評価にも過小評価にも直結する。

規模則の議論は、この分母の問題を学習側から照らす。言語モデルの損失がモデル規模やデータ量や計算量に対して滑らかなべき乗則に従うという見方は、闇雲な拡大ではなく配分を考える土台を与えた [1]。その後の見直しは、過大なモデルに従来より多くの学習トークンを割り当てる配分が学習効率の観点で有利になり得ることを示し、効率の問題をモデルの大きさの問題からデータ配分の問題へと広げた [2]。ここでいう有利さは学習計算の配分に関する主張であり、推論時の重みの重さやキャッシュの重さまで含めた総合判断ではない。学習の節で使う「効率」と推論の節で使う「効率」の分母が違うことを、読者はここで予告として受け取ってほしい。総量と有効量、重みとキャッシュ、トークンと課題という三つの分離は、その予告を具体的な言葉に直したものである。

1.4 測定条件が変われば律速が変わる

同じモデルと同じ実装を使っても、測定条件が変われば律速は変わる。律速とは、全体の速さや費用を決めている支配的な要因のことであり、計算の速さ、保持できる量、読み書きの速さ、並列実行に伴う通信の四つに大別できる。効率化の手法は、この四つのうちどれを緩和するかが違えば、効く条件も違ってくる。手法の名前だけを見て優劣を決めるのではなく、どの律速を緩める手法なのかを問う習慣が、本特集を通じた読み方の約束になる。

最も分かりやすい条件の違いは、一度に処理する量である。一つの要求への応答の速さを問題にする場合、待ち時間の短さが評価の中心になる。利用者は最初の文字が出るまでの速さと、その後の続きが出る滑らかさを感じる。この条件では、装置を十分に使えていない状態での無駄や、処理の切れ目での待ちが律速になりやすい。一方、多くの要求をさばく処理能力を問題にする場合、単位時間に完了する要求の数が評価の中心になる。この条件では、複数の要求を束ねて装置の稼働率を上げる工夫が効き、束ねた分だけキャッシュの置き場や読み書きの負担が増える。待ち時間の話と処理能力の話は、同じ装置の同じモデルでも結論が逆になることがあるため、分けて記録しなければならない。

文脈の長さも律速を動かす。入力を一気に読み取る段階では、まとまった計算を一括して実行できるため計算の能力が問われやすい。これに対して出力を一語ずつ生成する段階では、その都度重みとこれまでのキャッシュを読み出す必要があり、読み書きの帯域が問われやすい。文脈が短い中は重みの読み出しが相対的に目立ち、文脈が長くなるにつれてキャッシュの読み書きの比重が高まる。この段階の違いに着目し、入力読み取りと逐次生成のスケジューリングを分けたことが、生成モデル向けサービングの出発点にある [3]。前処理と生成を同じ待ち行列で一括して扱うか、段階ごとに分けて扱うかという設計の違いは、待ち時間と処理能力のどちらを優先するかに直結する。

同時実行の数と装置構成も同様に律速を動かす。同時に抱える要求が増えれば、キャッシュの総量は膨らみ、保持できる量が先に上限になることがある。複数の装置に分けると、一台あたりの保持量は緩和されるが、装置間の通信が新たな律速として現れる。帯域の広い装置に替えれば読み書きの律速は緩むが、計算の能力が追いつかなければ今度は計算が律速になる。つまり律速はモデルの属性ではなく、モデルと利用条件と装置の組み合わせの属性である。効率の比較は、束ねる量や同時実行数や文脈長や装置の別をそろえたうえでなければ意味を持たない。条件を明示しない比較は、本特集では成立しない比較として扱う。

後の学習効率の議論では、総パラメータ量と学習トークンの配分、計算量の使い方といった言葉が中心になる。損失の縮小を目的とする学習の場面では、重み全体の規模とデータ配分の関係が問われ、推論時のキャッシュや同時実行の話は背景に退く。これに対して後の推論基盤の議論では、有効パラメータ量、重みとキャッシュの分離、入力読み取りと逐次生成の区別、待ち時間と処理能力の区別が中心になる。軽量化や量子化の議論では、重みの保持量と読み書きの帯域、計算の簡略化の関係が中心になる。同じ「効率」という語が節ごとに違う分母と違う律速を指すことを、読者は各節の冒頭で確かめながら読み進めてほしい。

律速の見分け方

応答が遅いときはまず段階を問う。出だしが遅いなら読み込みと帯域、続きがもたつくなら復号と帯域、混雑時に崩れるなら束ね方と容量を疑う。

重いのが重みなら総量と精度、重いのが文脈なら保持と同時実行、重いのが受け渡しなら配置と転送を疑う。速くしたいのが応答なら遅延、捌き量なら処理能力、費用なら仕事単価の欄を見る。症状から欄を特定し、欄に対応する節を読むことが、契約の実践である。

見積もりの順序

重みの見積もりでは、総量と精度の組み合わせを確認し、静的な確保量を定める。キャッシュの見積もりでは、文脈長と同時実行数の想定を確認し、動的な蓄積量を見積もる。費用の見積もりでは、トークン単価ではなく仕事単価で総額を試算する。三つの見積もりを別々に立て、合算して判断する。合算の前に各欄の前提を書き出し、前提が変わったときの崩れ方を見通す。見積もりは一度きりの計算ではなく、前提の変化に合わせた更新作業である。

欄の対応表の使い方

区別表は、暗記するための表ではなく、問いを立てるための表である。新しい手法に出会ったら、まず区別表の行を指さし、どの欄の軽さなのかを宣言する。次に、軽くなった欄と変わっていない欄を書き分ける。次に、評価条件の欄を確認し、単一系列か複数系列か、短文脈か長文脈か、単発か同時実行かを記録する。三つの手順を踏めば、手法の主張は欄の言葉に翻訳され、比較可能な形になる。区別表は、本巻全体の索引である。

1.5 重みとキャッシュの見積もりを分ける実践

効率の議論がかみ合わなくなる原因の多くは、重みの大きさとキャッシュの大きさを一つの大きさとして語ってしまうことにある。重みは置くか置かないかの世界であり、キャッシュはどれだけ溜めるかとどれだけ並べるかの世界である。この区別を最初に固定することが、以降の比較を意味あるものにするための契約になる。

重みの占有量は静的な量である。総パラメータ量と精度の組み合わせによって決まり、要求が来る前にはほぼ確定する。文脈を長くしても短くしても、同時実行を増やしても減らしても、この本体の大きさ自体は変わらない。学習時の規模拡大則の古典的な処方、損

失が模型規模やデータや計算量に対して滑らかに改善するという見通しを与え、密な模型を基準として考える土台を作った [1]。そこでは大きくすればよくなるという方向性が示されるが、推論時に本体をどこに置くかという問題は解かれない。規模の話と常駐の話と同じ重さで受け取らないことが大切である。

学習配分の見直しは、この静的な見方をさらに精密にする。大きな模型を十分に学習させていなかったのではないかという指摘は、模型規模と学習トークンの釣り合いを取り直す処方として理解できる [2]。ただし、この処方は学習の配分についての主張であり、推論時の常駐量を直接減らす主張ではない。学習が軽くなったという話を、そのまま推論の常駐が軽くなったという話として受け取ってしまうことが、読者が混同しやすい落とし穴である。前者は作るとき配分の話であり、後者は置くときの確保の話である。両者を別の欄に書き分けるだけで、多くの誤読は避けられる。

これに対して、キーとバリューのキャッシュの占有量は動的な量である。文脈が長くなれば蓄積が増え、同時実行が増えれば系列ごとの蓄積が積み上がる。ここで効いてくるのが、生成の前半と後半を分けて考える視点である。反復単位で細かくスケジューリングし、読み込み段階と生成段階を分けて捌くという運用の発想は、その後のスケジューラ設計の祖型になった [3]。

実践の要点は、主張の主語を取り違えないことである。ある手法が軽いと述べているとき、読者はまずそれが重みを軽くしているのかキャッシュを軽くしているのかを問う。さらに、その評価がどのような同時実行を仮定しているのかを問う。単一の系列で測られた軽さと、多数の系列を並べたときの軽さは別のものである。典型的な誤読は、活性化の疎さと常駐の軽さを混同することである。一語あたりに働く部分が小さいことは、その語の計算が軽いことを意味するが、本体全体を置かなくてよいことを意味しない。総量と活性量は別の欄である。同様に、トークンあたりの費用と課題あたりの費用も別の欄である。

契約の使い方チェックリスト

本体の確認——静的に確保すべき量は何で決まるかを問う。模型の選び方と精度の扱いが主語であり、文脈の長さや並べ方は主語ではないことを確かめる。蓄積の確認——動的に増える量は何に連

動するかを問う。文脈の長さ同時実行数のどちらが効いているのかを確かめる。段階の確認——読み込み段階の負荷と生成段階の負荷のどちらを論じているのかを問う。両者を混ぜた平均値で語られていないかを確かめる。並べ方の確認——単一の系列の話か複数の系列を並べたときの話かを問う。費用の確認——語られている軽さが重みの軽さか蓄積の軽さか計算の軽さかを問う。軽くなった欄と変わっていない欄を書き分けて受け取れることを確かめる。

区別の軸	両側の意味	本巻での使い方
学習と推論	一度きりの投資と繰り返しの継続費用	規模則・配分の議論と運用の議論を混同しない
ブリフィルとデコード	並列に読む段階と逐次に生む段階	演算律速と記憶帯域律速を分けて読む
計算と容量・帯域・通信	演算器・置き場所・読み出し・受け渡し	律速の所在を名指しする
遅延とスループット	応答の速さと捌き量	どちらを改善したのかを明示する
総量と有効量	総パラメータと稼働パラメータ	大きさの印象を稼働の実態で割り引く
重みとKV キャッシュ	固定資産と変動費	容量見積もりを分けて立てる
トークン単価と仕事単価	単位の安さと完遂の費用	費用の物差しを完遂側に置く

[1, 2, 3]

以上の見取り図は、あえて限定された範囲の主張だけに依拠している。規模と損失の関係については関係の形を取り出すにとどめ、学習トークンの配分については代表的な目安を見出しとして紹介するにとどめ、生成の管理方式については設計思想の水準にとどめる。異なる条件で測られた性能値を直接並べて優劣を

論じること、本稿では行わない。本稿が約束するのは削減対象の名指しである。以降の各論は、この物差しに沿って読まれることを前提とする。適用範囲を超えた一般化を避け、条件付きの主張は条件付きのまま受け止めること。それが本特集の効率論を読み解く際の作法であり、本稿の守備範囲の宣言である。

本節の読解上の境界

規模則の係数値・実験条件、配分目安の普遍化、処理手順の細部と速度幅には立ち入らない。異なる土俵の数値を並べた速さ比べは行わない。学習の物差しを推論の評価に流用しない。

2 Scaling から Conditional Compute へ

CONDITIONAL COMPUTE

言語モデルの規模拡大は、パラメータとデータと計算をそろえて増やすほど性能が伸びるという経験則に支えられてきた。緻密なモデルではすべてのトークンがすべてのパラメータの計算を負担するため、能力を上げようとすればそのまま計算量と費用が膨らみ、記憶容量や装置間通信の壁にも早くぶつかる。この行き詰まりをほどこいたのが、容量と計算を切り離す条件付き計算の発想である。学習されたゲーティングがトー

クンごとにごく一部の専門家だけを選び出し、残りは休ませることで、総パラメータ数を増やしてもトークンあたりの計算を抑えられる [4]。均等に割り振るための補助損失が添えられ、特定の専門家への偏りを抑える仕組みも一体で示されたことが重要である。ここでは細かな数式や長期系列での実験表には立ち入らず、仕組みの要点だけを調査水準で受け取る。

この着想を巨大規模で現実にする方向へ進めたの

が、専門家並列と自動分割の系統である。専門家を複数装置に分散配置し、トークンを振り分けて処理することで、単一装置の記憶容量を超える規模が扱えるようになった [5]。振り分け先があふれた場合の容量係数による扱いや、分割をコンパイラが管理可能にする工夫が、後の兆級モデルの学習基盤の前提となった。翻訳ベンチマークの細部や分割注記の全体には踏み込まず、疎な拡大を可能にした体制側の前進として押さえる。総量と稼働量を分けて考える習慣もここに始まり、後の稼働量と常駐量の経済学へとつながる。同時代の実装点同士の優劣比較はここでは行わず、系譜と割当の仕組みに集中する。

2.1 振り分けの設計問題

疎な層をどう振り分けるかは、質と通信と運用の三つ巴の設計問題になった。振り分けを一人だけに絞る方式は、仕組みを単純化し、装置間のやり取りと割り当ての管理を軽くする道を開いた [6]。トークンあたり一人か複数かという選択は、以降の細分化や共有専門家を評価する際の基準となり、方式の系譜を整理する物差しとして残った。ここでは規模拡大曲線や多言語の評価表には踏み込まず、単純化が何を節約し何を手放したかという交換関係だけを調査水準で受け取る。対照的に、専門家側がトークンを選ぶ方式は、視点を反転させたもう一つの到達点である [7]。各専門家が受け持てる量を固定し、トークンごとに選ばれる数が変わるため、容量の管理が明確になり、負荷の偏りへの向き合い方が変わる。事前学習の高速化の定量分析には立ち入らず、方式の違いが通信量や待避や常駐のどこに効くかという水準で整理する。いずれも万能ではなく、装置構成や系列長や運用条件で損得が変わる点に注意が要る。

2.2 細分化と共有化、そして稼働量の経済学

専門家を細かく砕き、共通の知識を担う共有専門家を添える設計は、条件付き計算の使い方を一段と経済的にした [8]。細分化された専門家が専門語彙のように振る舞い、汎用的な部分は共有部が引き受けることで、組み合わせの表現力と通信の節度を両立させる狙いがある。装置をまたぐ転送を抑えるため、振り分け先の装置数を絞る工夫も一体で示された。ここでは専門家粒度の走査などの詳細な比較表には踏み込まず、抽象水準の仕組みとして受け取る。この機構を受け継いだ分岐点として、注意機構の圧縮と疎な計算を同時

に前面に出した大規模モデルが現れた [9]。総量に対して稼働量を抑える姿で示され、鍵と値の保持を圧縮する機構と疎な演算を組み合わせることで、記憶と計算の双方に手を打つ姿勢が明確になった。定式化の全体や評価表の数値には立ち入らず、確立した水準の構成的事実として押さえる。

総量と稼働量を分けて見る習慣が定着すると、巨大さの印象は大きく変わる。開かれた重みで広く使われた方式は、8x7B 構成でトークンごとに二つの専門家を選び、稼働量を抑える姿として確立した水準で知られた [10]。この公開を境に、専門家の常駐や待避、振り分けの上乗せ費用といった運用の論点が共有の課題となり、巨大な重みを手元でどう保持し、必要な分だけをどう載せ替えるかという観点が効率の議論に定着した。計算の節約は記憶の負担と引き換えであり、動かさない重みもどこかに置かねばならず、通信と保持の費用が残る。単純な大小比較は意味を持たない。

2.3 学習を支える土台

疎な大規模化が現実の学習として回るには、低精度演算の土台が欠かせない。総量 671B の混合専門家モデルで、混合精度による大規模学習が実生産として示されたことは、その結節点である [11]。行列演算庫の側では、細粒度の低精度演算を担う実装がこの学習の片翼となり、形式の工夫が大規模学習の実行可能性に直結することが示された [12]。同じ系譜では、複数トークンの予測を学習目的に取り込む試みも示され、後の推論高速化の起点として位置づけられる。あくまで目的関数の由来として押さえ、速度の定量主張には踏み込まない。

巨大モデルを学習可能にする分割と配置の工夫は、計算を減らす技術ではなく、載せて動かすための技術として区別して理解したい。層内を複数装置に割り付ける並列化は、より大きな模型を収めるための方向であり [13]、最適化状態や勾配や重みの重複を排して学習可能な規模を伸ばす記憶最適化も、収めるための方向として整理される [14]。本特集では配置と割当の文脈に限定して扱い、分散学習の深掘りにはしない。最適化手法の側では、勾配更新に直交化を用いる実装が参照対象となり、学習の安定や高い学習率の許容が報告として伝えられた [15]。ただし参照先は論文ではなく実装であり、収束定理の根拠として扱わない。

量を積む方向と並んで、何をどれだけ混ぜるかという構成の方向が、近年の学習効率のもう一つの柱に

なった。小規模な代理模型で領域ごとの重みを定め、下流課題の知識なしに混合比を決める試みは、量の拡大を補う構成の工夫として位置づけられる [16]。知識の蒸留は、教師の柔らかな出力を手がかりに小規模模型へ能力を受け渡す古典的な出発点であり [17]、低ランク適応は事前学習済みの重みを凍結し低ランクの行列だけを学ぶことで微調整の記憶と保存を軽くする手法として知られる [18]。いずれも本特集では深掘りではなく、系譜の指し示しとして配置する。効率を一枚岩で論じず、局面ごとの費用として分けて考える習慣がここで整い、次の記憶と復号の議論へとつながる。

2.4 学習の割当としてのデータ組成

規模則の議論は計算量とデータ量の量的配分に目を向けがちであるが、何を混ぜて学ばせるかという組成の設計もまた割当の問題である。混合比の設計は、トークン選択型の疎なゲーティングが計算と容量を切り離したのと同じように対称的に [4]、データ側の計算と容量の切り離しを担う。すなわち、同じトークン予算でも組成が違えば到達する能力分布が変わり、疎なモデルにおける専門家の分化のさせ方も変わる。この意味でデータ組成は前処理の職人芸ではなく、条件付き計算と並ぶもう一つの割当機構である。

この割当を原理的に扱おうとした試みが、領域別の混合重みを事前に設計する考え方である。小規模な参照模型を用いて領域上の頑健な目的を最適化し、下流課題の知識なしに混合比を定めてから大規模な本学習に移す手順は、量の拡大を補う組成の拡大と理解できる [16]。どの領域に重みを置くかは、どの専門家を育てるかという問いと間接的に連動する。共通知識を担う共有専門家と細分化された専門家の振り分けが、粗い混合比の選択によって歪むこともあれば、整えられることもあるからである [8]。組成設計はしたがって、振り分けの経済学の上流にある割当問題として読むべきである。

蒸留は、この組成の議論を容量と品質の交換関係へと接続する。教師の柔らかな出力を目標として学ぶ仕組みは、硬い正解ラベルだけでは伝わらない分布の形状を移すことで、小さな容量でも高い品質に到達する道を開いた [17]。疎な大規模モデルが総パラメータと作動パラメータを切り離すのに対し、蒸留は学習済みの容量と転移可能な品質を切り離す。開かれた重みとして公開された疎モデルが下流の常駐や振り分けの関心と呼び起こしたように [10]、蒸留された小さな模型

は配置と推論の関心を引き寄せる。いずれも、大きな容量をどこに置き、どの部分だけを作動させるかという同じ経済の別表現である。

低ランク適応は、この系列の末端を微調整の効率として締めくくる。事前学習済みの重みを凍結し、学習可能な低ランク分解行列だけを挿入する方式は、微調整の記憶量と保存量を切り詰める [18]。ここで節約されているのは事前学習の計算ではなく、適応のための常駐と更新の対象範囲である。条件付き計算がトークンごとの作動範囲を絞るのに対し、低ランク適応は課題ごとの更新範囲を絞る。事前学習における混合比の設計と、適応における更新範囲の設計が、予算配分の上下流として対応していることが見えてくる。

2.5 載せるための技術と削る技術の区別

大規模化の技術はしばしば一括りに節約技術と呼ばれがちだが、載せるための技術と削るための技術は区別しなければならない。載せるための技術は、より大きなモデルを学習可能にするための配置と並列化の前提であり、それ自体は演算量を減らさない。疎モデルの系譜でいえば、自動分割による専門家並列と振り分けの仕組みが巨大化の前提となり [5]、簡素化された経路選択が通信と振り分けを軽くし [6]、細粒度化と共有専門家が通信制約のもとでの専門化を進めた [8]。これらはいずれも配置と作動の設計であり、削減そのものではない。

層内並列は、載せるための技術の典型である。層内部のテンソルを行列並列に分割し、最小限の通信操作で結合する方式は、単一装置の記憶容量を超える規模の学習を可能にする [13]。並列化が進んでも学習の総演算は減らず、むしろ通信という新たな費用が表面化する。疎モデルではこの通信費用が専門家の振り分けと常駐の問題として現れる。載せることは削ることの対立物ではなく、削るべき対象を明確化する前提である。

冗長性の排除は、載せるためのもう一つの柱であり、並列化とは異なる方向から記憶の問題を解く。最適化器の状態や勾配やパラメータが装置間で重複保持されがちであり、この重複が規模の上限を決める。重複を排除するためにそれらを分割配置し、装置数に応じて学習可能な規模を伸ばす方式は、適合のための技術として位置づけられる [14]。ここでも演算そのものは減っていない。減っているのは重複であり、増えているのは利用可能な総記憶である。冗長性排除は総パ

ラメータを置く場所を確保する技術であり、条件付き計算は置いた総量のうち作動させる部分を絞る技術であることがわかる。前者がなければ後者の経済は成立しない。

直交化更新の考え方は、これら二つとは層が異なる。勾配の更新に行列の直交化操作を適用し、学習の安定性を高めてより高い学習率を許すとされる方式は、配置ではなく更新則の設計に属する [15]。参照実装として共有された実装体に権威の所在があり、収束定理の供給源として読むべきものではない。更新則が安定化しても、専門家の常駐量や振り分け通信の総量が消えるわけではない。むしろ安定した学習が可能になることで、より細粒度な専門化への挑戦が可能になり、配置設計への要求がいっそう明確になる。

この三つを並べると、通信と常駐の見通しが自然に開ける。載せる技術が整うほど、残る律速は演算ではなく移動と滞在になる。専門家並列における振り分けの往復、容量超過時の扱い、装置制約のもとでの経路選択はいずれも移動の設計であり [5][8]、総量に対する作動量の比率や圧縮された注意機構との組合せは滞在の設計である [9]。微細化された量子化による行列積の土台が大規模な低精度学習を支えるように [11][12]、通信と常駐の土台がなければ条件付き計算の節約分は目減りする。節約は削った量ではなく、載せたうえで作動させずに済ませた量として初めて計測できる。次節以降の推論効率の議論は、この計測の作法を引き継ぐ。

2.6 補助損失と負荷分散の設計感覚

疎な混合専門家モデルの設計感覚は、能力と計算を切り離すところから始まる。すべての専門家を毎回働かせるのではなく、一語ごとに働く専門家を選び出すことで、保持する知識の総量と一語あたりの計算量を別の欄で大きくできる。この選び出しを学習可能にした初期の定式化は、疎なゲーティングと負荷を均すための補助的な損失を組み合わせたものであり、後のあらゆる選択方式の祖型になった [4]。補助損失の役割は、選択を強制することではなく、選択の偏りを穏やかにいさめることにある。完全に自由に選ばせれば、学びやすい一部の専門家に処理が集中し、残りが遊んでしまう。逆に均等を強く強制すれば、どの専門家も似たような振る舞いになり、専門化の旨味が薄れる。設計者はこの間で舵を取る。

偏りと効率の折り合いは、運用の言葉で考えると見

通しがよい。偏りが大きいと、一部の専門家に処理が殺到して溢れが生じ、他の専門家は待機する。計算資源はあるのに仕事の配分が悪く、全体の効率が上がらない。このむらを抑えるために補助損失を強めると、配分は平らになるが、専門家の個性が育ちにくくなる。どちらが正しいかという問いではなく、どの程度のむらを許し、どの程度の個性を育てるかという加減の問いである。

選択を単純化する方向は、この折り合いに対する一つの答えである。一語ごとに選ぶ専門家の数を絞り、受け渡しを簡素にすることで、通信と振り分けの負担を下げられる [6]。表現の幅は狭まるが、実装と運用の見通しはよくなる。容量の見方を反転させたのが、専門家側から処理を選ぶ発想である。各専門家が受け持てる容量を固定し、専門家が自分の得意な処理を選び取ることで、専門家ごとの仕事量は平らになる [7]。代わりに、一語が何人の専門家に診てもらえるかが文脈によって変わり、処理ごとの計算量が一定ではなくなる。この可変性を受け入れるかどうかが設計の分かれ目である。

細粒度化と共有化は、専門化の中身に対する工夫である。専門家を細かく刻んで専門語彙を増やし、共通知識を受け持つ共有の専門家を別に設けることで、分担の質を上げようとする考え方がある [8]。共通の言い回しや一般的な知識は共有側で受け止め、分野固有の知識は細かい専門家側で分担する。この設計は、単に数を増やす話ではなく、何を共有し何を分けるかという分担の設計の話である。読者は、負荷を均す仕掛けがどの層に置かれているのか、共有と専門の分担がどのように切られているのかを、順に確かめるとよい。

2.7 常駐と待避の予告

疎なモデルの経済は、置く場所の設計に直結する。一語あたりの計算が軽くても、本体全体を近くに置く必要が消えるわけではない。すべての専門家を常駐させるのか、一部を待避させて必要に応じて呼び出すのかという問いは、活性と総量の経済の裏側である。開かれた重みとして公開された疎な混合専門家モデルは、疎な活性の質の高さを広く確かめられるものにし、同時に運用側の関心と呼び起こした [10]。動かしてみたいという関心は、必然的にどこに置くかという関心につながる。

潜在注意による圧縮と疎な計算をともに前面に出し

た系譜は、記憶の軽さと計算の軽さを切り離さない読み方を示唆する [9]。文脈の蓄積を抑える工夫と、一語あたりの計算を抑える工夫は、別の欄の工夫でありながら、運用では同じ装置の上で出会う。後続の推論基盤を扱う章では、受け渡しと配置と並べ方の設計としてこの出会いが論じられ、長期的な記憶と運用を扱

う章では、蓄積の伸びと待避の判断として同じ出会いが別の角度から論じられる。本節の予告は、その二つの出会いに向けて、活性の軽さと常駐の重さを混同しないという契約をもう一度確かめるためのものである。

割当と配置の観点	問い	処方と限界
組成の設計	有限予算をどの分布に振り向けるか	参照模型による混合重みの事前設計 [16]。最適性は分布依存であり、定式化の細部は範囲外
蒸留と低ランク適応	容量を増やさず品質と適応を得るか	柔らかな教師目標による転移 [17]、凍結重みへの低ランク挿入 [18]。微調整効率率は事前学習効率と別物
層内並列による配置	単一装置に載らない層をどう載せるか	層内テンソル分割と最小限通信 [13]。総演算は不変であり、疎化の節約とは別勘定
冗長性排除と直交化更新	重複記憶と不安定更新をどう抑えるか	状態・勾配・パラメータの分割配置 [14]、直交化による更新整形 [15]。配置は前提であり、実装知は定理の代替にならない
対象と観点	問い	処方と限界
割当としての組成	有限予算をどの分布に振り向けるか	参照模型による混合重みの事前設計 [16]。最適性は分布依存であり、定式化の細部は範囲外
蒸留と低ランク適応	容量を増やさず品質と適応を得るか	柔らかな教師目標による転移 [17]、凍結重みへの低ランク挿入 [18]。微調整効率率は事前学習効率と別物
層内並列による配置	単一装置に載らない層をどう載せるか	層内テンソル分割と最小限通信 [13]。総演算は不変であり、疎化の節約とは別勘定
冗長性排除と直交化更新	重複記憶と不安定更新をどう抑えるか	状態・勾配・パラメータの分割配置 [14]、直交化による更新整形 [15]。配置は前提であり、実装知は定理の代替にならない
系譜の節目	設計の要点	残る交換関係
疎ゲート混合専門家	トークンごとに一部の専門家だけを稼働	偏りと補助損失の運用
専門家並列・自動分割	分散配置で単一装置の容量を超える	容量係数と通信の重さ
単純化された振り分け	一人選択で実装と拡張を見通す	豊かさと見通しの交換
専門家側選択	受け持ち量の固定で容量管理を明確化	あふれの扱い
細分化と共有専門家	組み合わせの表現力と転送の節度	管理の複雑さ
開かれた疎構成	常駐・待避・載せ替えの共有課題	重みの置き場所
低精度学習基盤	混合精度で大規模学習を成立させる	数値の安定と速度
分割・配置・記憶最適化	載せて動かすための方向	計算削減との混同に注意
組成の設計	混合比・蒸留・低ランク適応	割当と組成の二軸

[4, 5, 6, 8, 9, 10, 11]

本節の読解上の境界

同時代の実装点同士の優劣比較は行わない。数式の細部、評価表の数値、通信量の定量分析、収束定理の根拠には立ち入らない。分散学習の深掘りは範囲外であり、割当の前提としてのみ扱う。

3 Attention と Memory を減らす

ATTENTION AND MEMORY

長い文脈を扱うほど推論時の記憶は膨らむ。その最初の絞り込みは、鍵と値の持ち方を変えることから始まった。すべての照会頭が独自の鍵と値を抱える方式

では、系列が延びるにつれて保持すべき状態が増え続け、復号の足かせになる。そこで単一の鍵・値頭をすべての照会頭で共有する方式が提案され、保持量を仕

組みとして削る最も素朴な一歩になった [19]。品質面の代償の詳細な数値表には立ち入らず、確立した機構の水準で押さえる。続いて登場した束ねる共有方式は、複数の照会頭を少数の鍵・値頭に寄せることで、削減幅と表現力の折り合いをつけ直したものとされる [20]。学習済みの重みを起点に作り替えに近い形で移行できる点が実用上の鍵であり、全面学習なしに導入できる移行可能性が普及を後押しした。

一方、近似を一切入れずに速くする道もある。注意計算そのものは厳密に保ったまま、行列の分割と正規化の逐次化によって高帯域記憶との往復を減らし、実時間の効率を高める手法がそれだ [21]。複雑さの証明や速度表の水準には踏み込まず、厳密さと入出力効率の両立という役割で押さえる。版の扱いを補足する。厳密な入出力最適化の系列には中間版が存在するが、本巻は起点の提案と現行の復号カーネルの二点で代表させ、中間版の差分追跡には立ち入らない。中間版の列举は系譜の完全性には寄与するが、本巻の問いに必要な区別は、厳密さを保つ工夫と構造を変える設計の区別であり、版の新旧ではない。版の同定に紙幅を割かず、区別の物差しに集中することが、本巻の編集判断である。計算結果を変えずに入出力の動かし方だけを磨くのか、それとも計算や保存の対象そのものを変えるのか。この区別が本節全体の物差しになる。以降の疎化・線形化・状態削減・再利用のいずれも、どちらの側に立つのかを意識して読み分けたい。

3.1 疎化と局所化

すべてのトークンを見る代わりに、見るべき場所だけを選ぶ。階層的な圧縮と選択と局所走査を組み合わせ、実行環境に寄せた実装とともに最初から学習する方式は、推論時だけの刈り込みとは一線を画すとされる [22]。索引器の作りや共有の方式や学習形態の違いを取り違えないことが重要であり、似た名称の疎注意群の混同を防ぐことが特集全体の用語の要になる。対照的に、窓の中だけを見る局所注意は実績ある枯れた形であり、共有化された鍵・値の仕組みと組み合わせた運用例が生産的な基準になった [23]。以降の疎な大域設計や窓の再生をうたう仕組みは、この基準との対比で理解するのが見通しがよい。

新しい疎注意の登場を告げる公式発表は、告知の範囲として受け止める [24]。既存基盤の上に新しい機構を載せ、長い文脈の学習と推論の効率化をうたう内容であり、利用面の提供開始と重み・報告書・実装の公

開が合わせて告げられた。索引の数式や等価性の検証手順は報告書側に属する [25]。発表時点の価格改定の扱いも告知の水準にとどめ、効率と価格の伝達の速さを断定しない。発表と報告を対にして読み、役割の違いを保ちたい。

3.2 回帰と線形注意の系譜

疎化の中身を報告書の該当節の範囲でたどる。新しい疎注意は、少数の軽量な索引機構で関連の強い受け手を選び出し、細かい粒度で注目対象を絞る設計とされる。等価性に関する報告は抜粋の範囲にとどめ、互角性の主張は変種や評価条件に縛られるものとして、確立した水準を超えて引用しない。一方、回帰的な記憶の側にも系譜がある。差分規則による記憶更新に調節の門を加えた方式は、従来の回帰型の記憶制御を強めたものとして位置づけられ、後に続く線形注意系の直接の親になると整理されている [26]。その子にあたる線形注意の提案は、門をさらに細かくし、区分的な計算法と全域注意との交互配置で大域の流れを保つ設計を報告する [27]。比較表の数値は該当節の範囲として扱い、独立した再現は未確定とする。報告書の利用条件が再利用に制約を課す点にも留意する。

回帰と疎注意を組み合わせた実装点も現れている。一方の陣営の公開情報は、層ごとの外部記憶表をホスト側に置き、回帰層と疎注意層を一定比率で積む構成として、要約と傍証の範囲で整理する [28]。自己報告の評価値は検証待ちの提供者主張として隔離し、引用しない。ここで系譜の扱いが要になる。外部表を持つ設計と後に述べる条件付き記憶の提案との間に、導出関係を示す記述は収集範囲では見つかっていない [29]。似た圧力への収束的な応答、すなわち並行ないし未確定の関係として扱い、影響関係を主張しないのが正しい。一方、線形注意側の実行可能実装は、報告を超えた配置可能性の傍証になる [30]。ただし数値や精度の水準、性能の事実には踏み込まず、引用しない。呼称の衝突が未解決の別ラベルには触れず、混同を避ける。

3.3 保持を削る・使い回す・深さを絞る

保持する鍵と値を削る方向は、選ぶ仕組みを変えずに捨て方を工夫する。重要度の高い少数の受け手と直近の履歴だけを残す方式は、注意の値に効くのはごく一部のトークンだという観察に立つ [31]。冒頭のトークンが吸い寄せる現象への対処も欠かせない。最初の

受け手の鍵と値を残し、窓と組み合わせることで窓注意の性能を取り戻す工夫が示された [32]。層ごとの配分を変える発想もある。層によって注意の散らばりが違うことを利用し、保持枠を層間で山形に配る設計である [33]。本文の具体的な算法や評価の全文は未収集として、数値の比較には踏み込まない。いずれも計算式を近似するのではなく残す量と場所を絞る工夫であり、保存側の節約にあたる。

作った鍵と値を使い回すことも大きな節約になる。接頭辞の再利用を自動化する仕組みは、鍵と値の塊に印をつけて要求間で共有し、塊粒度の再利用を担う [34]。文書の本文を直接確認していないため、確立した役割の水準で述べる。一方、特定の実行基盤から切り離して再利用や退避や分離を担う層もある [35]。注意の数学を変えずに再計算を省く点で、提供時の節約にあたり、構造側の変更とは区別したい。条件付きの記憶という別の方向もある。古い埋め込み手法を現代化し、字句の圧縮や文脈に応じた調節で定数時間の参照を目指す提案である [36]。配分曲線の詳細や学習費用の精算には踏み込まず、報告の該当範囲で整理する。実装の展示の水準には踏み込まず、紙の主張の範囲で述べる。使い回しと条件付き記憶はどちらも記憶の節約だが、前者が作ったものの再利用、後者が呼び出す記憶の選択という違いがある。

深さ方向の計算を絞る発想もある。層ごとに上位選択で参加するトークンを絞り、系列と深さにわたって演算量を不均一に配る方式である [37]。専門家を振り分ける仕組みとは振り分けの対象が分類的に異なる。早期に抜ける工夫もある。層の脱落と早期終了の損失で学習し、浅い層の草案を残りの層で確かめる自己推測につなげる [38]。適応計算の先例としては、内部に分類器を添えて動的に抜ける古典的な工夫を、歴史の区切りとしてのみ位置づける [39]。深さの節約は注意の疎化や記憶の削減と直交しうる。どこを見るか、どれだけ残すか、どれだけ深く進むかを別々に絞ることで、正確な計算を保つ工夫と構造を変える設計の組み合わせが見通せる。

3.4 共有化の系譜をもう一度たどる

共有化の出発点は、すべての照会頭に対して鍵と値の頭を一つに束ねる発想である。照会側の多様性は残したまま、記憶側だけを共有することで、キャッシュの置き場所を節約できることが示された [19]。この節約が得られる代わりに、失われるのは鍵と値の投影の

個別性である。各頭が別々の射影で文脈を切り取る自由度は狭まり、共有された一組の記憶に依存せざるを得なくなる。共有化の得失は、記憶量の削減と表現の個別性の後退という言葉で理解するのが正確であり、性能の数値ではなく設計上の取捨選択として捉えるべきである。

この単純な共有を、照会頭の群として段階化したのが束共有の考え方である。照会頭をいくつかの群に分け、群ごとに鍵と値を共有することで、完全共有と完全個別の中間を取る [20]。群を細かくすれば個別性に近づき、粗くすれば記憶節約に近づく。この中間化自体は機構としては素朴だが、運用上決定的だったのは既存のチェックポイントからの移行可能性である。多頭で学習された重みを起点に、全体を一から学び直すことなく束共有型へ移せることが示され、配備済みの資産を捨てずに省記憶化できる道が開かれた [20]。新規学習の高負担を避けられることは、研究上の優劣以上に現場での採用可否を左右する。

共有化の延長線上には、頭を束ねるのではなく鍵と値自体を小さな潜在表現に圧縮し、その潜在を照会頭間で共有する方向がある。既存の強力な基盤から継続学習のみで移行し、構造変更を最小限に抑えながら疎な選択機構を載せる実例が報告されている [25]。この構成側の工夫に対して、復号カーネル側の最適化が相補的に用意され、両者が合わさって効率化の物語を構成している [40]。ここで注意すべきは、入出力の動かし方を洗練させて厳密な注意を保つ仕事とは役割が異なる点である。タイル化と逐次正規化により高帯域記憶への読み書きを抑えつつ近似なく計算する手法は、記憶の中身を変えるのではなく記憶の動かし方を変える [21]。共有化が何を保持するかを問い、厳密な入出力最適化がどう動かすかを問う。この区別を保つことが、系譜をたどる際の見通しを良くする。

3.5 疎化の二つの顔

疎化には、学習の段階から疎さを組み込む顔と、学習後の推論時に不要と見なされる記憶を落とす顔がある。両者はともに注意対象を減らすのが、適用できる時点と背負う制約がまったく異なる。混同すると、学習可能な選択と事後的な削除の区別が見えなくなる。

前者の代表は、圧縮と選択と局所走査を階層的に組み合わせ、学習過程から一貫して疎な注意を訓練する設計である。装置に整合したカーネルとともに端から端まで学習される点が、推論時のみ疎化する改修とは

本質的に異なる [22]。既存基盤の上に疎な注意を載せ、長文脈の学習と推論の効率化を掲げて導入された事例もあり、索引機構による候補付けと細粒度のトークン選択を共有潜在の下で動かし、継続学習で移行する構成が報告されている [24][41][25]。局所注意の参照点としては、滑走窓と束共有を組み合わせた実用的な基盤が重要であり、疎な大域設計が何に対して局所性を主張しているのかを測る物差しとなる [23]。学習時に疎さを組み込む道は、索引と局所性を訓練に含める代わりに、学習基盤と実装の作り込みを要求する。

後者の代表は、生成時に蓄積されたキャッシュから残すものを選ぶ削除の技術である。注意の価値が少数のトークンに偏る観察に基づき、重要度の高いトークンと直近のトークンを残す方策が提案されている [31]。また、系列先頭のトークンが注意を引き付ける受け皿として働く現象が知られており、先頭の受け皿と滑走窓を残すことで窓型注意の性能を回復できることが示されている [32]。さらに、層ごとに注意の異質性があることを利用し、層間でキャッシュ割当を不均一に配分する考え方があり、層を一樣に扱わない予算配分の視点を提供する [33]。これらはいずれも凍結済みモデルに適用できる軽さを持つが、削除である以上、削ってはならないものを削る危険と常に向き合う。

両者の使い分けは運用条件で決まる。学習や継続学習に介入できるなら、索引と局所性を組み込んだ疎さを訓練する道が選べる。すでに配備された重みを変えられず、長い文脈に対処しなければならないなら、事後的削除が現実解となる。相互作用として見逃せないのは、削除側が先頭受け皿の制約を受ける点である。いかなる削除方策も受け皿トークンを考慮しなければ後退するという制約は、削除の自由度に対する普遍的な留め金である [32]。また、層ごとに予算を考える発想は、削除だけでなく共有や選択の設計にも波及する。学習時選択と事後削除は競合ではなく、適用層の異なる補完関係にある。

3.6 記憶の三つの置き場所

記憶をどこに置くかという問いは、重みとキャッシュと外部再利用と条件付き参照を分けて考えると整理しやすい。常駐する重みは配備時に載り、要求間で動かない。増殖するキーと値のキャッシュは要求ごとに伸び、文脈が長いほど置き場所を食う。外部化された接頭辞や条件付きで引く表は、必要に応じて呼び出され、持ち運びと再利用の対象となる。何が動き、何

が留まるかを見ることが運用理解の核心である。

常駐と増殖の対比は顕著である。重みは一度載れば反復利用される静的な資産であり、移動の主体ではない。これに対してキャッシュは生成の進行とともに追加され、保持と転送の負担が増す。厳密な入出力最適化が帯域の動かし方を改善しても、増殖自体を止めるわけではない [21]。共有化や潜在圧縮が保持量を抑えるのは、この増殖の係数を小さくする働きである [19][20][40]。層ごとの異質性を踏まえた予算配分が効くのも、増殖が層ごとに一樣でないからである [33]。

外部再利用は、同じバイト列を再び計算せずに使い回す発想である。ブロック単位の照合により要求をまたいでキャッシュ片を再利用する仕組みは、粒度をブロックに定めた再利用の具体化である [34]。さらに、キャッシュの再利用や退避、分散を特定の推論基盤から切り離し、外部層として担う設計は、再利用と退避をまたぐ構成可能性を示す [35]。再利用は計算の省略であり、接頭辞が一致すれば待たずに使える。選択がその都度の注意対象を選び取る判断であるのに対し、再利用はすでに作られたものを運んでくる手配である。このため、再利用の成否は一致率と粒度、配置と転送の設計に依存し、モデルの賢さとは別の軸で効く。

条件付き参照は、記憶をすべて展開するのではなく、必要な断片だけを引く発想である。字句圧縮や複数頭のハッシュ、文脈に応じた調節、複数分岐の統合により、静的な表を参照計算として活用する設計が提案されている [36]。公式実装が公開され、配置や退避の振る舞いに実行可能な裏付けが与えられている [42]。ホスト側に置かれた大規模な表を引く層構成の事例も報告されており、表をどこに置き何を移動させるかという配置設計の重要性を示す [28]。ただし、この外部表と条件付き参照の設計との導出関係は確立しておらず、並行あるいは未解決として扱い、計算対記憶の圧力への収束的な応答と見るのが正確である [29]。回帰的記憶の親に当たる門付き差分更新の系譜や、それを発展させた注意設計の報告も、この文脈に連なる [26][27][30]。

結局、再利用と選択の区別は運用の言葉で言い直せる。再利用は当たれば得をする仕組みであり、外れても正しさは損なわれない。選択と削除は外すと品質に響く判断であり、受け皿や重要トークン、層別予算といった制約とともに設計しなければならない [34][35][31][32][33]。常駐する重みを厚くするか、増殖するキャッシュを痩せさせるか、外部に置いて運ぶ

か、引くべき断片だけ引くか。この置き場所の設計こそが、注意と記憶を減らす努力の実質である。

四分法の使い方

選別は窓の外側から何を残すかを定める操作であり、配置は残したものをどこに置くかを定める操作であり、追放は保持の上限のもとで何を捨てるかを定める操作であり、再利用は作ったものを運んでくる手配である。四者は相互に排他的ではなく、組み合わせうる直交軸である。重要度に基づく追放は、窓型の局所性とは異なる基準で保持対象を決めるため、窓が見落とす遠方の重要トークンを残せる可能性がある。一方、吸着点のような特異点の扱いを誤れば後退を招く。追放の方策を選ぶときは、重要度と直近性と特異点保持の三者をどう両立させるかを問う。配置の方策を選ぶときは、近くに置く速さと遠くに置く量のどちらを優先するかを問う。再利用の方策を選ぶときは、一致の粒度と共有の広がりやを問う。四分法は、記憶の節約策を分類するための物差しであり、組み合わせの設計図である。

3.7 潜在共有という第三の道

注意機構の記憶負荷を下げる試みは、大きく二つの発想に整理できる。一つは計算そのものを変えずに入出力を改善する方向であり、もう一つは計算や保持の構造自体を作り替える方向である。単一の鍵と値の頭をすべての照会頭で共有する方式は、その最も素朴で根本的なかたちである [19]。照会側の表現力は多頭のまま残し、記憶として保持すべき対象だけを束ねる。この束ねによって復号時に保持すべきキャッシュの占有量は構造的に縮小する。重要なのは、この共有が近似や枝刈りではなく、注意の保持対象の設計変更であるという点である。計算の厳密性を崩すのではなく、保持すべきものを減らすことで効率を得る。

この素朴な共有を、照会頭の群と共有される鍵と値の頭の群との対応へと一般化したものが、束共有の方式である [20]。すべての照会が単一の記憶を参照するのではなく、群ごとに異なる記憶を参照することで、記憶削減と表現力のあいだの中間点を設計できる。ここで運用上決定的だったのは、既存の多頭注意チェックポイントからの追加学習によってこの構造へ到達できるという性質である。最初から学び直すのではな

く、学習済みの重みを引き継ぎながら共有構造へ移行できることは、導入コストの観点から大きな意味をもつ。研究上の新規性とは別に、現場で使える効率化は、既存資産を活かせるかどうかで選別される。

第三の道としての潜在共有は、この系譜の延長上に位置づけられる。照会頭ごとに個別の鍵と値を保持するのでも、群ごとに鍵と値を保持するのでもなく、照会頭群で共有される潜在表現へと圧縮し、そこから必要に応じて鍵と値を再構成するという考え方である。定式化の細部には立ち入らないことが、本節の証拠上の制約である。潜在共有が買うものは、記憶の持ち方の自由度の向上である。トークンごとに鍵と値を並べるのではなく、共有潜在という中間層を設けることで、保持と再構成の分担を設計できるようになる。これは単なる圧縮ではなく、記憶の表現形式の変更である。

この潜在共有の方向が運用上の現実味をもつのは、既存基盤からの継続学習というかたちで導入されうる点にある。報告レベルで消費されている事実は、疎な注意の導入が、共有潜在を照会頭群で共有する動作様式のもとで、既存基盤からの継続学習として行われたことである [25]。潜在共有がゼロからの学習を要求せず、既存の重みと運用資産の上に積み増せるという運用的事実が重要である。もちろん、等価性や詳細な切り分けは抜粋範囲にとどまり、完全な本文検証は未了であるため、本節では性能の有無や程度について断定的な評価を下さない。

潜在共有の効率の話は、構造だけでは完結しない。復号カーネルとの対が、その効率を実際に引き出すためのもう半分をなすからである。この対関係を担うのが、潜在共有に最適化された復号カーネルとしての役割である [40]。構造側が保持対象を縮約しても、それを読み出し・再構成する実行系が対応していなければ、実時間上の利益には直結しない。逆に、実行系だけを磨いても、保持対象そのものが大きければ律速は残る。構造が潜在への圧縮を約束し、カーネルがその圧縮形式を前提とした復号を担うことで、はじめて記憶削減が実行効率へと変換される。単一共有から群共有へ、そして共有潜在へという流れは、記憶を束ねる粒度を粗くするだけの話ではなく、記憶の表現と実行の分担を再設計する流れとして理解されるべきである。

3.8 窓の再生と層の不均一性

長い文脈を扱う際のもう一つの実務的な出発点は、注意の窓を局所化することである。近傍のみを見る窓型注意と束共有を組み合わせた構成は、実用的な局所注意の基準として確立している [23]。どの幅が最適かといった定量的主張には踏み込まない。重要なのは、この構成が比較の基準点として機能していることである。局所化は計算と記憶を同時に抑える素朴で強力な手段であるが、窓の外側の情報をどう扱うかという問題を残す。

窓の再生とは、窓型注意が捨ててしまう情報を、何らかのかたちで再構成し、窓の限界を補う試みとして理解できる。代表的な現象として知られているのは、系列冒頭のトークンが注意の吸着点として振る舞い、その鍵と値を保持したまま窓を滑らせることで、窓型注意の性能が回復するという観察である [32]。窓を滑らせるだけでは失われるものがあり、その失われるもののなかに、見かけの重要度とは別に保持しなければならない特異点が含まれるということである。この特異点を無視して一律に捨て去る方式は、性能の後退を招く。本節では、具体的な再構成アルゴリズムの詳細や評価数値には立ち入らない。

この窓型の議論と並行して、層方向の不均一性に着

目する流れがある。すべての層に一律なキャッシュ予算を与えるのではなく、層ごとの注意の異質性を利用して予算配分を変えるという考え方である。錐体状に予算を配分する方式は、この層異質な配分の代表として位置づけられる [33]。記憶削減をトークン方向だけで考えるのではなく、深さ方向にも配分問題として考えることで、削減の自由度が増す。浅い層と深い層で注意の振る舞いが異なるのであれば、一律な保持は無駄を含む。配分の具体形やその効果の定量は未検証であり、本節では数値として引用しない。

保持すべきトークンを選別するという方向では、重み付きの重要度に着目する方式が確立している。小さなトークン部分集合が注意の価値の大半を担うという観察のもと、重要度の高いトークンと直近のトークンをキャッシュとして残す方式である [31]。分類上、この方式は追放型のキャッシュ削減に属し、選択型や配置型やオフロード型とは区別される。重要度に基づく追放は、窓型の局所性とは異なる基準で保持対象を決めるため、窓が見落とす遠方の重要トークンを残せる可能性がある一方で、吸着点のような特異点の扱いを誤れば後退を招く。あらゆる追放方策は吸着点トークンを考慮しなければならないというのが、運用上の留め金である。

節約の位置	代表的な仕組み	厳密か構造変更か
鍵・値の共有化	単一頭共有・束共有	構造変更（保持を削る）
入出力の最適化	タイル分割と逐次正規化	厳密（計算は変えない）
疎な選択	索引による注目先の限定	構造変更（見る場所を変える）
局所窓	窓内のみの注意	構造変更（大域を捨てる）
線形・回帰記憶	門付き差分と交互配置	構造変更（二次計算を置換）
保持の除去	重要度・吸着・層配分で残す	構造変更（残す量を絞る）
再利用と条件付き記憶	接頭辞共有・外部層・選択参照	厳密な再利用と記憶の選択
深さの適応	層別参加・早期終了・自己推測	構造変更（進む深さを絞る）

[19, 20, 21, 22, 27, 31, 33, 36]

本節の読解上の境界

由来の未確定な系譜は断定しない。呼称の衝突が未解決の別系列には触れない。数値表・核実装・速度の事実は引用せず、節範囲と告知範囲の読みに閉じる。

4 Bit を減らし、巨大モデルを手元で動かす

PRECISION AND LOCAL

巨大な言語モデルを手元で動かすとき、最初の壁は重みの置き場所である。初期の有力な答えは、ごく少

数の特徴次元に現れる外れ値だけを高精度計算に残し、残りの大部分を八ビット整数で処理する分解だっ

た [43]。外れ値次元は全体のごく一部でありながら量子化誤差を支配するため、そこだけを丁寧に扱うことで記憶容量を大きく減らしつつ精度劣化を抑えられる。続く一回性の重み限定の学習後量子化は、近似的な二次情報を手がかりに層ごとの再構成誤差を抑え、三から四ビット級の重みにまで圧縮しながら困惑度の増加を小さく保つ道を示した [44]。追加学習を前提とせず、少量の較正データで済む点が実務上の意味を持つ。

ここで分類を明確にしたい。重みだけを削る手法と重みと活性値の双方を削る手法は、誤差の出所も対策も異なる。さらに、学習後に合わせる手法と学習時から低ビットを前提にする手法、微調整時に量子化する手法は別の系統であり、同一の物差しで優劣を論じられない。鍵値キャッシュの量子化や最適化器の精度、保存符号化も重み量子化と同一視できない別軸である。詳細な外れ値の出現分析や層別誤差の導出には立ち入らず、概要水準の主張として扱う。

4.1 同時量子化の二つの道

重みと活性値の双方を八ビット化する道は、重みだけを削る道とは別の難しさを持つ。平滑化の発想は、活性値の振れを経路ごとの係数で均し、対応する分だけ重みを調整することで、再学習なしに重みと活性値の八ビット化を可能にする [45]。係数決定の数式導出には立ち入らず、難所の移行という要点に絞って扱う。一方、活性値統計から重要な重みを守る発想は、ごく少量の較正集合でも強い四ビット化を実現する [46]。すべての重みを一様に削るのではなく、出力への影響が大きい部分を保護し、残りに厳しい量子化を割り当てる。この重要度保護の考え方は、手元の実行環境で用いられる混合量子化や重要度行列の実践へと連なる。平滑係数の導出過程や顕著性分析の細部は摂取範囲外であり、機構の要点を概要水準で扱う。

4.2 器と削り方を分ける

手元で巨大モデルを扱う議論では、保存形式と量子化手法の混同がしばしば起きる。**GGUF は量子化手法ではなく表現／コンテナ形式である** [47]。この形式は、テンソル配置や付随情報の書式、記憶画像で読み出せる搭載方法など、局所推論の生態系が共有する器を定める。一方、重要度に応じた混合量子化や動的量子化の系列は、その器に載せるテンソルへ適用される手法であり、形式自体の性質ではない。同じ器に入っ

ていても、中身の削り方が違えば振る舞いは変わる。この区別が、容量と品質の議論の前提になる。

参照実装としての局所推論実行系は、この形式の読み書きと多様な量子化系列、中央演算装置と画像演算装置への分割実行を一体で担い、新規モデルの初期対応の受け皿になってきた [48]。層の一部を装置間で分担し、収まらない分を主記憶側に置く運用が、手元再現の現実的な姿である。公開文書で示される大容量モデルの器の用意は、手元の記憶容量に収まる選択肢の案内としてのみ扱う [49]。品質指標や多段予測による高速化の数値は提供元による測定であり、独立検証が未了のため事実として採用せず一般化しない。

4.3 浮動小数点の低ビット化

浮動小数点の低ビット化は、整数化とは別の設計軸である。八ビット浮動小数点の二形式の組み合わせは、指数部と仮数部の配分が異なる二者を用途に応じて使い分ける混合精度の方法論として提示された [50]。さらに、微小スケリングの公開仕様は、小さな要素群ごとに共有指数を備えることで、狭いビット幅でも精度を保つ枠組みを与える [51]。群幅三十二での共有スケールと、四ビット要素に共有指数を組み合わせた取り決めが、その具体例である。この系列の配備例として、高密度部を八ビット微小浮動小数点に置き、専門家部を四ビット微小浮動小数点に置く検査点構成が、文書化された手順の水準で報告されている [52]。鍵値キャッシュの量子化や最適化器精度とは軸が異なることに注意したい。配送符号の査読は行っておらず、実装正当性の断定はしない。文書化の水準を超えて語らない。

4.4 適応と原生低ビット

微調整の効率化は、量子化と適応の接点にある。凍結した四ビット基盤に低ランク付加器を組み合わせる方法は、大規模モデルの振る舞いを保ったまま学習対象を小さくし、単一の画像演算装置でも扱えるようにする [53]。ここで最適化器の精度や記憶管理は、重み自体の量子化とは別の軸であり、混同できない。一方、学習時から低ビットを前提にする原生低ビットの方向は、学習後に削る発想とは系統が異なる。一ビットで学習する変圧器の試みは、十分な規模で高精度に匹敵する振る舞いを示しうると報告される [54]。三値重みによる一・五八ビットの試みは、数十億規模で同等の困惑度や課題性能を保ちつつ遅延や記憶、流通

量、電力の利点を報告する [55]。ただし、特定企業を離れた産業配備は未確認であり、方向性の証拠として扱い配備の証拠としては扱わない。

4.5 総量と作動量の落差

巨大な混合専門家モデルを手元で動かす現実には、総量と作動量の落差にある。不均質実行の枠組みは、この性質を利用し、中央演算装置と画像演算装置に専門家を配置換えしながら必要分だけ読み出すことで、容量と計算を切り離す [56]。総量が大きくても作動量が小さければ手元の可能性が開ける。最新の旗艦級混合専門家モデルの手元移植は、変換器や疎注意、予測表などの構成を含む実装報告として公開され、検証が報告者側の記録として示されている [57]。予測表の主記憶挙動も実装記述の水準であり、独立した再構築や測定は行っておらず、正当性の断定はしない。実践者の受容記録は、配置に依存する観察として扱い、検証済み事実とはしない。技術的事実や経済的事実、許諾条件の事実をこの記録に依拠して述べることはしない。

4.6 外れ値と重要度の二つの見方

巨大モデルの精度を保ったままビット数を減らす試みが、一様な丸めではうまくいかない理由は、誤差が均等に広がるのではなく、ごく一部の場所に集中するからである。一部の特徴次元には突出して大きな値が現れ、一部の重みは活性化の振る舞いと結びついて強く保護に値する。どこを丁寧に扱うべきかを見極めれば、全体を高精度に保つ必要はなくなる。この見極めに二つの流儀がある。一つは外れ値そのものを分離する考え方であり、もう一つは活性化の統計から重要度を読み取る考え方である [43][45][46]。

外れ値駆動の分解という流儀を切り開いたのが、突出した外れ値次元だけを高い精度に残し、残りの大部分を低いビット幅で計算する手法である [43]。全体を一律に低ビット化すれば、外れ値が量子化範囲を引っ張り、通常の値の分解能が犠牲になる。そこで計算の経路を分け、少数の扱いにくい次元だけを別扱いにする。こうした分離によって、制約の強い記憶容量でも巨大なモデルの推論が可能になるという方向性が示された。ここでの発見は単なる実装の工夫にとどまらず、後に続く手法が共有する前提となった。すなわち、活性化に現れる外れ値こそが量子化の難しさの中心であり、その扱いをどう設計するかが手法の性格を決めるという見方である。

もう一つの流儀は、活性化の統計を手がかりに、守るべき重みを見つける考え方である。重みだけを低ビット化する学習後量子化の枠組みでは、層ごとの誤差を二次情報を用いて補償する手法が代表的であり、学習し直すことなく低いビット幅へ到達できることが示された [44]。これに対して、重みと活性化の双方を低ビット化しようとする、活性化側の外れ値が新たな壁になる。この壁を、チャンネルごとの平滑化によって重み側へ難しさを移し替えることで乗り越えようとしたのが、重みと活性化の同時量子化を学習なしで可能にする手法である [45]。平滑化は、外れ値の激しいチャンネルを穏やかに均し、その分だけ対応する重み側に負担を寄せる操作として理解できる。活性化の量子化を容易にする代償を、比較的对処しやすい重み側で引き受けるという分担の発想である。

活性化から重要度を読み取る方向をさらに推し進めたのが、顕著な重みを保護する手法である [46]。この手法は、活性化の統計を観察することで、どの重みがモデルの振る舞いを強く左右するかを判定し、その重要な重みが量子化誤差で損なわれないよう優先的に守る。重要な少数を厚く守り、それ以外を思い切って削るという配分が、低いビット幅でも精度を保つ鍵になる。この重要度保護の考え方は、後に手元での実行環境における混合量子化や重要度行列の実践へと直接つながっていく。較正データの儉約性もここから生まれる。必要とされるのは大規模な学習用データではなく、モデルの振る舞いの癖を読み取るための少量の文章である。学習し直しを要せず、手元の少量の観察だけで適用できることは、手元の装置との相性を決定的に良くする。こうした軽さが、後の共有容器と結びついて、手元での試行錯誤を広げた。

二つの見方がともに一様な削減を上回るのは、精度の予算を誤差の痛い場所へ集中投下するからである。外れ値の分離は痛みの発生源を隔離し、平滑化は痛みの置き場所を移し替え、重要度保護は痛みに弱い箇所を事前に補強する。いずれも、全体を同じ粗さで扱うのではなく、観察された振る舞いに応じて扱いを変えろという点で共通している。

4.7 形式の標準化がもたらしたもの

手元で巨大モデルを動かす実践を支えたのは、個別の量子化手法の巧妙さだけではなく、それらを運ぶ器の共通化である。コンテナ形式は、テンソルの配置や付随するメタ情報、読み出しのための載置の仕方を定

め、実行環境が同じ手順で多様なモデルを開けるようにする [47]。メモリへの写像によって大きな重みを効率よく扱い、中央演算装置と画像処理装置の間で処理を分担する実行形態と組み合わせることで、容量の大きな重みと実際に計算する部分とを切り離す運用が可能になった [48]。この器の上に、量子化の系列が載せられる。特定の量子化の系列は器に格納される対象であり、器そのものの性質ではない。ここで改めて確認しておきたいのは、GGUF は量子化手法ではなく表現／コンテナ形式であるということだ。

共有された器がもたらした第一の変化は、比較可能性である。同じ容器、同じ読み手、同じ実行手順の上に異なる量子化が載るとき、利用者は手法の違いを同じ土俵で比べられる。変換器や読み込み手順、分割実行の仕組みを共有できることは、手順の共有という第二の変化を生む。新しいモデルが現れたとき、器と手順が共通であれば、移植の作業は一から作り直す仕事ではなく、差分を埋める仕事になる。実際に、混合エキスパート型の大規模モデルの手元への移植は、変換器やグラフ、検証手順の報告を伴いながら、参照実装の上に積み上げられる形で進められた [48][57]。派生した実行系による文書化や配布の整備も、同じ器を前提とするからこそ利用者に届きやすくなる [49]。

第三の変化は、異種配置という運用の定着である。総量の大きな重みを手元の記憶容量に収め、実際に働くごく一部だけを高速な装置で計算するという分離は、混合エキスパート型の重みと記憶の問題に対する実践的な答えとなった [56]。重みの置き場所と計算の場所を切り離す考え方は、容器による写像や分割実行の仕組みと相性が良く、容量と速度の異なる装置を組み合わせる運用を後押しした。仕様が器を定め、手法が載せ物を定めるという分担が、比較と共有の循環を回したのである。

4.8 低ビットの三つの時間軸

低ビット化をいつ行うかという問いは、誤差がどこで生まれるかという問いと表裏一体である。学習が終わった後に丸めるのか、適応のための学習と組み合わせるのか、それとも学習そのものを低いビット幅で設計するのかによって、誤差の源も、対処の姿も変わる。この時間軸の違いを見分けることが、手元で動かすための選択を誤らないための助けになる [44][53][54][55]。

第一の時間軸は、学習後の量子化である。学習済み

の重みを、学習し直すことなく低いビット幅へ変換する立場であり、重みのみを対象とする一回的な変換がその代表である [44]。ここでの誤差は、事後的な丸めから生まれる。対処は、層ごとの誤差を補償する工夫、外れ値の扱いを変える平滑化、重要な重みの保護といった、観察と補償の技術になる [45][46]。利点は適用の軽さであり、手元の少量の観察で済み、学習基盤を持たない利用者でも試せることである。限界もまた明確であり、学習過程そのものに手を入れない以上、丸めによって失われた情報を取り戻すことはできない。

第二の時間軸は、量子化と適応を組み合わせる立場である。低いビット幅で凍結した土台の上に、小さな学習可能な付加部分を載せて調整する手法がその代表であり、量子化の選択が微調整の効率を直接左右することを示した [53]。ここでの誤差は、単なる丸めの誤差ではなく、限られた表現力の上でどれだけ目的の振る舞いに寄せられるかという適応の誤差を含む。土台を軽くすれば手元の装置でも調整が可能になり、適応の自由度を残せば用途への合わせ込みができる。この時間軸は、量子化と適応という別々の営みが、精度の予算を通じて結びついていることを教える。削り方の選択が、学び方の効率を決めるのである。

第三の時間軸は、はじめから低いビット幅で学習する立場である。二値や三値の重みで大規模な変換器を学習し、その規模拡大の振る舞いを全精度と対比可能なものとして示そうとする方向がこれにあたる [54][55]。ここでの誤差は事後的な丸めではなく、低い表現力のなかで学習が安定に進むかという学習そのものの難しさから生まれる。対処もまた、学習手順や規模則、表現形式の設計になる。微小な粒度で共有縮尺を定める形式の標準化や、少数の要素形式の整備は、この方向を支える土台として理解できる [50][51]。実際に、専門家ごとに異なる精度を割り当て、密な部分や埋め込みや出力層の扱いを分けた構成が、運用の現場で用いられる事例として報告されてきた [52]。ただし、研究としての方向性の証拠と、広く配備された実績とは区別されねばならない。後者については開かれた問いとして残すのが、証拠の範囲に忠実な態度である。

さらに見落としてはならないのは、重みのビット幅とは別の軸が二つあることである。一つは学習や適応のための記憶であり、最適化の状態や中間結果が占める容量は、重みそのものの軽さとは別に考えねばなら

ない。もう一つは、生成の過程で蓄積される鍵と値の記憶であり、その量子化は重みの量子化とは異なる誤差の振る舞いを持つ。重みを削ること、学習の記憶を抑えること、生成時の記憶を抑えることは、それぞれ誤差の源も対処も異なる別問題である。三つの時間軸と二つの別軸を混同せず、どの軸のどの誤差に対処しているのかを問い続けることが、ビットを減らして手元で動かす試みを読み解くための確かな物差しになる。

4.9 較正データの集め方と落とし穴

較正データは学習のための訓練集合ではなく、すでに学習を終えたモデルを丸める際に基準を定めるための小さな参照集合である。再学習を伴わない事後量子化では、この参照集合から決まる尺度や保護対象の選び方が最終的な誤差の出方をほぼ決めてしまう。標本が少ないこと自体は前提であり、問題は量ではなく被覆にある。特定の文体や特定の課題に偏った文脈だけを集めると、量子化の基準がその偏りに引きずられ、実利用の文脈で想定外の劣化を招く。

最初に被覆すべきは、外れ値を生む通路である。活性化には一部の次元だけが突出して大きな値を取り続ける性質があり、その分離の発想が後の多くの量子化手法の出発点となった [43]。この突出はすべての文脈で一様に現れるのではなく、文脈の種類や長さによって現れ方が変わる。較正集合がおとなしい文脈ばかりで構成されると、突出する通路の規模を見誤り、丸めの幅を楽観的に決めてしまう。較正集合には、静かな文脈と荒い文脈の両方を代表させる必要がある。

次に被覆すべきは、守るべき重みである。活性化の統計から顕著な重みを見極めて保護する考え方は、ごく小さな較正集合からでも強い低ビット化を可能にするとされる [46]。ここでの要点は、少ない標本でもよいという楽観ではなく、少ない標本だからこそ顕著性の判定が標本の選び方に敏感になるという留保である。較正文脈が特定領域に偏ると、特定領域では重要に見える重みだけが保護され、別領域で効く重みが見落とされる。顕著性は模型に固定的に備わる性質というよりも、どのような文脈で活性化を観測するかによって変わる関係的な性質に近い。

さらに、通路ごとの分布の代表性も欠かせない。活性化の量子化の難しさを重み側へ移す平滑化の考え方は、通路ごとの尺度を較正から決めることを前提に、重みと活性化をともに量子化する正統的な経路を開い

た [45]。この移し替えがうまく働くのは、較正集合が各通路の振る舞いを代表している場合に限られる。平滑化は万能の前処理ではなく、較正の被覆が前提となる調整である。

領域偏移に対する感度は、手法の種類によって現れ方が異なる。重みのみを丸める一回限りの事後量子化は、活性化を高精度に残す分だけ同時量子化よりは頑健に見えるが、誤差補償のために重み間の関係を用いる手法では較正文脈の統計的性質が補償の効き方に影響する [44]。顕著性保護を用いる手法は較正活性化に直接依存するため領域依存が残り [46]、平滑化を用いる手法は通路分布の代表性に依存するため領域依存が増す [45]。外れ値分離の発想はこれらに共通する前提を与えたが、分離のしきい値自体は領域に依存する [43]。較正集合を他手法の流用で済ませると、前提のずれがそのまま誤差になる。

落とし穴として多いのは、較正を単なる形式的手続きとみなし、任意の公開文脈の断片で代用することである。公開文脈の断片は入手しやすいが、実利用の指示や対話とは分布が異なる。別の落とし穴は、較正における良さだけを見て本番の良さを推測することである。較正集合の上で誤差が小さいことは、較正集合の上での適合がうまくいったことを意味するに過ぎず、未見の領域での振る舞いを保証しない。対策は、較正集合を目的別に分け、代表例と先鋭例と外れ例を意識的に含め、利用想定領域を外さないことである。手法を選ぶ前に較正を固定するのではなく、手法の前提に合わせて較正を集め直す姿勢が肝要になる。

4.10 削り方の選び方

削り方の選択は、精度と記憶量と実行基盤の三者の折り合いである。何を犠牲にできるかによって正解が変わるため、手法の優劣を一般論で決めるのではなく、利用の型から逆算するのが近道になる。

推論のみを再学習なしで軽くしたい場合は、重みのみの事後量子化が第一選択になる。活性化を高精度に残すため計算経路の変更が小さく、既存の実行環境への持ち込みが比較的容易である。手元の推論の定番としてまず検討すべきはこの系統である。重みだけでなく活性化も量子化して処理の効率や整数演算器の活用まで狙う場合は、同時量子化の系統が必要になる。ただし、較正の質への依存が増し、領域が大きく変わる利用では劣化が出やすい。また、実行環境が同時量子化の計算様式に対応していなければ、理論上の効率は

実効の速度に結びつかない。

量子化したまま振る舞いを变えたい場合や、少量の追加学習で下流課題に合わせたい場合は、適応結合型が向く。凍結した低ビットの母体に低ランクの適合を組み合わせる考え方は、単一の計算機での大規模モデルの適応を可能にしたとされる [53]。ここでは量子化と適応が相互作用し、精度の選び方が適応の効率を直接左右する。推論のみで足りる場合に無理に適応を組み合わせる必要はなく、母体の振る舞いを变えたくない場合にも向かない。

最初から低ビットで学習する流派は、手元の定番としてではなく研究の方向性として位置づけるのが妥当である。二値で学習する試みは全精度に匹敵する拡張の振る舞いを示し、低ビット学習の方向性を開いた [54]。その後の三値による後継手法は、同規模で全精度に匹敵する性能と効率面での利点を主張した [55]。ただし、特定の主体を超えた産業展開は未確定として扱うのが証拠の範囲に忠実であり、定番の推論手段と同列に扱うべきではない。一方、微小区分化による低精度形式の専門家混合での利用は、提供基盤における具体的な配置例として記録されている [52]。研究の方向性と提供の実例を混同せず、学習の土台ごと変える覚悟があるかで判断するのがよい。

ここで改めて確認すると、容器形式は量子化手法ではなく格納形式であり、格納される量子化群は形式に適用された手法であって形式自体の性質ではない [47]。形式の名で精度や速度を語らず、適用された手法と実行環境の組み合わせで評価する姿勢が要る。

整理すると、再学習なしの推論の軽量化なら重みのみ、効率化まで求めるなら同時量子化、振る舞いの変更を伴うなら適応結合型、学習手法自体の探求なら最

初からの低ビット、提供基盤が対応する明確な適用箇所なら微小区分化形式、容量と計算の切り離しなら容器と実行環境の工夫、という順で考えるのが実務的である。いずれの場合も較正の被覆と領域想定のみ示を欠かさず、形式と手法の混同を避けることが、手元での再現性と見通しを守る近道になる。

4.11 手元実行の可否をどう見極めるか

旗艦級のモデルを手元で動かせるかどうかは、総量の大小ではなく、器と手順と配置の三点がそろいかどうかで決まる。第一に、器の有無である。対象模型の重みが共有容器の形式で配布され、テンソル配置と付随情報が読み手の実行環境で開けることが前提になる [47][49]。配布の存在は可能性の存在であり、品質や速度の保証ではない。第二に、手順の有無である。変換器や分割実行や不均質配置の手順が、報告者の検証記録とともに示されていることが必要である [48][57]。手順があっても、その検証は報告者の構成での検証に留まり、別の構成への転用は保証されない。第三に、配置の見通しである。総量に対する作動量の比、退避経路の設計、主記憶と帯域の見積もりが、自分の装置の顔ぶれと合うかどうかを見極める [56]。三点のいずれかが欠けても、手元実行は成立しない。逆に三点がそろえば、総量が大きくても可能性は開ける。最小ビット幅での記憶適合の目安や、精度優先での選択肢の幅は、案内としてのみ扱い、品質や速度の事実としては採用しない [49]。移植報告の透明性は評価できるが、正確性の裏書きにはならない [57]。可否の判断は、器・手順・配置の三点照合で行い、総量の印象では行わないことが、手元実行の読みの作法である。

手法の系統	向く場面	向かない場面・留保
重みのみの事後量子化	再学習なしで推論のみを軽くし手元の計算機に載せたい場面	活性化の量子化による加速を狙う場面や較正の代表性が乏しい場面
同時量子化	整数演算を活かし処理の効率まで求める場面で分布が安定している場合	利用領域が大きく変わる場合や実行基盤の対応が乏しい場合
適応結合型	丸めたまま下流課題に寄せたい場合や単一計算機で振る舞いを变えたい場合	推論のみで足りる場合や母体の振る舞いを变えたくない場合
最初からの低ビット学習	学習手法自体を研究し将来の効率化の方向性を探る場合	手元の定番推論や産業展開を前提とした導入には向かない場合
学習用低精度形式と微小区分化	学習基盤と提供基盤が対応し適用箇所が明確な場合	対応しない環境への持ち込みや形式のみでの精度向上を期待する場合
容器形式と実行環境の工夫	容量と計算を切り離し大規模な混合専門家を手元で動かす場合	形式を手法と混同した選定や実行環境の成熟を見込まない導入の場合

削る対象と時点	代表的な手法	分離すべき軸
重み限定・学習後	外れ値分離・再構成誤差の抑制	活性値同時量子化と混同しない
重み活性同時・学習後	平滑化・重要度保護	重み限定と誤差の出所が異なる
微調整時の量子化	凍結基盤と低ランク付加	最適化器精度は別軸
原生低ビット	一ビット学習・三値重み	学習後量子化の代替と断定しない
浮動小数点の低ビット	混合精度・微小スケールリング	整数化と別の設計軸
保存の器	表現・コンテナ形式	量子化手法ではない
局所実行	分割実行・不均質配置	総量と作動量を分ける

[43, 44, 53, 47, 50, 54]

実践者の受容—配置依存の観察

手元配備の可否や外部退避と主記憶と帯域が隘路になること、量子化選択と実行系の成熟が使い勝手を左右することは、実践者の報告として共有されている [57, 49]。ただし配置に依存する観察であり、検証済み事実とはしない。技術的事実や経済的事実の根拠には用いない。

本節の読解上の境界

GGUF は表現・コンテナ形式であり量子化手法ではない。提供元測定の品質・速度数値は隔離し、独立検証まで一般化しない。符号庫の査読や再構築は行わず、産業配備の証拠としては扱わない。

5 1 token ずつ待たない

DECODING AND BUDGET

生成モデルの待ち時間の多くは、一トークンずつ順番に確定させなければならない直列性に由来する。投機的デコーディングはこの構造を正面から置き換えるのではなく、小さなドラフト側が複数トークンを先に提案し、標的モデルがそれらをまとめて検証する手順に変える [58, 59]。受け入れられた接頭辞だけを確定し、外れた位置からは標的分布に従って引き直すため、出力の分布を保ったまま無駄な逐次呼び出しを減らせる。標的モデルの再学習を要しない点も重要で、既存の重みをそのまま検証器として使える。速度の鍵は受容率であり、ドラフト案がどれだけ標的に受け入れられるかが全体の効率を決めるという整理が、以後の研究を貫く評価軸になった。ただし受理条件の厳密な証明や速度表の全体は本稿の消費範囲に含まれず、機構と保証の骨格に限って扱う。直列性の緩和と推論予算の節約は隣り合うが別問題である。

5.1 先読みの質を上げる

ドラフトの質を上げる方向で転機になったのが、特微量レベルで先読みする方式である [60]。トークン表層ではなく中間特徴量から次の展開を予測し、単層の軽量復号器と木構造の注意機構で複数候補を束ねて検

証する。基幹モデルの微調整を要せず出力分布を変えない無損失の範囲が保たれ、量子化やコンパイルと組み合わせ可能とされる点も実務上の含意が大きい。先行する多頭方式は、凍結した基幹モデルに軽量な複数ヘッドと木検証を付加し、別個のドラフトモデルを不要にした前例として位置づけられる [61]。貪欲復号での無損失性は保たれる一方、非貪欲生成の品質は限定付きの対象であり、無条件の保証ではない。発展形では木を文脈依存に動的化し、受理モデルの予測で展開を変える [62]。ただし木構造や学習データのアブレーションは部分的であり、単一の測定条件の数値を普遍定数として扱うことはできない。

複数トークンを同時に学ぶ訓練が、推論の先読みに転用される流れも押さえておきたい。共通胴体の上に独立な複数出力ヘッドを置き、次トークン予測に加えて先のトークンを補助課題として学ぶ方式では、訓練時の追加負担なしに標本効率が上がり、規模が大きいほど効果が伸びると報告されている [63]。現在の実装面では、推論サーバの文書において標的モデル自身の多トークン層が草案を作る方式が案内され、別個のドラフトモデルを要しない運用が示されている [64]。ただし文書本文の全体は未取得であり、主張は要約と下

流の裏づけに限定される。ベンダ側の速度数値は隔離し、独立の確認が済むまでは引用しない。

5.2 足し算の効率

テスト時計算の配分に目を向けると、大きなモデルに置き換える前に考えるべき選択肢が見えてくる。難易度適応的な検証器探索と分布改訂を組み合わせた最適配分では、素朴な最良候補選択の反復に対して効率改善が本文の要旨として報告されている [65]。検証器の訓練の詳細は未取得であり、万能の優劣として読むのではなく、基礎能力が残る領域での条件付きの効率論として受け止める必要がある。単純なテスト時拡張の事例では、数万件から千件への難易度と多様性と品質による精選が土台になる [66]。思考を打ち切ったり末尾に待機語を付して延長したりする予算強制が核になり、短すぎる思考を延ばして推論を修復する操作として働く。予算の付け方が結果を左右するという点で、計算資源の配分はモデル選択と同等の設計変数である。

5.3 引き算の効率

長い思考を短く保つ学習的な試みでは、制限を報酬設計に織り込む発想が採られている [67]。制限を越えた出力は報酬計算の前に破棄される厳格な上限のもとで強化学習を行い、上限を段階的に締めながら要点を選び直す。本文の結果表では、蒸留由来の小規模学習対象において正答率をほぼ保ったまま生成長がおおむね半減する様子が示されている。温度や試行回数や上限長を明示した手順とともに示され、評価条件の束縛が明示されている点が重要で、汎用的な短縮保証ではない。発見的な打ち切りではなく学習された方策として位置づけられ、重みを更新する手法と推論時の置換策の境界が保たれている。背景には大規模強化学習で推論能力を引き出し小型高密度モデルへ蒸留する流れがあり [68]、対象外の規模や数学外課題への一般化は未確定である。訓練費用の会計は抽出されておらず、推論効率の改善と学習費用の総量は切り分けて読む必要がある。

考えさせれば考えさせるほど良いわけではないという観測が、予算設計の現実味を支えている [69]。長い推論では正答率が上がってから下がる山形の推移が見られ、正しかった答えを捨ててしまう反転事象が記録されている。計算量を罰則付きの効用で捉える枠組みでは、費用に敏感な条件では早期停止が同等の精度で

大きく節約するという整理が示されている。例示の算術を請求書の実額と混同せず、費用対効果の考え方として読むことが重要である。運用界面では、数値で推論努力を指定する系列と、思考の可否や段階離散の強弱で指定する系列という、明示的予算の操作系が報告されている [70]。ただし界面文書の全体は未取得で、段階意味の把握は要約水準に留まる。復号の待ちと思考の浪費を混同せず、層の異なる無駄として設計する視点が求められる。

5.4 受容率という単一の物差し

投機的デコーディングの成否を決めるのは、草案の巧拙でも検証の厳しさでもなく、両者の噛み合わせが受容率という一つの量に畳み込まれた結果である。草案が先に複数候補を並べ、標的モデルがそれをまとめて検証し、受け入れられた接頭辞だけを残す。この往復では、草案がどれだけ速く巧みに書いても、標的の分布がそれを撥ねつければ前進は生まれない。逆に、標的の判定が甘ければ速く進むが、守るべき分布そのものが崩れる。だから受容率は単なる中間指標ではなく、速度と正しさの交換を一本化する物差しとして残り続ける [58]。

この枠組みに厳密な形を与えたのが、受容判定の定式化である。草案分布と標的分布のずれを補正する受容条件を明示することで、分布を保ったままの加速が何によって保証されるのかが見えるようになった [59]。無制約な近似ではなく、受容条件を通った分だけが標的分布の再現として正当化される。この条件があるからこそ、後の手法は貪欲生成の厳密さを保つ部分と、非貪欲生成で品質保証の範囲を限定せざるを得ない部分とを切り分けて議論できる。

草案の質と標的の厳しさは相補的であるという見方はここから自然に導かれる。軽量の複数頭で基幹を凍結したまま草案を出す方式は、別立ての草案モデルを持たずに検証の木構造で受容の機会を増やす発想である [61]。貪欲生成の範囲では分布の不変性が保たれる一方、非貪欲生成では品質の主張は限定付きになる。この限定は弱さではなく、受容率の物差しで見た正直な適用範囲の宣言である。特微量水準で草案を作る方式は、この相補性をさらに押し進め、表層の語彙予測ではなく内部表現の連続性に草案の根拠を求める。本体を微調整せずに出力分布を変えない立場は、量子化やコンパイルといった実装上の最適化と組み合わせやすいことも含めて、受容率を下げないための設計として

読める [60]。

草案の木を文脈に応じて動的に組み替える考え方は、受容率を事後的な平均値ではなく、文脈ごとの予測対象として扱う転換である [62]。ただし、この物差しの適用範囲には開いた問いが残る。とりわけ、バッチが大きくなったときや装置構成が変わったときに受容率と実効速度の関係がどうずれるかは、現時点の証拠では確定していない。受容率が高いことと運用条件で速いことが常に一致するとは限らない。受容率を不変の真理ではなく、条件付きの物差しとして扱う姿勢を保つべきである。

5.5 学習の改善としての先読み

複数トークン予測は、しばしば復号の工夫としてだけ語られるが、その出自は学習の改善である。共有された基幹部に独立な複数の出力頭を付け、未来の複数位置を同時に予測させる補助課題は、学習時の負担を実質的に増やさずに標本効率を高め、規模が大きくなるほど効果が育つ方向に働く [63]。速く生成するための仕掛けを後付けするのではなく、未来を見越して学ぶことがそもそも良いモデルを作るという順序の逆転にある。

実装面での帰結は、標的モデル自身の先読み層が草案を担い、別立ての草案モデルを不要にする形態である。文書化された実装経路は、要約水準の証拠に裏打ちされた範囲ではあるが、主要な系統が先読みを目的関数に取り込む流れと整合的である [64]。旗艦的な系統における採用を、単なる高速化の流行としてではなく、目的関数の出自として読むことが、この小節の狙いである。学習時に未来を予測させたモデルは、復号時に未来を草案できる。推論基盤がその能力を直接呼び出せる形で整備されたとき、先読みは研究の工夫から運用の前提へと位相を変える。

この整理は、推論様式の系譜にも接続する。強化学習で育てた推論様式を小さな生徒モデルに移す流れは、現在の利用の錨として残る [68]。先読みの採用史も同様に、個別の高速化手法の列挙としてではなく、どの世代が何を目的関数に据えたかという系譜として読むべきである。学習を良くすることが復号の選択肢を増やし、復号の選択肢が学習の価値を運用に変える。この循環こそ、先読みを単なる先回り生成と呼ばず、学習の改善として扱う理由である。

5.6 予算表で考える

テスト時計算を一枚の予算表として眺めると、計算を足す方向と削る方向は対立する主張ではなく、同じ表の左右に見える。足す方向の代表は、検証器付き探索や分布の改訂を難易度に応じて配分する考え方である。難しい問いには探索を厚くし、易しい問いには生成の手直しで済ませる。この適応的な配分が、単純な多数決的な基準を上回る効率を生むというのが、計算最適という言葉の実質である [65]。検証器の学習の細部は未確定だが、難易度適応という濃淡の存在自体は揺らがない。

同じ表の足す方向のもう一つの姿が、思考の打ち切りと強制延長である。厳選された少量の高難度・多様・高品質な問いに対して、思考を途中で切ること、待機を促す合図で延ばすことも、どちらも予算操作として同列に扱える。量や多様性や難易度をめぐる詳細な切り分けは構造的水準にとどまるが、予算を外から与えるという操作の素朴さと、その効果の堅牢さが要点である [66]。

削る方向の代表は、厳しい上限のもとで強化学習により短く考えることを学ばせる方法である [67]。上限を超えた出力は正誤にかかわらず報酬計算から除外されるため、モデルは限られた長さに収まる解法を獲得せざるを得ない。この手法が依拠する推論蒸留の系譜は、強化学習で育成した推論様式を生徒系統に移す流れに属する [68]。

削る方向を正当化するもう一つの視点が、考えすぎの分析である。長い推論は常に良いわけではなく、正しかった答えを途中で捨てる反転を伴いながら、伸ばすほど精度が頭打ちから低下に転じることがある [69]。反転機構の因果分析は部分的であり、費用の算術は請求書ではなく例示だが、予算表に費用の欄を設けるべきだという主張は明確である。運用界面における明示的な予算つまみ、すなわち数値や段階で推論の努力量を結び付ける機構は、この費用意識の生産形態である [70]。

予算表を閉じるときに欠かせないのが、学習費用と推論費用の分離である。強化学習による短縮の学習費用は抽出されておらず、推論時の節約と混同してはならない。読者が持つべき表は、行に難易度、列に足す操作と削る操作、欄外に学習費用と推論費用を置いたものである。復号の直列性と推論予算の浪費は同じ機構ではなく、隣り合う課題であることを忘れずに、こ

の表を運用の道具として使ってほしい。

5.7 多頭草案から特徴量草案へ

生成言語モデルの復号が本質的に直列であるという事実は、運用の現場では待ち時間の構造そのものを規定している。次トークンを一つずつ確定させる過程では、並列計算資源が潤沢にあっても、系列方向の依存が解消されない限り先へ進めない。この直列性を緩和するための基本発想が、草案と検証の分離である。小さく速い仕組みで先の並びを仮置きし、大きな標的模型でまとめて検証し、受け入れられた接頭辞だけを確定させる。ここで要となるのが受容基準の形式化であり、検証段階で標的分布を保つための条件を明確化したことが、その後の多様な草案手法の共通土台となった [59]。受容率が支配的な枠組みであるという見方は、この形式化に由来する。

この流れにおいて、基幹を凍結したまま軽量な多頭出力を付加する発想は、運用上の転換点となった。別個の草案模型を用意する方式では、草案側の保持や適合や更新が負担となり、標的が替わるたびに作り直しが生じやすい。これに対して、凍結された基幹の上に複数の復号頭を載せ、木構造の検証と組み合わせる方式は、標的を育て直すことなく先読みの能力だけを接ぎ木する [61]。基幹の重みを動かさないことは、既存の振る舞いを保ったまま加速だけを狙うという保守的な選択であり、配備済みの資産を活かす観点からは相性が良い。検証が木構造であることも重要である。単一の線形な先読みではなく、分岐を含めた候補束をまとめて検証することで、わずかな分岐の違いを吸収し、受け入れの機会を広げる。

しかし、表層のトークン予測としての草案には、原理的な停滞がある。トークンは離散的であり、表層の選択はしばしば鋭く切り替わる。文脈上は自然な続きであっても、表層の記号としては別の選択肢が競合し、草案が外見上の不一致で棄却される場合がある。この種の不一致は、草案の質の問題というより、表層という観測面の粒度の問題である。受容率の観点からは、表層の不確実性がそのまま検証通過の難しさに跳ね返る。ここに、表層予測の延長では効率の天井が見えやすい理由がある。

この停滞に対する転回が、特徴量水準での草案である。復号器内部の特徴は、離散記号に切り替わる手前の連続的な状態であり、文脈の蓄積をより滑らかに担っている。特徴量草案は、この連続性に着目し、単

層の復号器で特徴の先行きを予測し、そこから表層を導くことで、表層直接予測の不確実性を回避する [60]。特徴が文脈の履歴を圧縮したかたちで保持しているため、草案は表層の揺れに引きずられにくく、意味的な続きを保ったまま先へ進める。基幹の微調整を要さず出力分布の不変性を保つ設計は、多頭草案の系譜が築いた保守性を継承している。

草案の木を固定的ではなく動的に扱う段階に入ると、受容の科学はさらに運用寄りになる。静的な木は、あらかじめ定められた分岐幅と深さで候補を展開するため、文脈の難易や確信の偏りに応じた調整が効きにくい。文脈依存の受容予測は、この要請に応えるものであり、文脈ごとの受け入れやすさを見積もりながら木の展開を加減する [62]。確信の分布が運用中に移り変わることを前提とし、草案の資源配分をその場で組み替える発想である。貪欲な生成と非貪欲な生成の扱いの違いは、この系譜全体の品質保証の範囲を理解する上で欠かせない。貪欲な設定では無損失性の保全が明確であり、一方、温度や多様性を伴う非貪欲な生成では、品質の主張は限定付きとなり、無条件の無損失性をうたうものではない [61]。運用者にとって重要なのは、加速が分布の変質と引き換えになっていないかを見極めることであり、貪欲域での保証と非貪欲域での限定を区別して受け止める姿勢が求められる。

5.8 運用に載せる先読み

先読みの考え方が研究段階の技巧から運用基盤の機能へ移るとき、文書化された実行環境の案内が果たす役割は大きい。代表的な推論実行基盤の文書では、標的に内在する先読み層が草案を担い、別個の草案模型を要しない経路が示されている [64]。この案内の要点は、加速のために外部の付属物を探して適合させる必要がなく、標的自身の構造に由来する先読みをそのまま検証に回せるという点にある。文書水準の主張に留まり、特定版の改訂番号や対応版の下限に踏み込んだ断定は避けるべきであるが、下流の配備行列との整合が示唆されている範囲では、文書化された経路が配備の手段となっているという理解は妥当である。

この運用化の背景には、目的関数の系譜としての多トークン予測がある。共有される胴体の上に独立な出力頭を置き、将来の並びを補助課題として学ぶ方式は、学習時の負担を大きく増やさずに標本効率を高め、規模が育つほど恩恵が表れやすいとされる [63]。ここで注目すべきは、目的関数が単に学習を助けるだ

けでなく、復号時の再利用への伏線となっていることである。目的関数由来の先読みであることは、後付けの草案頭とは異なり、胴体の表現自体が先読みに適したかたちで育っていることを含意する。

主力系譜における先読みの採用は、この目的関数来歴が実装の来歴へ転化した局面として読める。先読み層を備えた標的が第一級の選択肢として配備され、文書化された実行経路に載ることは、先読みが例外的な高速化ではなく、既定の構成要素として扱われ始めたことを示す [64][63]。目的関数の設計と実行基盤の対応がかみ合うことで、学習時に培われた先見性が、追加の外部機構なしに検証へ供給される。特定の製品名や版数に結びつけた断定は避け、系譜としての採用という水準で捉えることが、証拠の範囲に即している。

この運用化を遠景として支えているのが、大規模強化学習に由来する推論系譜の存在である。当該系譜は、強化学習によって推論能力を引き出し、その推論様式を高密度の継承模型へ蒸留することで、推論の振る舞いを広く行き渡らせたと位置づけられる [68]。本稿では、この系譜を評価表の水準で論じることはいない。全過程の学習管路や蒸留の評価表は未摂取であ

り、主張は系譜的位置づけに限定される。推論様式の継承が進んだことは、先読みの運用化と無関係ではない。推論が長文化し、思考の継続が運用コストに直結する状況では、復号の直列性を緩和する先読みと、思考の予算を制御するつまみとが、隣接する関心として浮上するからである。

予算のつまみとしての努力量制御界面は、運用に載った先読みの隣に置かれるべきもう一方の操作面である。現行の対話型界面の系譜では、推論への努力配分を段階的に指定したり、思考の有効化や段位の切替えて振る舞いを変えたりする明示的な予算操作が整いつつあることが、要約水準で報告されている [70]。要点は、思考の長さを成り行きに任せず、利用者が意図した予算感で抑揚を付けられることにある。つまみの存在は、推論の質と費用の均衡を利用者側へ委ねる設計思想の表れであり、先読みによる加速と相補的である。先読みが確定までの待ちを短くするのに対し、予算つまみは考える量そのものを加減する。両者は機構としては別でありながら、運用の体感としてはともに待ちと費用の制御に寄与する。

予算表の軸	足す操作	削る操作	欄外の費用
難易度の配分 草案の質	検証器付き探索と分布改訂 受容率の向上と動的木	打ち切りと強制延長 先読み学習の転用	検証器の育成費用は別勘定 別立て模型の不要化は運用の 利得
思考の長さ 努力量の指定 分布の保存	待機合図による延長と立て直し 数値と段階のつまみ 受容条件の統制	上限制約の学習と早期停止 中程度での回収 置換策との混同の回避	短縮の学習費用は未抽出 界面仕様は要約水準 重み更新と置換は別機構

[58, 60, 63, 65, 66, 67, 69, 70]

待ちと予算の設計	仕組みの要点	隣接するが別機構
投機的検証 特徴量の先読み 先読みの学習 検証器付き探索 予算強制 学習の短縮 過思考の知見 明示的予算	小さな草案を標的が一括検証 中間特徴量から複数候補を束ねる 複数トークンを補助課題として学ぶ 難易度適応と分布改訂 打ち切りと延長で思考を修復 上限制約の強化学習で要点化 山形推移と反転の記録 努力量の数値指定と段階指定	分布保存と受容率が鍵 表層予測の限界を越える 学習の改善と復号の再利用 基礎能力が残る条件付き 短すぎる思考の延ばし 重み更新と置換策の境界 止める判断が精度維持 要約水準の把握

[58, 60, 63, 65, 67, 69]

本節の読解上の境界

証明の細部、速度表の全体、アブレーション、訓練費用の会計、対象外規模への一般化には立ち入らない。ベンダ側の速度数値は隔離する。直列性と予算浪費を一括りに語らない。

6 Serving で消える無駄

SERVING EFFICIENCY

大規模言語モデルの運用で最初に詰まるのは、重みの大きさではなく生成途中で膨らむ記憶である。ここでの転換点は、鍵と値の履歴を連続領域として抱え込まず、小さな塊に分けて必要な分だけ結び付けて管理する発想である [71]。塊単位になれば再利用や受け渡しの単位も揃い、空きの再利用が細かく回る。到着順に束ね直す連続的な束ね方は、この記憶管理があって初めて安定する。その後の接頭辞再利用や分離構成、外部への退避はいずれも、履歴を塊として照合し移動できることを前提にしており、この層の有無が後段の工夫の効きを決める。本稿では詳細な表設計や条件付き測定全体には踏み込まず、確立した機構としての役割に限って扱う。伝えたいのは、**構造が巧妙なモデルでも記憶の管理と束ね方が粗ければ運用の効率は伸びない**という順序である。

6.1 重なるの使い回し

対話や道具呼び出しを重ねる使い方では、同じ前置きを何度も計算し直す無駄が膨らむ。ここでの工夫は、接頭辞の共有関係を木のように捉え、一致する部分の計算結果を使い回すことである [72]。制約付き生成のための状態管理を圧縮して持つ発想も、同じ枝刈りの延長にある。さらに、再利用の仕組みを特定の実行機構の内部に閉じ込めず、外部の層として切り出して複数の機構から使えるようにする発想がある [35]。使い回しの設計は応答の速さだけでなく費用の読みやすさにも効く。粒度の違いも重要で、塊の照合による再利用と木による管理は互いに含み合う関係ではなく、得意な共有の形が異なる。どちらか一方が他方を包摂するものとして語らず、使い分けとして理解すべきである。本稿では設計の細部や条件付き評価の全体には踏み込まず、繰り返し呼び出しの無駄を減らす確立した役割に限って述べる。

6.2 段階の分離

生成は前半と後半で性質が異なる。前半は文脈を一気に読み込む計算中心の段階であり、後半は一つずつ言葉を紡ぐ記憶帯域中心の段階である。両者を同じ資源プールで同じさばき方に縛ると、どちらかに合わせた調整がもう一方の無駄になる。ここでの発想は、段階ごとに適した資源の集まりを用意し、役割分担で全体をならすことである [73]。ただし分離には代償があ

る。生成途中の記憶を受け渡す転送が新たに生まれ、特化の利益と受け渡しの費用の釣り合いが問われる。記憶を分散した共有資源として束ね、必要に応じて高速な転送で結ぶ考え方は、この釣り合いを運用する土台になる [74]。本稿では分離の配置解析や費用表、分散保持の手順や評価の全体には踏み込まず、確立した役割の範囲で扱い、補完的な記録に基づく部分は抽象の範囲に留める。

同時処理では、長い入力の読み込みが短い応答の生成を邪魔するという噛み合わせの悪さが出る。読み込みを小さな断片に分けて生成の合間に滑り込ませるやり方が、その処方箋である [75]。断片化すれば生成の遅れを抑えながら読み込みを進められ、全体の詰まりがほぐれる。後続の展開は、その釣り合いを運用条件に合わせて整える方向に進んだと位置づけられる。運用条件が変われば最適な区切り方も変わるため、一つの正解を固定せず調整の幅を持たせることが重要になる。本稿では分割手順の細部や得失の分析の全体には踏み込まず、補完的な記録に基づく確立した役割の範囲で述べる。

6.3 核と通信と基盤

運用の軽さは、待ち行列の工夫だけでは決まらない。圧縮された注意の復号に寄せた核は、構造側の工夫と対になる実行側の工夫であり、復号の重さを直接引き受ける [40]。多数の専門家に分散する構成では、分散と結合の通信が全体の重さを左右し、通信のさばき方が計算の工夫と同じ重みを持つ [76]。行列積を細かな粒度で低精度化する土台は、大規模な学習の数値的な成立を支え、学習と運用をつなぐ [12]。複数の技法を一つの実行基盤に同居させる共有核の考え方は、個別の最適化を組み合わせるようにする接着剤である [77]。核と通信と数値基盤の三層が揃って初めて、待ち行列の工夫が実を結ぶ。本稿では核の実装や数値解析、通信量の測定全体には踏み込まず、確立した役割に限って述べ、個別の高速化数値は引用しない。

運用の仕上げでは、既存の先読み部品を外して別の推測の仕組みに置き換えるといった選択が現れる [78]。公開された導入手順は生きた文書であり、特定の固定構成での確認はその構成での測定に留まる [79]。他の画像や上限設定、異なる資源量への転用は

保証されず、文書の更新に伴う揺らぎを前提に読むべきである。構造の効率と運用の効率は別の勘定であり、両者を混同すると投資の順序を誤る。まず漏れを塞ぎ、次に構造の利益を取り込む順序が堅実である。本特集の一貫した含意は、構造が効率的でもさばき方が粗ければ利益は漏れ、運用の設計こそが効率の上限を決めるということである。

6.4 待ち行列の約束を守る

運用における約束とは、計算量の少なさではなく、締め切りと同時進行の捌き方である。模型が軽くなっても、待ち行列の先頭で長い処理が居座れば、後続の応答はまとめて遅れる。利用者が感じる破約の多くは、演算器の不足ではなく、置き場所の競合から生じる。とくに生成推論では、入力を一気に読む段階と、一語ずつ吐き出す段階とで資源の使い方が異なり、両者を同じ列に無造作に並べると、一方の重さが他方の締め切りを壊す [73][75]。

この競合をほどく鍵は、記憶を連続領域として抱え込めないことにある。鍵と値の履歴を細かな塊に分けて管理すれば、断片化で使えない隙間が生じにくくなり、空いた場所に次の要求を詰め続けられる [71]。重要なのは、単に詰め込み率が上がるのではなく、優先度に応じた中断と退避に柔軟性が生まれることである。急ぐ応答のために譲る、長い入力のために一時的に外す、終わった分から回収する。そうした取り回しが利くからこそ、締め切り順の約束が運用として書ける。

長い入力の扱いは、その柔軟性を試す典型である。入力読みを一括で通すと、そのあいだ吐き出し側が止まり、体感遅延が跳ねる。これを小さな断片に区切って吐き出しの合間に相乗りさせれば、管の泡を埋めながら、吐き出し側の停滞を抑えられる [75]。接頭辞の再利用も、待ちの短縮として読み直せる。共通の前置きを塊単位で使い回せば、同じ入力を繰り返し読む無駄が消え、列の進みが速くなる [72][34][35]。

最後に、受け入れの見方を分けておきたい。合成的な負荷で測った受け入れの良さは、あくまで測定条件付きの記録であり、そのまま運用の棄却挙動を統治しない。実運用では、締め切りを守れない要求は待たせるのではなく断る判断が要る。約束を守るとは、すべてを受け入れることなく、受け入れられる範囲を見極め、溢れを早めに返すことである。

6.5 受け渡しの経済学

分離は、得意分野への特化で稼ぎ、受け渡しの費用を払う設計である。入力読みと吐き出しを別々の資源溜まりに置けば、それぞれの計算と記憶の性格に合わせた置き方ができる。ただし分離した瞬間に、履歴の受け渡しという新たな勘定が発生する。特化の利得から受け渡しの費用を差し引いた残りが、実質の利得である [73]。

この勘定を理解するには、記憶を付随物ではなく一級の資源として扱う見方が要る。鍵と値の履歴を、計算の傍らに置かれた一時物ではなく、溜めて、集めて、運べる資源とみなせば、設計の選択肢が変わる。履歴中心の分離は、この資源観を前面に出したものであり、記憶の置き場所と運び方を運用の中心に据える [74]。

外部化された再利用と退避の層は、同じ経済学を実行機関から切り離す。特定の実行機関に閉じない形で履歴の使い回しや退避や受け渡しを担えば、再利用の仕組みと退避の仕組みを組み合わせやすくなる [35]。退避先の選び方も、経済計算の一部である。装置内の記憶に置ききれない履歴を、計算から少し離れた大きな記憶へ逃がせば、同時進行の余裕が生まれる。逃がす先を近くに置けば戻しは速いが置き場所の節約は小さく、遠くに置けば節約は大きいが戻しに時間がかかる。万能の正解はなく、置き場所ごとの出し入れの費用を見ながら決めるほかない。

運用の評価は、だから総量の伸びだけを見てはならない。分離で処理量が伸びても、受け渡しで食われた分を差し引かなければ、正味は分からない。受け渡しの経済学とは、特化の華やかさではなく、差し引きの地味さを直視する姿勢である。利得から費用を引いた残りが正である配置だけが、約束を守りながら残る。

6.6 文書と実装の対応

文書は生きており、ある固定構成での検証は、そのまま他の構成へ移らない。ある像とある設定とある同時実行の上限で確かめられた振る舞いは、その組み合わせの記録であって、別の像や別の上限での保証ではない [79]。手順書が生きて動く段階では、検証の記録は特定の時点の姿を留めたものにすぎない。固定された教科書として読むのではなく、変化する運用手帳として読む。どの組み合わせで確かめたのか、その確かめはどこまで移せるのか、その二点を欠かさず確認す

る読み方が要る。

設計の置き換えは、仕様と実行の対進化として現れる。従来の投機部品を捨て、別の方式の起草と検証へ切り替えるような変更は、模型側の取り替えだけで完結しない。実行側がその起草を受け止め、検証の流儀を合わせ、後方装置と歩調をそろえて初めて振る舞う[78]。核となる演算や通信や土台も、同じく組み合わせで読むべきものである。注意機構に寄り添う復号核、専門家並列の送受信を担う通信庫、低精度行列積を支える演算庫、そして頁化や疎や投機の経路を束ねる共有土台[40][77][76][12]。運用者の本当の関心は、技法の名前ではなく、どの組み合わせが振る舞うかである。文書に載る手順、模型側の来歴、実行側の受け止め、核と通信と土台の顔ぶれ、再利用と退避と受け渡しの置き方。これらがそろって初めて、ある配置は動く配置になる[79][80][34]。

6.7 同時実行と約束の両立

実運用における同時実行の課題は、単に多くの要求を捌くことではなく、混雑時にも応答の切れ目を破綻させないことにある。生成推論では入力を一気に読み込む前処理段階と一語ずつ生成を続ける復号段階とで計算と記憶の性格が大きく異なり、前者は一時的に大量の計算資源を消費し、後者は保持した鍵と値の記憶を繰り返し参照しながら少しずつ進む。この非対称性を無視して素朴に束ねると、長い前処理が復号の列を塞ぎ止め、待ち行列の先頭が全体を滞留させる。ここで問われる約束とは、特定の応答時間の数値ではなく、同時実行数が増減しても前処理の割り込みと復号の継続とが互いを壊さないという振る舞いの約束である[73][75]。

記憶管理の土台として重要なのは、鍵と値の保持領域を連続した巨大な領域として確保するのではなく、不連続な小片の集合として管理し、断片化をなくす発想である[71]。断片化が消えれば、長さの異なる要求を詰め合わせても空きが無駄にならず、到着した順に実行列へ継続的に取り込む一括処理が現実的になる。重要なのは、単に詰め込み率が上がるのではなく、優先度に応じた中断と退避に柔軟性が生まれることである。

その自由度の上に置かれた代表的な一括処理の考え方が、復号の歩みと切り分けられた前処理とを相乗りさせる方式である。長い前処理をひとまとめに実行すれば復号が長く待たされ、逆に前処理を後回しにすれば

装置が遊ぶ。この板挟みに対し、前処理を小さな塊に切り分け、各実行刻みに復号の継続と一緒に少しずつ混載させることで、管の泡を埋めながら前処理の侵入が復号の遅れへしみ出さないようにする。これが確立された役割として知られる方式の要点であり、詳細な切り分け手順や評価の細部に立ち入らずとも、方向性は明確である[75]。

この混載の発想は単発の工夫で終わらず、処理量と応答の切れ目との折り合いを運用として扱う方向へ継ぎ足して発展した。後継の系譜は、混載という機構を残したまま、運用者が回せる調整つまみを明示化し、全体の捌き量を上げる設定と応答の切れ目を守る設定との間の緊張を、運用方針として選べる形にした点に意義がある[75]。混載とは対照的に、段階の性格差を配置の分離で解こうとする考え方もある。前処理と復号とを性質の異なる計算資源の集まりに振り分け、各段階の計算と記憶の性格に合った資源を当てることで、混載の調整を配置の適材適所へ置き換える。この分離は、各段階が互いの邪魔をせずに伸びるという利点をもたらす一方、切り分けたことの代償として鍵と値の受け渡しを必ず支払うことになる[73]。

運用者が実際に回す調整つまみは、記憶や配置の有無ではなく、一括処理の組み立て方に集中する。たとえば一つの実行刻みにどれだけの前処理片を混ぜるか、同時に入れる要求の顔ぶれをどう組み合わせるか、前処理片の切り幅を細かくして復号への影響を抑えるか粗くして前処理の進みを速めるかといった選択である。これらはいずれも処理量を上げれば応答の切れ目が厳しくなり、切れ目を守れば捌き量が鈍るという同じ緊張の別の現れである[75][71][73]。記憶の小片化が進み、混載と分離という両極の手段が揃った後では、残る差は運用としての一括処理方針に集約される。混雑時に何を優先し何を先延ばしにするかという方針こそが、利用者から見た切れ目の守られ方を決める。

6.8 置き場所の選び方

鍵と値の置き場所をめぐる選択は、計算し直すか、近くに残すか、遠くに預けて呼び戻すかという三つの道の折り合いである。生成推論では一度作った鍵と値を捨てれば次は作り直しになり、残せば記憶を食み、遠くに預ければ呼び戻しの移送を払う。接頭辞の再利用も退避も分離も、この同じ物差しの上でしか評価できない。近くに置くほど呼び戻しは速いが置ける量は

限られ、遠くに置くほど量は置けるが呼び戻しの負担が増す [35][34][74]。

接頭辞再利用の要点は、要求をまたいで同じ出だしの計算を繰り返さないことにある。実装上の肝は一致の見方を小片単位の散列に預けることであり、小片ごとに指紋を取って等しさを判定することで、要求の境界を越えた使い回しが機械的に可能になる。この粒度の選び方が運用の振る舞いを決める。小片を大きく取れば判定は軽くなるが、少しの違いで全体が不一致となり再利用の機会を逃しやすい。小片を細かく取れば一致の機会は拾いやすいが、判定と管理の手間が増える。さらに小片単位の一致は、構造的な共有の見方とどちらが優れるとも言えない別種の選択であり、運用の重なり方によって向き不向きが変わる [34]。

内側で捨てるか外側で残すかは、再利用と退避とを貫くもう一つの選択である。個々の実行機構の内側だけで鍵と値を管理すれば、破棄の判断は速く単純になる。これに対し、特定の機構から切り離された外側の層に再利用と退避と分離の機能を移せば、使い回しの機会は機構の境界を越えて広がる。ただし外側化はただではない。層をまたぐ移送と整合性の管理が要り、近くに置く場合に比べれば呼び戻しの道のりは長くなる。したがって外側に置く選択は、共有による作り直しの節約が移送の負担を上回るときに正当化される [35]。

遠くに預ける考え方を突き詰めたものが、鍵と値を第一級の分散資源として集めた溜めの設計である。この設計では鍵と値の置き場を特定の計算装置の脇から切り離し、容量の大きな集約記憶や永続層を束ねた溜めとして扱い、高速な転送路で結ぶ。共有が効く運用では、一度作った鍵と値が別の計算主体からも呼び戻され、作り直しの重複が減る。反面、溜めが遠くなるほど呼び戻しの道のりは長くなり、転送路の混雑や層の段差が応答の切れ目に響く [74]。

近い退避と遠い退避の経済は、容量の多寡だけでは決まらない。近くに置く退避は呼び戻しが速く、層の段差が小さい。短い待ちで復帰できる反面、置ける量は脇の余裕に縛られる。遠くに置く退避は置ける量は大きく、共有の効果も期待できる反面、呼び戻しには転送の負担が伴う。作り直しが軽い短い接頭辞であれば、遠くからの呼び戻しを待つより作り直した方が速い場合がある。作り直しが重い長い接頭辞や繰り返し現れる定型の出だしであれば、呼び戻しの負担を払ってでも溜めに残す方が得になる [35][74][34]。

配置の選択を難しくするもう一つの事情は、検証された構成の転用が利かないことである。配備の手引きは生き物のように漂流し、夜ごとの更新で前提が動く。ある構成の下で確かめられた配置が、別の構成の下でも同じ振る舞いをするとは限らない。手引きに載る配置例は出発点としては有用だが、そのまま持ち込めば同じ切れ目が守られるという保証にはならない [79]。

実務的には、置き場所の選び方を次の順で考えると見通しが良い。まず要求の重なりを見る。出だしの重なりが濃い運用では接頭辞再利用の粒度を整えることが先決である [34]。次に共有の広がりを見る。機構の境界を越えた使い回しに意味がある運用では、外側の層に機能を移すことが効く [35]。さらに置ける量と呼び戻しの道のりを見る。量の不足が支配的なら集約された溜めを第一級の資源として扱い、切れ目の厳しさが支配的なら近い溢れ先に寄せる [74]。最後に固定された構成での検証に留める。確かめた構成の外へ持ち出すときは再測定を前提とする [79]。

6.9 切り幅の調整つまみ

前処理片の切り幅は、運用者が直接回せる代表的な調整つまみである。切り幅を細かくすれば、復号の合間に滑り込ませやすくなり、前処理の侵入が復号の遅れへ滲み出すのを抑えられる。代わりに、前処理全体の進みは遅くなり、長い入力完了が後ろ倒しになる。切り幅を粗くすれば、前処理は速く進むが、復号側の停滞が長引き、体感遅延が跳ねる。この往復は、処理量と切れ目の折り合いそのものであり、正解の幅は運用の優先順位で変わる。切れ目を守りたい対話用途では細かめに、捌き量を上げたい一括処理用途では粗めに寄せるのが自然である [75]。切り幅と同時に入れる要求の顔ぶれも調整対象になる。長い前処理と短い対話を混ぜれば、短い方が長い方に引きずられる。顔ぶれを揃えるか、優先度で割り込ませるかも、待ちの配分を決める。装置の理論性能ではなく、切り幅と顔ぶれの組み立てが、誰がどれだけ待つかを決める。混雑の波が来たときに遅れが一部の要求に集中するか全体に薄く広がるかも、この組み立てで変わる。切り幅の調整は、実装の細部ではなく、約束を運用に翻訳する作業である。

6.10 優先度と割り込みの設計

混雑時には、すべての要求を平等に扱うことが最善ではなくなる。急ぐ対話応答のために、長い一括処理を一時的に譲らせる判断が要る。塊単位の記憶管理があれば、譲らせた分を退避させ、空いた場所に急ぎの要求を詰め、終わった分から回収する取り回しが利く[71]。優先度の設計は、静的な等級付けではなく、締め切りと待ちの動的な配分である。長い入力のために

一時的に外す、終わった分から回収する、急ぐ応答のために譲る。この取り回しが利くからこそ、締め切り順の約束が運用として書ける。割り込みの設計では、譲らせた側の飢餓を防ぐ手当ても要る。譲り続けると長い要求がいつまでも終わらず、全体の捌き量が落ちる。譲りの上限と復帰の順序を決め、飢餓と切れ目の両方を守る。優先度は固定値ではなく、混雑の度合いに応じた動的な配分であり、その配分を決める方針こそが一括処理方針である。

運用の無駄と処方	仕組みの要点	払う代償・留保
記憶の断片化	塊単位の管理で再利用を細かく回す	表設計の細部には立ち入らない
同じ前置きの再計算	接頭辞の木共有・外部層	粒度の違いは使い分け
段階の性質差	前半向きと後半向きに分ける	受け渡し転送の費用
分散保持	記憶を第一級資源として束ねる	保持自体にも費用
読み込みと生成の干渉	断片化して合間に滑り込ませる	区切り方は条件依存
復号の核	圧縮注意に寄せた実行	構造側と対の工夫
専門家並列の通信	分散と結合のさばき	計算と同等の重み
低精度の土台	細粒度の行列積	学習と運用の接着剤
共有核	複数技法の同居	世代差の吸収
生きた手順書	固定構成での確認	他構成への転用は慎む

[71, 72, 73, 74, 75, 40]

本節の読解上の境界

設計の細部、条件付き評価、配置解析、費用表、手順の全体には踏み込まない。文書の対応は特定構成での確認に留まり、他構成への転用はしない。合成的な受け入れ確認は測定として扱い、実運用の拒否動作と切り分ける。

7 「大きなモデルを毎回呼ぶ」必要はあるか

ROUTING AND SPECIALIZATION

すべての問いをいきなり最大のモデルに投げるのではなく、安価で軽いモデルから順に試し、足りなければ強いモデルへ引き継ぐというのがカスケードの基本的な発想である [81]。問いに対する採点器と、どのモデルへ振るかを決める振り分け器、そしてここで止めてよいかを判断する打ち切り判定を組み合わせ、精度を保ちながら全体の計算量や利用料を抑える。ここで見逃せないのは、料金体系がモデルごとに異なる点である。入力と出力のどちらに重く課金するかは様ではなく、しかも料金の並び順自体が対象の課題によって変わる。つまりカスケードは単なる節約術ではなく、課題特性と料金構造を踏まえた割り当て問題として理解すべきものである。運用ではトークン単価ではなく課題完遂あたりの費用で捉える。ただし本稿では当時の料金体系に基づく報告として扱い、現在の価格

にはそのまま当てはめない。最適化の定式化の全体や詳細な数表の再現には立ち入らず、仕組みと考え方に焦点を当てる。

7.1 学習型の振り分けとその脆さ

問いごとに強いモデルと弱いモデルを使い分ける学習型の振り分けでは、問いの難しさや性質を見極める予測器が要になる。代表的な手法では、人手の好みデータから勝敗予測を学び、強いモデルが必要な問いだけを強いモデルへ回す [82]。狙いは、簡単な問いを安価にさばき、難しい問いだけに高価な計算を充てることで、全体としての費用対効果を高めることにある。ただし好みの分布は時間とともに変わり、学習時の分布と運用時の分布がずれば予測の当たり外れも変わる。判定を担うモデル自体の偏りや、評価用デー

タの混入対策の限界も、結果の読み方に影響する。詳細な成果数表や構造の比較検討には立ち入らず、データと予測に基づく振り分けという設計思想と、その前提条件に焦点を当てる。したがって学習型振り分けは、万能の節約装置ではなく、分布の近さと見極め精度に依存する条件付きの仕組みとして捉えるべきである。

振り分けによる節約の話には、必ず留保条件を付けなければならない。好みデータで学んだ振り分け器は、分布が変わると脆さを見せることが指摘されている [83]。学習時と同じような問いが来る間は節約と精度の両立が成り立っても、話題や言い回し、難しさの分布がずれると、強いモデルに回すべき問いを弱いモデルで済ませたり、その逆が起きたりしやすくなる。この指摘は、先行する節約の報告を否定するものではなく、正しく使うための条件を付け足す役割を果たす。どのようなずれに弱いかは、要約として伝えられている範囲にとどまり、本文の詳細な条件分けには立ち入らない。一般化して語れる段階ではなく、条件の細部は未確認として扱うのが筋である。運用では定期的な見直しと、振り分けの当たり外れを監視する仕組みが前提になる。

7.2 生成しない判断という特化策

生成で長い文章を作る代わりに、状態を受け取って型付きの確率的な答えを返すという特化型の事例がある [84, 85]。並列に動作する標本抽出器と独自の学習手法の組み合わせで作られ、文字列の生成ではなく選択肢の選択や点数付けといった操作を直接行うと説明されている。学習手法の内部詳細は公開されておらず、外形的には並列性と出力の意味づけの範囲で理解するしかない。確信度は多くの予測を束ねた集計としての較正を意図したもので、個々の答えの正しさを保証するものではないと説明されている。入力文章のみで英語中心であり、分類や振り分け、採点、抽出、分岐といった流れの中の判断部分に向くという位置づけであり、**自由な文章生成を置き換えるものではない**。較正の実質的な質や英語以外の精度は独立には確かめられておらず、手元の内容で試すことが前提とされている。契約の細部には立ち入らず、置き換えではなく使い分けの事例として読むべきである。

提供元は、この特化型について高速性と低廉さをうたっている [86]。短時間での応答や、従来の大型モデルに対する大幅な速度・費用の優位が、自社で組んだ

業務処理の評価として示されているとされる。ただし測定は特定の手元の環境から行われ、比較対象の動かし方に独自の包みが挟まる点や、参照回答の偏りなども開示されており、数字を額面通りに受け取ることはできない。したがってこれらは確立した事実ではなく、**提供元による主張として受け止める必要がある**。独立した再現は未確認であり、速度や価格、処理量の主張はその留保付きで扱う。英語以外の精度は弱いことが明記され、手元の内容での確認が推奨されている。価格や処理上限は文書時点の記載であり、将来の変更によって変わり得る。

この特化型をめぐる外部の情報は、補強的な位置づけにとどまる。有志の覚え書きは一次文書と整合的な記録を残しているとされるが、仕様の典拠にはならない [87]。愛好者による再現の集積地も、類似の仕組みを試す手がかりを集めている段階とされ、査読を経た独立再現が確認されたわけでもない [88]。応答の型や確信度の導出など契約の細部は、照会先の資料に属する事柄であり本稿では立ち入らない [89]。較正を確かめる公開手順も見当たらないというのが現状の記録である。

7.3 契約面の読み方

応答の型や確信度の導出、別名指定と版表示の扱い、料金形や上限の記載は、照会先の資料に属する [89]。概念説明で掴んだ仕組みの理解を、契約の確認で裏打ちする順序が正しい。仕組みが優れていても、契約の条件が運用に合わなければ採用できない。確認すべきは、型の定義、確信度の導出の開示範囲、版の記録方法、料金と上限の時点、言語別の注意書きである。契約面の読み違いは、そのまま精度と費用の読み違いにつながる。仕組みの章と契約の章を対にして読むことが、特化型の導入判断の作法である。以上を踏まえ、本稿ではこの事例を、定型の判断を安く速くさばくための現時点での特化策として位置づけ、生成一般の代替とは区別する。導入の可否は、自前の課題での試測と監視を前提に判断すべきである。

7.4 閾値の調整が設計の核心である

カスケードの実体は、大きなモデルをいつ呼ぶかを決める判定装置である。採点器が候補回答の良し悪しを見積もり、振り分け器が次にどのモデルへ回すかを選び、停止判定がここで打ち切るか続けるかを決める [81]。採点器が甘ければ止めるべきでないところで止

まり、厳しすぎれば止められるはずの問いまで上位へ送ってしまう。停止規則はその二つの誤りを最終的に引き受ける場所であり、閾値の設定が設計の核心になる。

選択肢は模型の並べ替えだけではない。問い方を変える工夫や、近似的な処理で済ませる手も同じ俎上に載る。問いを言い換えて安い模型でも解ける形に整える、少数の例を添えて出力を安定させる、検索や要約などの前処理で入力を短くする、といった手段は、いずれも精度と費用の交換関係を動かす。まず問いと前処理の側でできることを尽くし、それでも残る難問だけを上位へ送る順序が自然である。

費用の数え方も課題ごとに行う必要がある。入力と出力で単価が異なる料金体系では、長い生成を伴う課題と短い判定で済む課題とで有利な模型が入れ替わる [81]。机上の単価比較で決めた構成は、運用に入った途端に崩れることが多い。閾値の調整は、緩めるか締めるかの方針選択である。緩めに止めれば費用は下がるが見逃しが増え、厳しく止めれば品質は守りやすいが上位呼び出しが増える。どちらが正しいかは課題の痛みによって変わる。一度決めた閾値を固定値として扱わないことである。問い合わせの分布が変われば、学習時の選好データとのずれが生じ、振り分け器の当たり外れの勘所もずれる [82][83]。閾値は設計時に決めて終わりではなく、運用しながら直すものと考えべきである。

そのため、カスケードには監視の循環を組み込んでおきたい。止めた問いと上げた問いの比率、上位へ送った問いのうち実際に品質が上がった割合、課題別の費用と誤り率を、版ごとに眺める。ずれが見えたら閾値を動かすか、そもそも振り分けの前提が崩れていないかを確認する。逆に監視なしで閾値だけを決めると、節約の仕組みが形骸化し、大きな模型を毎回呼ぶ構成と実質的に変わらなくなる。閾値の運用こそがカスケードの本体である。

7.5 確信度との付き合い方

確信度付きの出力を扱う際の出発点は、その意味を取り違えないことである。ここでの較正は、多くの予測を束ねたときに、確信度に見合った頻度で当たりが出るという集計的な性質を指す。個々の回答について、この確信度だからこの回答は正しいという保証を与えるものではない。この点は提供側の説明自体が明示している注意であり、利用者側が勝手に読み替えて

はならない [85]。確信度は判断の材料にはなるが、判断そのものではない。

実務的な使い方は、分解して上げていく形に寄せるのが無理がない。大きな問いを小さな判定に割り、分類や振り分けや採点や抜き出しや分岐といった原子的な問いにして、組み立ては確定的な処理側で行い、確信度が低いものだけを大きな模型へ上げる。この分解と上げ方の指針は、提供側の概念説明が示す想定用途そのものである [85]。確信度はその境界を引くための物差しであり、境界の引き方は課題の責任の重さで変える。型があるから正しいのではなく、型があるから運用の手当てがしやすい。その違いを見失うと、型への安心感が精度への過信にすり替わる。

日本語を含む非英語の扱いには明確な弱さが残る。英語中心で、非英語では精度が落ちると説明されており、自分の内容で試すよう注意が添えられている [86]。多言語の注意書きを軽く見て英語での感触のまま閾値を決めると、日本語の運用で見逃しが増える。日本語の読者を想定する本調査の文脈では、この点は譲れない。日本語の実データで小さく試し、言語別の誤り率と確信度の対応を確かめてから基準を決める必要がある。較正の振る舞いについても、公開された較正手順が見当たらず、独立した裏づけは見つかっていないという限界が残る。確信度の数字が工程表に載っていても、その数字の作られ方と確かめ方が開かれていない以上、数字を額面で受け取って自動判定に直結させるべきではない。

試し方の規律も欠かせない。まず範囲を絞り、代表的な問いを集めて、確信度ごとの的中率を自分の分布で描く。全体的中率だけを見ず、確信度の帯ごとに分けて眺める。運用で流れる比率に合わせて標本を組まないと、試験で決めた閾値が現場で機能しない。別名で版が切り替わると背後の挙動が変わり得るという指摘もあり、応答に含まれる版表示を記録に残して閾値の履歴と突き合わせる工夫が有効である [86]。小さく試して小さく直す循環を回せるかどうか、確信度付きの部品を使いこなせるかどうかの分かれ目になる。

7.6 前段のさばき役としての居場所

全体像のなかでこの種の特化型の居場所を考えると、前段のさばき役がしっくりくる。すべての問いを置き換えるものとしてではなく、振り分けの前段で易しい問いをさばき、難しい問いを後段へ渡す役として

置く発想である。置き換えかさばき役かという枠組みの違いは、評価の問い方を変える。置き換えの枠組みでは平均精度の優劣が問われるが、さばき役の枠組みでは、さばいた範囲での誤り率と、後段へ渡した範囲での改善と、全体の費用と手間の合計が問われる。特化型を単体の賢さで値踏みすると見誤る。全体の流れのなかでどれだけ手間を減らしたかで見べきである。

導入の判断は、自分の分布での試験に委ねるほかな。公開の説明は提供側による報告として受け止め、独立した再現が未了であることを前提に扱う。関連する再現の試みは集約の手がかりとしてのみ位置づけられ、一件ずつの精査や査読付きの再現としては確立していない [88]。照合のための覚書的な記録は参考にはなるが、仕様の典拠として引用すべきではなく、内容の変わりやすさにも注意が要る [87]。分布のずれのもとでは振り分けの利得が条件付きになるという要約段階の注意も、この試験優先の姿勢を後押しする [83]。

費用の勘定には監視の手間も含める。閾値の見直し、標本の作り直し、版切り替わりの確認、言語別の品質点検は、いずれも運用費である。これを削って部品単価だけを数えると、安く見えて高くつく構成になる。

縮退動作の用意

適合が崩れた場合の縮退動作を用意する。強い候補への一時的な寄せ、安全側への閾値倒し、重要業務の直送区分の確保が代表例である。縮退は後退ではなく、利用者への影響を抑えるための正規の動作である。縮退の条件と手順をあらかじめ決め、適合劣化の検知と連動させる。縮退中の記録を残し、再調整の判断材料にする。縮退を恥とせず、計画された保守作業として位置づけることが、分布が動く環境での現実的な強さになる。

監視の記録項目

止めた問いと上げた問いの比率、上位へ送った問いのうち品質が上がった割合、確信度と正否のずれ、業務区分ごとの費用と誤り率を記録する。委譲率と完結率の推移、版切り替えの履歴、言語別の誤り率を添える。記録は週次や版ごとに眺め、ずれが見えたら閾値を動かすか前提の崩れを確認する。記録なき節約の主張は根拠を失う。監視

の記録こそが、振り分けの持続可能性の証拠である。

大きな模型を毎回呼ぶ必要があるかという問いへの答えは、呼び方を工夫すれば減らせるという一般論に尽きない。どこで止め、どこで上げ、どこで見直すかを決めた運用だけが、その答えを持ち帰れる [81][84][85]。

7.7 料金表の読み方

料金表は買い物の値札ではなく、振り分け設計のための作業仮説として読む必要がある。単価だけを横並びで比較すると、どの呼び出しが実際にいくらかかるのかが見えなくなる。呼び出し回数、プロンプトの長さ、生成される回答の長さ、途中で止める判定の有無、失敗時の再試行や上位模型への委譲まで含めた総量で考えなければ、机上の安さと請求時の高さが食い違う [81]。

第一に、入力と出力の非対称性に注意したい。入力の処理が相対的に軽く出力の生成が重い料金体系もあれば、入力と出力の双方に従量が生じる体系もある。この違いは些細に見えて、振り分けの結論を左右する。入力が長く出力が短い要約や分類では入力側の負担が効き、逆に説明文や長文作成のように出力が長い仕事では出力側の負担が効く。同じ二つの候補でも、仕事の形が変われば有利不利が入れ替わる。安いとされる候補が、出力の多い仕事では総額で逆転することは珍しくない。

第二に、費用は業務単位で会計する必要がある。料金表の単位量あたり価格は材料費の目安にすぎず、実際に請求されるのは業務を終えるまでに消費した総量である。代表的な依頼を数種類選び、入力と出力の実測量、委譲率、再試行率を記録し、業務あたり費用として整理することが欠かせない。全部入りの平均値ではなく、頻度の高い依頼、出力が長い依頼、品質要求が厳しい依頼といった区分ごとの費用を持つことが、見積もりの精度を上げる。

第三に、価格の陳腐化を前提とした読み方の規律が要る。公開された分析における価格は、分析時点の記録であり、現在の請求額を保証しない。料金体系は改定され、割引や上限、速度制限、無料枠の扱いも変わり得る。したがって、文献や報告書の金額部分は現在の事実として引用せず、当時の条件下での比較構造だ

けを学ぶ姿勢が求められる。価格が変われば結論が変わる可能性があることを最初から織り込み、単価の更新手順と再評価の周期を決めておく。価格表を固定値として埋め込むのではなく、差し替え可能な前提として管理する。この規律ある扱いが、古い金額に引きずられた誤った節約主張を防ぐ。

第四に、なぜ単位量あたり最安が誤解を招くのかを整理しておきたい。単位量あたり価格が最も低い候補は、一見すると多用すべき対象に見える。しかし、下位候補の回答品質が足りず、上位候補に頻繁に委譲されるなら、期待費用は下がらない。むしろ、下位候補の呼び出し分が上乘せされ、最初から上位候補に送るより高くなる場合もある。費用と精度を同時に見る方式では、同じ費用で精度を上げる方向と、同じ精度で費用を下げる方向の二つの読みがあり、どちらで比較しているのかを明示しないと節約の主張は空転する [81]。

7.8 分布が動く運用へ

振り分けは導入して終わりではなく、分布が動くことを前提に運用する仕組みである。振り分け器は過去の選好資料から強い候補と軽い候補の使い分けを学ぶが、学習に使った選好の分布と運用時の依頼分布がずれば、想定した節約や品質は保たれない。選好資料は特定の時期、特定の参加者、特定の課題集合のもとで集められたものであり、新しい業務が混ざると、勝敗予測の当たり方が変わる。この老化は避けられないため、導入時に一度測って終わりにするのではなく、鮮度を管理する対象として扱う必要がある [82]。

選好資料の老化は、静かな形で進む。急な故障ではなく、少しずつ委譲の判断が鈍り、強い候補に送るべき依頼が軽い候補に残り、あるいはその逆が増える。新しい分野の依頼が増えたとき、言い回しが変わったとき、候補の世代が入れ替わったときには、過去の勝敗の物差しがそのまま通用しない。資料収集時の課題設計や判定者の傾向も、振り分け器の癖として残る。選好資料は資産であると同時に賞味期限のある前提であり、いつ集めた何に基づく判断かを運用者が言える状態にしておくことが、分布変化への備えになる。

そこで重要になるのが、振り分け器と業務の適合度監視である。適合とは、振り分け器の予測と実業務の成功が噛み合っている状態を指す。監視の対象は、強い候補への委譲率、軽い候補での完結率、差し戻しや再試行の頻度、業務区分ごとの成功率といった、業務

の手触りに近い指標である。適合が保たれている間は節約の主張にも根拠があるが、適合が崩れた後の節約数字は根拠を失う。選好に基づく振り分けの利得は、振り分け器と業務が噛み合っているという条件のもとでのみ成り立つことが、要約段階の知見としても整理されている [83]。この条件付きの読み方を徹底し、適合度の記録と節約の主張を切り離さないことが、運用の誠実さにつながる。

再調整のきっかけは、あらかじめ言葉にしておくことが望ましい。業務区分の構成が大きく変わったとき、新しい依頼種別が一定割合を超えたとき、委譲率や完結率が管理幅を外れたとき、候補の版や系列が切り替わったとき、といった具合である。再調整とは、閾値の付け替えだけでなく、選好資料の追加や作り直し、評価課題の更新、停止判定の見直し、強い候補と軽い候補の役割分担の再定義を含む。軽微なずれなら閾値操作で吸収できるが、分布自体が動いた場合は資料と評価の更新が要る。脆弱性の知見は、欠陥の指摘としてではなく、保守信号として受け止めることが生産的である。分布変化のもとで選好型の振り分けがもろさを示すという要約段階の指摘は、振り分け導入を見送る理由ではなく、点検項目を具体化する材料である [83]。どこで崩れやすいのか、どのような業務変化に弱いのか、どのような監視があれば早期に気づけるのか。節約の主張には適合条件を添え、適合が崩れた場合の縮退動作を用意する。脆弱性を恥とせず記録し、点検と縮退と再調整を一連の環として回すことが、分布が動く環境での現実的な強さになる。なお、この節の脆弱性をめぐる記述は要約段階の範囲に留まる。運用者が持ち帰るべき要点は、細部の数値ではなく、条件付きで読む姿勢、適合を測り続ける姿勢、きっかけを決めて作り直す姿勢である。

振り分け導入の確認事項

導入前の確認として、まず入出力の内訳で見積もること。入力単価だけでなく出力側の負担を分け、対象業務の量的特徴と組み合わせて総額を試算する。次に業務別に実測すること。代表的な依頼区分ごとに委譲率や出力長の分布を取り、業務あたり費用として整理し、平均値だけで判断しない。続いて価格は記録時点のものとして扱うこと。参照日と前提を明記し、料金改定時の差し替えと再計算の手順を決め、現在の事実として固定

しない。さらに選好資料の鮮度を確認すること。いつ誰のどのような課題に基づく判断かを記録し、新分野や新世代の候補が増えた場合の更新方針を持つ。加えて適合度の監視と管理幅を決めること。委譲率、完結率、成功率、確信度と正否のずれを追い、幅を外れた場合の対応を定める。最後に縮退と再調整の手順を持つこと。適合劣化時に安全側へ倒す動作と重要業務の直送区分を用意し、閾値修正から資料更新までの再調整の担い手と順序を決めておく。

運用の問い	止める判断	上げる判断	見直しの合図
閾値の置き方 確信度の読み	緩めれば費用減と見逃し増 帯ごとの的中率で描く	厳しめなら品質保持と費用増 低い帯だけ上位へ渡す	分布ずれで勘所がずれる 言語別・版別の対応を確認
問いと前処理 費用の数え方	言い換えと例示と短縮を尽くす 入出力の量と回数を含む	残る難問だけ上位へ 課題完遂あたりで比べる	前処理の改善が先 単価表の一律優劣は捨てる
監視の循環	止めた割合と改善率を眺める	前提崩壊の有無を確かめる	形骸化の前に閾値を動かす

[81, 82, 83, 85, 86]

呼ばないための設計	仕組みの要点	条件と留保
段階的な引き継ぎ 学習型の振り分け 振り分けの脆さ 型付きの特化判断 速度・費用の主張 確信度の扱い 外部の覚え書き 再現の集積地 較正の手順	安価な順に試し不足分だけ上位へ 好みデータから勝敗予測を学習 分布ずれで当たり外れが変わる 並列標本と確信度つき選択 自社評価での優位の提示 集計としての較正の意図 一致の記録 手がかりの収集段階 公開手順が見当たらない	当時の料金体系の報告 分布の古びに依存 要約範囲、一般化しない 生成の代替ではない 提供元主張、再現未確認 個の正しさの保証ではない 仕様の典拠にしない 確立した裏付けなし 読み替えに慎重

[81, 82, 83, 84, 86]

本節の読解上の境界

料金は文書時点の記載であり現在価格に当てはめない。脆さの条件細部は未確認として一般化しない。特化型は現時点での特化策であり、生成一般の代替として語らない。独立再現と較正手順の確認までは代替として扱わない。

8 2026 年の実装点

2026 CAPSTONES

この節は四つの実装点を順位付けしない。優劣表を作ることが目的ではなく、各実装がどの層で計算と記憶と通信を削っているかを機構の積み重ねとして読み解くことが目的である。比較軸は総パラメータと作動パラメータ、注意機構と記憶保持の設計、数値精度、復号の直列性、サービングと実行環境、手元での実行可能性、文書化された代償、そして証拠の質である。四者の証拠の深さはそろっていない。本文まで構造的に読み込めた報告もあれば、要旨の範囲に限定される報告もある。ベンダー側の比率や速度表現は測定条件

付きの主張として扱い、独立検証のない数値は事実として引用しない。実装リポジトリは存在証明の水準にとどめ、コード監査や独自の再実行は行っていない。手順書類は更新でずれる生きた文書であることを前提にする [90, 91, 92]。

8.1 大規模疎活性化と記憶圧縮を重ねた設計

一つ目の実装点は、552B の総量に対して、入力では 8B、出力では 16B を作動させる非対称な構成を土台に、注意と記憶の保持を同時に圧縮する設計である [90][93]。この入出力の非対称は文書化された Causal Encoder-Decoder 設計の一部であり、汎用的な疎活性化のみによるものではない。復号時と事前充填時で作動量を使い分ける条件付き計算、疎な注意の複数動作様式の共有、学習時から低ビット化された大域記憶、局所窓と再生機構による持続記憶の削減、条件付き記憶の追加が重ねられている。チェックポイントの同一性表示は、因果符号化復号化器や第二世代の疎注意、条件付き記憶、推測復号の構成を含むものとして、模型面の文書で示される [93]。

数値精度の結び付きがサービング要件を決める。専門家部分と密部分と埋め込みで精度体系が異なり、低精度経路は対応する加速器と実行基盤を条件とする。復号の高速化は単一の推測復号方式に置き換えられ、従来の多トークン予測部は当該チェックポイントでは用いない設計選択となっている。配備手順は集約配置と分離配置の双方を試験範囲として文書化する [94]。推論努力量の数値対応や特定加速器での検証拒否時の代替動作まで含むが、手順書は更新される生きた文書であり、特定構成での検証は他構成や他画像に転用できない [79]。受容率の合成測定は測定値であり、実サービングの棄却動作とは区別する。実行文書の対応は、模型符号の文書として伝えられる [80]。

8.2 残差と最適化と疎注意を共同設計した構成

二つ目の実装点は、百 B 級強の総量に対して作動量を一桁 B 級前後に抑えつつ、ホスト側に置かれる表を組み合わせる構成である [91]。報告本文の範囲で読み取れる要点は、候補変更を損失と指標、学習と事前充填と復号の費用、安定性への影響の三軸で評価する姿勢、分岐と素子ごとのゲートを持つ残差変形、表の配置と語彙配分の検討、対象パラメータごとの最適化手法の割り当てである。損失と下流精度が乖離し、表の拡大が損失を単調に下げても精度は飽和するとい

う観察が共同設計の主張を支える。設定表は層様式の比率や単一の予測層の存在を特定するが、自己報告の性能表は条件検証を待つ主張として隔離する。公式解説は刷新の枠組みを示す告知水準であり、実質は報告本文の裏付けに依拠する [95]。実装集積所は派生版の区別を結び付け、運用版とプレビュー版の混同を防ぐことが引用上の要点である [96]。注意機構の設定や評価値は、公開の読み物としても示される [28]。

実行側は、特定加速器での配置報告と地域計算環境への移植の存在で裏付けられる。加速器上の報告は接頭辞一致率を条件とする配置の存在証明としてのみ扱い、倍率表現はキャッシュ局所性の測定でもある点に注意して事実化しない [97]。報告面の取得は行っておらず、数値の独立検証やハーネス分解も未了である。地域計算環境への移植は実装存在の権威であり、変換器や低ランク結合を含む文グラフ、多数専門家からの上位数選択、予算付き疎注意、ホスト側の表処理などが実装記述として確認される [57]。検証方法や既知の実装上の隙間も報告者の記述として透明化されているが、独自の再構築や測定は行っておらず、移植の正しさは報告ベースの検証に依拠する。

8.3 線形注意系の実行可能性

三つ目の実装点は、注意の二次計算を線形系の記憶更新に置き換える系列に属し、互換性と運用面の配慮を前面に出す設計である。実装集積所は核計算と実行統合経路とチェックポイントの公開を一体として示し、報告を超える配備権威の役割を担う [98]。ただし集積所の複製や監査は行っておらず、独立再現は未実行である。核実装と実行経路の組み合わせは、報告を超える実行可能性の証明として配置水準で評価するが [30]、核の数値計算やサービング性能は未消費であり事実として引用しない。名称衝突の可能性がある別系列の機構とは切り分け、混同しない。運用者の受容報告は補助的な位置付けにとどめ、技術的事実や経済的事実の根拠には用いない。

8.4 要旨範囲に限定される四つ目の実装点

四つ目の実装点は証拠の扱いが異なることを最初に明示する。技術報告については要旨のみを消費し、本文の採用詳細や強化学習基盤の設計や評価表は未消費である [92]。したがって本文で裏付けられた言語では記述しない。数百 B 級の総量に対して作動量を数十 B 級前後に抑える基礎模型として、他所発の疎注

意機構の採用と言語化されるが、その意義付けは要旨範囲の採用事実にとどまる。混合深度として、チェックポイントの同一性表示 [99]、開発者文書の相対比率 [100]、実装集積所の共有機構と予測層と言及される配置表を組み合わせる [101]。開発者文書の計算量や記憶量の相対比率は、基準を名指したベンダー相対の測定として条件付きで扱い、測定条件は未確認として隔離する。集積所の数値は集積所報告を超えて裏付けない。未解決のプール名称は事実には用いない。模型面の更新表現や順位表現は二次的として事実化しない。起動時の出来事は告知水準として位置付け、抜粋要約の範囲を超えない [41]。

手元実行は量子化物の配布と記憶適合指針の存在が要点であり [49]、量子化品質表や速度主張は提供者測定として独立確認を待つ。横断すると、復号高速化の置換判断、手元実行の現実性、効率から価格への波及の三点が代償と証拠の質を示す。推測復号の方式置換は意図的な設計選択であり、受容に関する数値は特定負荷での測定に限定される [78]。生きた手順書の変動、合成受容と実棄却の区別、条件付き比率の隔離、再現の未了を前提にすれば、四者を機構の選択の違いとして理解できる。順位ではなく、どの隘路を何で削り何を代償にしたかが結論である。

四つの実装の要点对照

DeepSeek 系は条件付き計算と記憶圧縮の層化で総量と作動量を分け、部位別精度と手順書の検証範囲で運用条件を縛る。Qwen 系は残差ゲートと表参照の共同設計で計算と参照を分担し、損失と精度の乖離を設計の根拠に据える。Kimi 系は注意の保持形式そのものを線形記憶へ置換し、互換優先と実装存在で配置可能性を示す。GLM 系は要旨限定の深さを守り、カード・文書・リポジトリの混合深度とベンダー比率の隔離で比較に供する。四者に共通するのは、作動量の設計思想の違いと、証拠の深さに応じた言葉の選び方である。

8.5 作動量の設計思想の違い

総量と作動量の分離は、この年の実装点に共通する関心であるが、その切り分け方は各設計で異なる。DeepSeek 系の Flash は、符号化時と生成時で働く量を変える条件付き計算を中核に据える [90][93][79]。常に全体を動かすのではなく、場面ごとに必要な部分だ

けを作動させる思想であり、報告書本文と配備用手順書の双方で構造として確認できる。この選択は、常駐と通信の設計に直結する。動かさない部分の重みや記憶をどこに置か、分離配置における鍵値受け渡しをどう担うか、専門家並列の振り分けをどの実装経路でさばくかといった配備条件が、作動量の思想と一体で記述される [94][79][80]。作動量を絞ることは単なる節約ではなく、置き場所と運び方の再設計を要求する。

Qwen 系の次世代 Flash は、残差経路のゲート制御による絞り込みと、表参照による補完を組み合わせることで、総量と作動量を分ける。重みとして保持する知識と、表として引く知識を別系統に置き、計算で解く部分と参照で済ませる部分を分担させる共同設計が本文で主題化されている [91][28][95]。表は主記憶とは異なる場所に常駐し、必要に応じて引かれるため、加速器側の作動量は抑えられる一方で、ホスト側の表保持と変換時の作業記憶が実装上の関心となる。この振る舞いは移植報告における検証記述からも読み取れ、表の行照合や分割前処理の同一性確認が移植正当性の柱に置かれている [57][96]。計算を減らす代わりに置き場所を増やすのか、置き場所を集約する代わりに計算を許容するののかという分岐が、ここでは明確に表の有無として現れる。

Kimi 系の線形系は、作動量の重心を注意の選択そのものから外す方向を選ぶ。二次的に増える注意行列を保持・移動させるのではなく、漸化的に更新される状態に記憶を畳み込むことで、層ごとの選択的作動という発想を置き換える。公開リポジトリがカーネル実装と統合経路、チェックポイントの公開を一体で示すことは、この思想が実装可能性と切り離せないことを物語る [98][30]。常駐するのは注意の断片ではなく圧縮された状態であり、通信するのは点数行列ではなく状態更新に必要な最小限の情報となる。ただしカーネル実装の数値や性能は未消費であり、思想としての方向性と配備可能性の存在は語れても、運用像の断定には進まない。この留保は欠落ではなく、証拠の深さに応じた言葉の選び方である。

GLM 系の Flash に関する相対比は、隔離されたベンダー主張として扱い、事実としては引用しない。注意計算量や鍵値量の削減を示す相対表示は、測定条件が未確定の文書面での記述に留まるため [100][101]、本稿では削減幅の多寡を語らず、相対比が単独では設計思想の比較材料にならないことのみを確認する。推論努力量の段階制御も、作動量とは別軸の設計であ

る。手順書が示す努力量の概念は、復号の振る舞いや推論の深さを運用時に切り替えるためのもので、重み側の条件付き計算と混同できない [79][78]。いずれの設計も優劣で並べず、分離の仕方とその代償の置き方の違いとして読むことが、本節の契約である。

8.6 注意と記憶の選択の違い

疎な索引型、予算付き疎型、線形記憶型、他所発機構の抽象採用型という四つの注意選択は、何を残し何を動かすかという点で対照的である。DeepSeek 系の Flash が採用する疎な索引型は、大域の鍵値を残したまま索引機構で選ぶ方式である [90][79][80]。残すのは全体であり、動かすのは選ばれた分だけである。ここに窓付き注意と再生機構が加わる。近傍窓で局所性を担保しつつ、再生機構で永続的な占有を抑えることで、長期文脈における常駐の増大を構造的に抑える。圧縮を担う層を限定する設計も、同様の発想の延長にある。すべてを一律に圧縮するのではなく、効果の見込める箇所に限定することで、保持と移動の総量を抑える。

Qwen 系の予算付き疎型は、予算という物差しで密と疎を切り分ける。予算内に収まる範囲では密計算と等価であることが移植報告の検証で確かめられ、予算を超えた範囲で選択的注意に移る [91][57][97]。残すのは層全体の表現力であり、動かすのは予算で区切られた有効分である。この方式の帰結は、接頭辞一致率への依存として現れる。ベンダー側的高速化表示が接頭辞一致を条件とするのは、注意機構の測定であると同時に局所性の測定でもあるからであり、本文脈では数値の多寡を引用せず、条件付きであるという読み方だけを残す [97][95]。予算付きであることは、無条件の削減ではなく、文脈の再利用のされ方に依存する削減であることを意味する。

Kimi 系の線形記憶型は、注意を残さない選択である。鍵値を残して選び直すのではなく、状態に畳み込んで逐次更新することで、記憶の保持形式そのものを変える。第三者実装としてのカーネル実装と統合経路の存在は、この置換が文書上の提案に留まらず実行可能な形を持つことを示す [98][30]。ただし数値的性質や提供性能は未消費であり、保持形式の転換という設計事実と、運用品質の確定という事実を区別する。GLM 系における他所発機構の採用は、移植可能性の事実として読む。DeepSeek 系に由来する機構が別系統の旗艦に取り込まれたことは、機構の可搬性を示す

独立した傍証であり、報告書要旨の範囲で確認できる [92][101]。保持と移動の内訳に踏み込んだ記述はここでは行わず、採用という選択の存在とその意義に限定する。四者は序列ではなく、残すものと動かすものの配分の違いとして並置される。

8.7 証拠の深さの違いと読み方

本文消費、要旨限定、告知水準、移植報告という四つの深さは、許される言葉の範囲を変える。本文まで消費された資料は、構造と実装条件を語ることができる。DeepSeek 系の報告書本文における条件付き計算や疎な索引の構成、推論後学習の工程の骨格は、構造的範囲での読解を許す [90]。Qwen 系の報告書本文における残差ゲートや表配置、最適化手法の割り当て、損失と精度の乖離という共同設計の主題も、同様に本文読解としての言葉を許す [91]。配備手順書を全文消費した場合も、検証済み配置や推論努力量の意味づけといった運用条件を事実として語れる [79][94][80]。ただし、いずれの本文消費においても、個別表の数値や詳細な除去実験の内訳、評価基盤に結びついた検証を欠く比較表示は引用しない。構造を語れることと、数値を事実として語れることは別の事柄であり、本稿では前者に留める。GLM 系については報告書要旨のみを消費した範囲で論じ、本文の詳細や学習基盤の設計、表の内訳に踏み込んだ断定は行わない [92]。

告知水準の資料は、方向と構想の表明として読む。公式発信における刷新点の枠付けや、前任者との対比による向上の言い回しは、ベンダー側の枠付けであって測定事実ではない [95][97][41]。加速器上的高速化表示も、接頭辞一致などの条件と一体で読むべき局所性依存の表示であり、条件を離れた効率の事実としては引用しない。移植報告水準の資料は、実装存在の証拠として読む。共同体移植における変換器や疎注意の実装、表参照のホスト側処理の記述は、実行可能性の存在を示す [57][96]。量子化文書における形式一覧や適合指針の記述も、同様に存在と手順の証拠である [49][98][30]。ただし移植正当性は報告者の検証条件に結びついた検証に依拠し、独立再構築や独立測定を経ていないため、正確性の断定には進まない。量子化品質の表示は測定者限定の表示であり、高速化表示はベンダー測定であって確認を要することが境界として残る。

未解決の固有名は事実として用いない。GLM 系リポジトリに現れる未解決の共有資源名は、同一性が

定まらないためいかなる事実にも引用しない [101]。本稿でいうカーネル実装は線形系に属する実装を指し、同名の別分野の機構との混同をあらかじめ排除する [30][98]。手順書の変動も読み方の一部である。配備手順は更新を前提とする生文書であり、固定された構成での検証は他の構成や同時実行上限に転用できない [79][94]。合成的な受容と実際の棄却の区別も同様である。処理能力測定様式における合成的な受容は測定に留まり、実提供を支配するのは実際の棄却の挙動であるため、受容数の多寡を運用品質として引用しない [79][78]。深さに応じた言葉を守ることが、順位付けを排して実装点を読むための条件である。

8.8 DeepSeek 系 Flash の読み方

DeepSeek 系 Flash は順位で読む対象ではなく、疎な活性化と記憶圧縮の重ね合わせとして読む対象である。報告書面とカード面と手順書面の深さが非対称であることを最初に固定し、仕様の事実は報告と手順書の相互補強で支え、図表に属する数値は事実として引用しない読み方が必要である [90][93]。ベンダー側の発表文言を測定事実と混同せず、HOST側への退避や分離配置の条件を実装記述として切り分け、リポジトリや手順書に記録された配置検証の範囲を越えて一般化しない [94][79]。

疎な活性化の読みどころは条件分岐型の活性化である。事前充填時と復号時で活性規模の扱いが分かれ、復号時の持続活性と事前充填時の活性が別条件として仕様化されている点が重要である [90][93]。大規模混合専門家構成の全体規模と実際に働く部分の規模を混同せず、全体が大きいことと毎回働く部分が小さいことを別事実として保持する。カード面はこの分離をチェックポイントの同一性として固定し、報告面は機構設計として条件分岐型活性化の位置づけを与える。すなわち活性規模の分離は単なる効率の飾りではなく、後段の精度別条件や配置検証と連動して運用可能条件を決める前提である。

記憶圧縮の読みどころは単一手法ではなく層化である。疎な注意機構は全体再利用と再索引と再利用の三態様を備え、全体の鍵値と索引鍵を共有する設計として報告面に構造消費されている [90]。学習時に低精度全体鍵値を導入しても劣化が限定的であるという枠付けは報告の枠内での主張として受け止め、独立検証済みの事実で格上げしない。そこに有界再生を伴う狭域注意の層が重なり、持続的な占有の削減が別の層とし

て積み上がる [90][93]。全体鍵値の圧縮は長期文脈の保持費用に対応し、有界再生は持続的に残る部分の上限に対応する。両者を足し合わせた単一の削減率として読むのではなく、どの層がどの費用を抑えるのかを分けて読むことで、配置ごとの振る舞いの違いが説明可能になる。

精度別の提供条件は部位ごとに精度を使い分ける設計として読む。専門家部分と密部分と埋め込み部分で精度の割り当てが異なり、低精度経路が配置条件と結びつく [93][79][80]。特定のバックエンドでのみ低精度経路が成立するというバックエンド条件付きの理解が不可欠であり、バックエンドが揃わない配置に同じ条件を転用しない。集約配置と分離配置で文書化された経路が異なり、Dynamo 面の記録は検証済み構成に限定された存在として扱う [94]。HOST側への退避配置は検証範囲として記録された実装記述であり、万能の記憶節約策として一般化しない。手順書に属する手順の検証は固定された構成での証拠に留まり、他の構成への転用は成立しないという生文書の限定を守ることが、精度別条件の誤用を防ぐ。

投機復号との対応付けは置換として読む。従来の多トークン予測モジュールを廃し、DSpark のみを投機手段とする意図的な置換が要点である [78][79]。半自己回帰的にブロック単位で草案し、振り分けられた専門家群と偏り補正と確信度ヘッドを組み合わせ、チェックポイント内で草案する構成として手順書面に消費されている [78]。適応的検証と全ブロック検証の分岐は配置条件付きであり、特定の加速器面では適応的検証を行わず全ブロック検証に固定する条件が明示されている。この条件分岐を他設計に一般化しないことが重要である。受容に関する数値は紙面級のアブレーションではなく作業負荷測定に留まるという限定を厳守し、合成的な受容の測定値を実際の提供の棄却挙動と混同しない。実提供を統治するのは実際の棄却を伴う評価であり、処理量測定用の合成的な受容は測定に留まる [79][78]。

努力量操作は数値化された推論努力量として読む。低段階と高段階と最大段階の区別があり、応用界面との対応にずれがあることが手順書面に実装挙動として記録されている [79]。努力量を上げれば品質が単調に上がるという単純な理解ではなく、努力量と最大トークンと終了理由の相互作用として読むことが必要である。ベンダー側の効率文言をそのまま運用約束に読み替えず、HOST側の資源条件とリポジトリに固定さ

れた手順の検証範囲で運用可否を判断する姿勢が、DeepSeek 系 Flash の実装点を読む際の基本である。

8.9 Qwen 系 Flash-Next の読み方

Qwen 系 Flash-Next は残差ゲートと表の共同設計として読む対象であり、順位で読む対象ではない。公式報告面とカード面とリポジトリ面と移植面の深さが異なり、各面が担う事実の種類を混同しないことが重要である [91][28][96]。ベンダー側の向上文言を測定事実
に格上げせず、ホスト側に置かれる表の記憶挙動を実装記述として切り分け、リポジトリに記録された版境界と実行時対応表を越えて数値を混ぜない読み方が求められる。

残差ゲートと表の共同設計が第一の読みどころである。候補変更を損失と評価と費用と安定性の三軸で評価するという設計姿勢が報告面の構造として消費されており、ゲート付き残差や表配置や語彙割当や最適化手法の割当の節がその具体化である [91]。層様式として密なゲート層と疎注意層の交互がカード面に固定され、単一の多トークン予測層が付随する構成が裏付けられている [28][91]。ホスト側に退避される表は単なる付属物ではなく、語彙側の表と注意側の疎化が連動して設計されている点が共同設計の核心である。細部のアブレーション表や評価の細目が未取得でありベンダー側比率が隔離対象であるという限定を守り、構造の存在と設計意図の読みに留める [91]。

損失と精度の乖離の観測が第二の読みどころであり、この設計の理論的主張を支える。表語彙を増やすと損失は単調に下がるが精度は飽和するという所見が報告面の本文消費として与えられ、共同設計の主張として位置づけられている [91]。損失だけを見て規模拡大を正当化する読み方も、精度だけを見て表を無用とする読み方も、いずれもこの観測を捉え損なう。公開ブログ面の向上の枠付けは発表面の枠付けに留まり、微細ブロック化された疎注意や最適化手法の割当の実質は報告面の裏付けに依存する [95][91]。前任者との対比の向上言語をベンダー側の枠付けとして扱い測定事実
に引用しないことが、乖離の観測を正確に保つ条件である。

前提条件付き存在としての加速器面の記録が第三の読みどころである。特定加速器集合上での事前充填と復号の利得や前任者対比の倍率は、前提条件一致率付きの存在としてのみ読むべきものである [97]。前提条件が一致した状態の測定は注意機構だけの測定ではな

くキャッシュ局所性の測定でもある。ブログ本文未取得であり図表が独立検証や足回りの分解を経ていないという限定を厳守し、数値を事実
に引用しない。版境界の理解も同じく重要であり、プレビュー版と本番応用界面版の文脈長や道具利用の区別がリポジトリ面に束縛されており、版をまたいだ数値混合を防ぐことが正しい引用の前提である [96]。実行時対応表の言及はリポジトリ面の被覆範囲として存在するが、本文未取得であり細目は未確定として扱う [96]。

移植報告による検証が第四の読みどころである。共通推論基盤への移植は変換器や低ランク結合を伴うテキストグラフや多数の専門家構成や疎注意やホスト側表のハッシュや既存の視覚経路を含む実装の裏付けとして統合されている [57]。検証方法は困惑度照合や文章一致や予算内での密な一致や予算超過時の類似度や表の行索引やチャンク化された事前充填のビット同一性や構成試験の合格として移植報告面に記録され、検証条件として明示されている [57]。既知の実装上の隙間も移植議論面に透明化されており、マイクロバッチをまたぐ畳み込み状態の PR 内修正とビット同一性の証明や保存経路と鍵の再送の修正や量子化モデルでの不一致ロジット差や共有ファイル変更の追加分のみの限定とハッシュ照合が含まれる [57]。移植の正当性は移植報告の検証に依存し独立した再構築や独立測定を経ていないという限定を守ることが必須である [57]。ホスト側の表記憶挙動として大規模な表や分割やハッシュやストリーミング変換器やピーク回避が実装記述として与えられているが、これも実装記述の範囲を越えて事実化しない [57]。

8.10 Kimi 系 Linear の読み方

Kimi 系 Linear は線形記憶置換を互換優先で実装した設計として読む対象であり、順位で読む対象ではない。報告を越える配置の裏付けがリポジトリ面に存在すること、カーネルと経路の実装存在が第三者実行可能性を与えること、独立再現が未了であることの三点を同時に保持する読み方が必要である [98][30]。ベンダー側の効率文言を測定事実
に読み替えず、リポジトリに記録された公開範囲を越えてコード監査済みであるかのように語らないことが、この設計の誤読を防ぐ。

線形記憶置換は注意記憶を線形記憶で置き換える設計として読む。二次費用の記憶を線形費用の記憶に置き換えることで長期文脈の保持費用の構造を変える意

図があり、互換優先設計と一体で理解すべきものである [98][30]。置換を単なる近似や圧縮として読むのではなく、記憶の置換が提供条件や実行時対応の前提をどう変えるのかという観点で読むことが必要である。公開チェックポイントが第三者による公正比較再現の経路を開くという指摘は、この置換が報告上の主張に留まらず配置可能な主張であることを示すが、現時点で独立した柱を欠くという限定と対で保持しなければならない [98]。数値や性能を事実引用せず、置換の存在と設計意図と公開範囲の読みに留めることが、順位付けしない読みの基本である。

互換優先設計は既存の実行時や界面を壊さずに置換を導入する姿勢として読む。リポジトリ面がカーネルと統合経路とモデルのチェックポイントを公開し、報告を越える配置の裏付けを構成している点はその具体化である [98]。第三者が実行可能な実装として存在することが、互換優先の証拠である。コード未取得・未監査であり独立再現未実行であるという限定を守り、公開の存在を検証済みの正当性と混同しない [98][30]。

カーネルと経路の実装存在は配置可能性証明として読む。第三者実行可能なカーネル実装と経路の対が報告を越える証明を構成し、単なる提案ではなく実行可能な設計であることを示す [30][98]。カーネルの数値面や提供性能面は未消費であり事実引用しないという境界を厳守し、存在の確認と数値の確認を明確に分ける [30]。存在の確認は公開面の事実であり、数値の確認は別途の検証を要する事実である。

同名衝突排除は証拠衛生として明示的に行う。本稿というカーネルとは Kimi 系 Linear に属するカーネル実装とその経路を指し、同頭字語を持つ無関係の他実体を指さない [30][98]。頭字語の一致をもって機能や性能や来歴を混同しないことが、引用の正当性を保つ条件である。再現限定は現在進行形の不足として読む。公開チェックポイントが公正比較再現の経路を開くことは、不足を解消するための条件が整いつつあることを意味するが、不足自体が解消されたことを意味しない [98]。第三者による再現が実行されていない現状では、主張の独立した柱は不在であり、引用は公開面と実装存在面の範囲に留まる [98][30]。ベンダー側の主張を隔離対象として扱い、リポジトリに固定された公開バイトのみを根拠にする読み方が、Kimi 系 Linear を順位付けなく正しく位置づける道である。

8.11 GLM 系 Flash の読み方

GLM 系 Flash は深さの非対称性を明示して読む対象であり、順位で読む対象ではない。基礎報告面は要旨限定であり、カード面と文書面とリポジトリ面は混合深度であり、ベンダー側比率は隔離対象であり、未解決名は事実引用しないという四点を同時に守ることが必要である [92][99][100][101]。手順書面の記録を手順書面の深さとして扱う読み方が求められる。

要旨限定の守りを改めて述べる。基礎報告面は要旨消費に留まり、本文全体の注意採用細目や強化学習基盤設計や評価表は未消費である [92]。したがって本文全体の検証済みであるかのような言語を用いないことが必須であり、基礎モデルの注意機構の役割は要旨文脈の事実としてのみ扱う [92]。他研究室起源機構の旗艦採用が移植可能性の傍証になるという意義付けは採用事実の意義であり、機構細目の検証ではない。要旨限定は一時的な不足ではなく本稿の証拠境界であり、この境界を越えた言語を用いないことが GLM 系 Flash の読みの前提である。

カード面と文書面とリポジトリ面の混合深度が第二の読みどころである。カード面はチェックポイント同一性面として系譜タグや許諾や低精度版や構成タグや兄弟版の存在を固定し、来歴の継承を裏付ける [99]。カード掲載の向上言語や評価言語は二次的として事実引用しないという境界を守る [99]。文書面は相対比率の枠付けとして注意計算量や鍵値量の削減を基準明示で与えるが、測定条件未確定として未取得である [100]。リポジトリ面は配置表や共有機構や多トークン予測の受容率向上をリポジトリ報告として文書化するが、リポジトリ未取得でありリポジトリ報告を越えた確認を欠く [101]。三面の深さを均一化せず、同一性は同一性として、相対比率は相対比率として、配置表は配置表として別々に読むことが必要である。ホスト側の記憶適合の案内も、リポジトリ面や文書面の深さを越えて検証済み事実に格上げしない [49][101]。

ベンダー側比率の隔離が第三の読みどころである。文書面の相対比率もリポジトリ面の共有機構や受容率向上の図も、地域量子化面の品質表や速度表も、いずれもベンダー側測定や足回りに縛られた報告として事実引用しない [100][101][49]。削減倍率や受容率向上分や品質表の最上位値や記憶適合の目安は、測定条件や足回りや量子化条件に縛られた存在であり、独立検証を経ていない [100][49]。二面間の一致をもって独立

検証の代わりにしないことが重要である。地域量子化面の存在確認として低ビットから高精度までの量子化の利用可能性や記憶適合の案内や共通推論基盤のフォーク経路やデモ設定の存在は確認できるが、品質数値の検証は未了でありフォーク内容の監査も文書記述を越えない [49]。速度の主張はベンダー側測定として独立確認を要するものに留め、評価表は足回りに縛られた報告として全体を隔離対象とする [49]。

未解決名の扱いが第四の読みどころである。共有プール系の名称は未解決であり事実引用しない [101]。解決を続けるかは別途の判断に委ね、本稿では未解決であることを明示して欠測として残す。未解決名を用いて機構の優劣や系譜を語らないことが、証拠衛生の基本である。ベンダー側の記述に未解決名が登場しても、それを確定名称として後段に転用しない。

初出発表面の扱いが第五の読みどころであり発表面

の深さとして読む。効率機構の初出は価格改定や応用界面公開や重みと報告とカーネルの公開を伴う日付付き伝達事象として抜粋要約で検証されているが、全頁消費を経ていないため発表面の深さに留まる [41]。計算節約の価格転換という経済面の意義は事象の意義であり、GLM 系 Flash 固有の機構検証ではない。この初出面を GLM 系の機構証拠に直結させず、他研究室起源機構が旗艦に採用されたという研究室横断の採用の文脈理解に留める [41][92]。要旨限定の守りに戻れば、GLM 系 Flash の注意採用細目は未消費であり、初出面の発表記録をもって細目検証の代わりにしないことが必須である。ベンダー側の発表とホスト側の地域実行案内とリポジトリの配置記録を別層として重ね、深さの非対称性を保ったまま比較に供する読み方が、GLM 系 Flash の実装点を誤りなく位置づける道である [92][99][100][101][49][41]。

比較の軸	一つ目の設計	二つ目の設計	三つ目・四つ目
総量と作動量	総量 552B / 入力 8B・出力 16B	百 B 級強と一桁 B 級前後	線形系・数百 B 級と数十 B 級前後
注意と記憶	複数様式の共有と窓再生	予算付き疎注意と表	実行経路・要旨範囲の採用
数値精度	部位別の精度体系	表と残差の共同設計	核の数値未消費、相対比率
復号	単一推測方式への置換	移植報告の検証	要旨範囲の置換の限定
運用と実行	集約・分離の手順書	加速器配置と移植	集積所の存在証明
手元の可能性	手順書の対応	移植の存在	量子化物の配布
証拠の深さ	本文と手順の裏付け	本文と移植の裏付け	配置水準・要旨限定

[90, 91, 98, 92]

本節の読解上の境界

順位付けは行わない。四つ目の技術報告は要旨のみの消費であり本文言語では記述しない。ベンダー比率・速度表現は条件付き主張として隔離する。手順書は生きた文書であり他構成に転用しない。未解決の名称・衝突は事実引用しない。

9 数字をどう読むか

MEASUREMENT LITERACY

推論の速さは一つの数字では語れない。まず応答の出だしまでの待ちと、生成が続く区間のトークンあたりの間隔と、単位時間あたりの出力と、同時実行や到着率の負荷をかけたときの全体処理量を分けて捉える必要がある [102, 103]。文脈が長くなれば出だしの読み込みが重くなり、同時実行を束ねれば待ち行列の振る舞いが変わり、装置が変われば同じ模型でも数字が動く。推論に割く手間を増やせば出力が長くなり、単位時間あたりの量の見え方も変わるため、手間の指

定を欠いた速さの比較は成り立たない。計測文書に即した道具立ては、これらの指標を負荷条件とともに記録することを求める [103]。同時実行数や要求の送り方、計測区間の切り方、対象装置の特定を欠いた毎秒あたりの量は、再現の手がかりにならない。文書上の注意書きが多い計測枠組みは、数値の出所としてではなく、比較が壊れる理由を教える教材として扱うのが適切である [104]。後継として名指しされた存在についても、単独の製品としての姿は確認できておらず判

断を保留し、断定しない。

9.1 実課題の系譜を読む

実課題の解決力を測ろうとすれば、現実の開発現場にある課題報告とそれを確かめる試験に行き着く [105]。ただし初期の集め方には壊れた課題や仕様の曖昧さが混ざり、試験を通っても直したことになる場合や、何を直すべきか定まらない場合が残る。この妥当性の綻びを直視し、人手で選び直した精選版や、長期にわたる作業を想定して作り直した難所版が生まれたのは自然な流れである [106]。難所版では人手で確かめられた課題が複数の保管庫から集められ、公開用と商用と非公開の区分が設けられて、混入や漏えいへの備えが意識されている。ここで守るべき作法は、区分の異なる数字を混ぜて並べないことである。公開の場で見える成績と、別条件の区分の成績は、集め方も漏えいの備えも異なるため、素朴に比べられない。個々の足回りの結び付きの特定は留保されており、本文の詳細な手順の読み込みも残されている。要旨に即した設計の理解にとどめ、現物の再実行による裏取りに踏み込まない。したがって個別の数値は隔離し、ここでは妥当性をめぐる設計の系譜だけを学ぶ。見かけの順位を新たに作らないことが、この節の約束である。

端末に向かう代理作業の評価は、応答文の見栄えではなく、操作を終えた後の容器の状態で合否を決める点に特徴がある [107]。ただし同じ名を掲げていても、改訂を経れば課題集合や判定手順は別物になり得る。名は契約ではないというのが、版の違いが教える第一の教訓である。別の注意例として、評価の足回りの選び方で成績が動くことが記録されている。複数の足回りのうち良い方を採る流儀が残れば、数字は実力だけでなく足回りとの相性をも映す。標準化された正確さ側の枠組みが担うのもこの役割であり [108]、枠組み名を欠いた正確さの言及は比較の土台を欠く。現物の保管庫や課題設定の細部までは立ち入らず、ここでは個別の数値を引用しない。

9.2 長さや報告の構え

長い文脈をうたう表示と、実際に使える長さは別の事柄である [109]。段階的に長さを上げながら、取り出しや飛び越しのある照合、まとめ上げ、問い応答を等級づける考え方は、このずれを可視化する。有効な長さを等級づけることは、能力の物差しであると同時

に、記憶と移動の費用を問う効率の物差しでもある。本文の課題群や採点手順の細部の読み込みが残る以上、等級づけの内訳に踏み込んだ断定は避け、要旨に即した理解にとどめる。一方で広い評価を志す立場は、場面と指標と適用条件を掛け合わせ、多面的な報告の透明性を重んじる [110]。販売側の表を検証する側の物差しもここにあり、条件の開示を求める姿勢が共通する。保持の費用と読み込みの重さを見据え、長い文脈の表示をそのまま実力と受け取らないことが肝要である。

数字を並べる前の作法を束ねる。第一に、利用場面の束ねを欠いた比較をしない。第二に、来歴の異なる区分を混ぜない。第三に、版と足回りや道具利用を添える。第四に、混入と課題完遂の費用を見る。出だしと生成中と全体を分け、版と足回りや道具と手間を添え、区分を混ぜず、混入と総費用を見る。この確認を欠いた数字は並べない。新たな順位付けを作らず、条件付きで読む姿勢だけを持ち帰る。

効率化の主張を読むとき、見出しの数字だけを取り出して並べる読み方は成立しない。本特集で扱う数字は、生成速度や遅延や同時実行の条件、文脈長の測り方、装置の素性、推論に投じた思考量、ベンチマークの版、足場や道具への接続の有無、混入への備え、課題を最後まで解き切るための総費用といった、測定を支える条件と一体でしか意味を持たない。条件を欠いた数字は比較不能であり、条件を添えた数字はじめて検討の対象になる。見出しの大小ではなく、どのような手続きで何が測られ、何が測られていないのかを問い、同じ土俵の数字だけを同じ土俵で読む。そのために、代表的な物差しの読みどころを四つの見取り図に整理する。各見取り図は、個別の得点や件数を覚えるためのものではなく、次に新しい数字に出会ったときに持ち帰れる確認項目を身につけるためのものである。

SWE-bench 系の見方

課題解決型の評価を読む際は、まず報告と検証の手続きに目を向ける。課題文と対象リポジトリと修正候補と試験群が一組になり、修正前には失敗し修正後には成功すべき試験と、修正の前後で成功し続けなければならない試験の二系統で検証する。この二系統の組み合わせが、表面的な修正の見かけ倒しを抑え、既存機能の破壊を見逃さない

ための要である。手続きの設計思想を理解するには [105] の課題設定に立ち返ることが近道であり、試験による検証を課題解決の定義そのものに据えた点に、この系列の特徴がある。単なる正答の一致ではなく、リポジトリの状態を変化させたうえで試験が通ることが求められるため、読む側も、どのリポジトリのどの状態に対してどの試験が実行されたのかを問わなければならない。

この系列では、初期の収集に含まれる不良課題や仕様の曖昧さへの反省から、精選と再設計が妥当性の証拠として位置づけられてきた。精選版の考え方は、壊れた課題や検証として弱い課題を取り除き、人手の確認を重ねて検証可能性を高めることにある。後継の再設計は、長期にわたる開発作業を扱うために、人手で検証された課題群を改めて編成し、混入に強い収集と人手による補強を組み合わせた点に意義がある。設計の変遷を追うには [106] の再設計の意図を読み解くことが欠かせない。重要なのは、精選や再設計が得点の上積みのためではなく、物差しとしての正当性を高めるための営みだということである。

さらに、この系列の分割の規律は厳格である。公開用と商用と保持用の分割は、混入耐性と運用目的の違いを反映したものであり、互いに混ぜて読むことはできない。公開用の数字と保持用の数字は、出所も管理も比較の意味も異なる。分割をまたいだ比較は、物差しの取り違いであり、見かけの優劣を作り出すだけである。読む側の作法は単純であり、図ごとにどの分割の数字なのかを確かめ、分割が明示されていない表は比較の対象から外すことである。足場や道具の指定が図ごとに残されていない場合も同様であり、足場の違いは結果の違いに直結するため、足場の記載なき比較は成立しない。

Terminal-Bench 系の見方

端末作業を扱う評価では、応答文の見かけではなく、作業後の容器の状態が採点の対象になる。命令列の実行を終えたあとのファイルや設定や実行結果が、課題ごとに用意された結果駆動の検査で確かめられる。過程の記録がどれほど整っていても、最終状態が検査を通らなければ解決とはみなされない。逆に、過程が想定と違って、最終状

態が検査を満たせば解決として認められる。この考え方は、実務の作業が成果物の状態で評価されることに対応している。この採点観の全体像をつかむには [107] の課題設計と採点思想を一体で読むことが有効である。

この系列でもう一つ重要なのは、名称は契約ではないという点である。同じ呼び名であっても、版が変われば課題群も検査も難しさも変わりうる。版の違いを無視して呼び名だけで数字を並べると、別物の比較になる。読む側は、必ず版の指定を確認し、版の異なる数字を同一系列の時系列としてつながない。版の特定は、細部の注記ではなく、比較可能性の本体である。

加えて、足場への依存はこの系列でも避けられない。道具の与え方、許される操作の範囲、試行の回し方、検査との接続の仕方によって、同じ模型でも振る舞いが変わる。特定の足場で良かった結果が別の足場でも再現するとは限らない。足場の同一性が確認できない数字同士は比べず、足場の記載を欠く表は方法の開示不足とみなす。この慎重さが、端末作業の数字を実務の見通しに変えるための条件である。

長文脈と監査可能性の見方

長い文脈の主張を読むときは、公称と実効を区別する。受け付け可能な長さと、実際に課題解決に使える長さは別の事柄である。段階的に長さを延ばしながら、検索や多段の辿りや集約や質疑応答の遂行を確かめる段階的検証が、その区別を可視化する。短い文脈では解けることが、長い文脈でもそのまま保たれるとは限らない。長さが延びると、必要な情報の取り出しも、分散した情報の結びつけも、全体の要約も、質が変わる。したがって読む側は、どの長さの段階で何が測られ、どの長さから崩れ始めるのかを、段階別の姿として読む。長さ別の段階的評価の考え方を理解するには [109] の設計思想が手がかりになる。

一方、推論基盤の効率を監査可能な形で比べるには、模型の指定だけでは足りず、場面の指定が要る。到着過程や遅延制限や装置の特定を固定する手順が、比較の土台になる。到着の仕方を変えれば待ち行列の振る舞いが変わり、遅延制限を変えれば通せる要求の質が変わり、装置を変えれば同

じ模型でも速度の意味が変わる。場面の束縛こそが、二つの推論数字を同じ土俵に載せる条件である。この場面束縛の考え方を身につけるには [102] の方法論の役割を押さえることが近道である。

さらに、計測道具の注意書きを学びとして読む姿勢も欠かせない。道具によっては、顧客側の制約や分かち書きや標本抽出の交絡が、測定値に混ざり込むことが文書化されている。そうした注意書きは、道具の欠陥を責めるためではなく、比較不能の所在を教えるためにある。特定の道具の数字を測定の根拠として引用せず、むしろ何を固定しなければ数字が動いてしまうのかを学ぶ。その代表が [104] の位置づけであり、測定源ではなく非比較性の教師として扱うことで、道具の扱い方が見えてくる。

枠組みの開示を読む

精度側の評価を読むときは、足場の身元が数字の一部であることを忘れてはならない。同じ課題名であっても、評価の足場が違えば、課題の切り出しも採点の手順も適応の方法も変わりうる。足場の標準化は、こうしたぶれを抑え、再現可能な形で課題群を実行するための役割を担う。標準化の意義を理解するには [108] の役割に立ち返ることが有効である。足場の同一性が確認できない精度の数字は、同じ課題を測ったことにならない。同じ名の別足場という落とし穴を避けることが、精度の数字を実務に生かす第一歩である。

広い評価の主張を読むときは、透明性の姿勢が問われる。場面と指標と適応の方法を明らかにし、複数の観点からの報告を束ねることで、単一の順位づけに還元されない評価が可能になる。特定の表で都合の良い場面だけが切り取られていないか、指標の選び方に偏りがいないか、適応の手順が明示されているかを確認する。その規範をつかむには [110] の全体論的な評価の姿勢が手がかりになる。開示の厚みが、評価の信頼の厚みである。負荷条件付きの記録を読むときは、並行性と到着率の指定が数字の本体であることを押さえる。出力側の処理量、初回応答までの待ち、応答間の間隔、要求処理量といった指標は、負荷のかけ方と一体でしか意味を持たない。こうした負荷条件つ

き記録の作法を理解するには [103] の文書化された測定の考え方が参考になる。同時実行や要求率の指定、表と記録の対応が、数字を検証可能なものにする。後継への移行が示唆されている場合も、未解決は未解決のまま残す。後継の製品面が特定されていないなら、推測で補わず、開示の範囲で止める。

9.3 物差しの持ち帰り

持ち帰るべき確認は四つである。第一に、課題解決の数字は手続きと分割で読む。試験による検証の二系統を確認し、精選や再設計の来歴を確かめ、公開用と商用と保持用を混ぜず、足場の指定なき比較を避ける。第二に、端末作業の数字は最終状態と版と足場で読む。容器の状態に対する結果駆動の検査を確かめ、名称の同一性を契約と取り違えず、版の指定と足場の同一性を比較の条件にする。第三に、長文脈と効率の数字は階段と場面と注意書きで読む。段階的に延ばす検証で公称と実効を区別し、到着と遅延と装置の固定で場面をそろえ、道具の注意書きを比較不能の所在を教える学びとして使う。第四に、枠組みの数字は開示の厚みで読む。足場の身元を数字の一部とみなし、場面と指標と適応の開示を求め、負荷条件付き記録の並行性と到着率を確認し、未解決の後継関係は推測で閉じない。

この四つの確認の先に、総費用の視点を据える。速さの数字は、待ちと間隔と処理量と同時実行の条件のもとでの姿であり、文脈長の数字は、段階別の実効の姿であり、精度や解決の数字は、足場と版と分割のもとでの姿である。試行と修正と検証を含めた課題完遂の費用を見渡し、条件をそろえた数字だけを比べ、条件の異なる数字は並べない。物差しの持ち帰りとは、得点の記憶ではなく、この照合の習慣である。

9.4 混入への備えを読む

学習データに評価課題が紛れ込む可能性は、公開された課題ほど高まる。見かけの解決が暗記の映し込みかもしれない以上、集め方の備えとともに読む必要がある。収集時の時期の管理や人手による作り替え、外部からの不正な回避を抑えるための検査の追加は、漏えいを完全には消せないまでも、混入の影響を抑えるための当然の構えである [106]。公開用と商用と保持

用の分割も、この備えの一環である。公開された部分で調整された工夫が保持用の部分でも通用するのは別問題であり、異なる出所の数字を並べて優劣を語ることはできない。混入への備えが設計に織り込まれているかどうかは、物差しの信頼を測る物差しである。

読みの作法	確認すること	混ぜてはいけないもの
局面を分ける	出だし・間隔・全体処理量	待ちと滑らかさと捌き量の一括
場面を束ねる	到着・遅延上限・装置の同一性	条件欠きの毎秒量
区分を分ける	公開・商用・非公開の来歴	区分違いの成績の並置
版を添える	改訂・足回り・道具・手間	名だけの言及
混入に備える	集め方の備えと漏えい対策	暗記の映し込み
総費用で見る	速さと正確さと手間	片手落ちの比較
長さを疑う	公称上限と有効長のずれ	表示のままの実力視
報告を開く	場面・指標・適用の開示	代表値への畳み込み

[105, 106, 107, 109, 102]

本節の読解上の境界

個別の数値は隔離し、設計の系譜と作法だけを学ぶ。現物の複製や再実行には踏み込まない。後継の単独姿は未確認として保留する。新たな順位付けは作らない。

10 結び——層として読む効率

FINAL SYNTHESIS

本巻がたどったのは、効率化が単一の物差しから層の重なりへと変わる過程である。規模拡大の経験則を起点に、条件付き計算への分岐、注意・記憶の共有化と疎化・線形化、重みの低ビット化と局所実行、逐次復号の先読みと推論予算の管理、運用の束ね・再利用・分離、安価計算への振り分けと特化型の使い分けが、各層の律速に応じて積み重なる。容量と計算の分離が全層を貫く転換であり、総量の競争から稼働量・常駐量・保持量の設計へ関心が移った。

並行する競合関係も見えた。密な規模拡大と疎な条件付き計算、厳密な入出力最適化と計算・保存を変える構造、学習後量子化と原生低ビット、ドラフト検証による待ち削減と推論予算の節約、集中運用と分離運用、汎用生成と型付き特化判断。いずれも万能ではなく、作業負荷・文脈長・同時実行・装置条件で損得が変わる。どの隘路を、どの条件のもとで、何で削ったか。この問いを手放さないことが、効率の議論を曇らせない条件である。

未解決の系譜は残る。外部記憶表を持つ設計と条件付き記憶提案の導出関係、相手陣営の由来文書の公表時期、呼称の衝突、特化型の独立再現と較正手順、後

備えの記述がない数字は、暗記の可能性を排除できない数字として扱い、比較の対象から外す。読む側は、混入対策の章があるかどうかを探し、ない場合はその数字を割り引いて読む習慣を持つべきである。

継製品の単独姿、節単位の消費が残る方法論、頂点実装の図別ピンは、いずれも未解決のまま残す。境界の明示は弱さではなく、検証の順序を守る手続きである。数字を読む作法を身につけ、条件付きの主張は条件付きのまま受け止めること。それが本巻の持ち帰りである。

層の読み方を実践に移すとき、三つの順序が助けになる。第一に、削減対象の名指しである。学習か推論か、前処理か復号か、演算か容量か帯域か通信か、遅延か処理能力か、総量か有効量か、重みかキャッシュか、トークン単価か仕事単価か。対象を名指ししないまま手法を選ぶと、効くはずの条件と外れる。第二に、条件の束ねである。到着のさせ方や遅延の上限、同時実行や文脈長、装置の顔ぶれ、版と足回りと道具と手間をそろえて初めて、二つの数字は並べられる。第三に、深さに応じた言葉である。本文まで読んだこと、要旨だけ読んだこと、告知だけ聞いたこと、移植の存在だけ確かめたことを区別し、それぞれに許される言葉で語る。この三つを守れば、効率の議論は曇らない。

層ごとの要点を振り返る。学習の層では、配分の見

直しが器の大きさからデータの割り当てへ問いを移し、組成の設計が混合比を予算として明示化した。条件付き計算の層では、容量と計算の分離が総量の競争を作動量と常駐量の設計へ変え、載せる技術と削る技術の区別が投資の順序を正した。注意と記憶の層では、保持対象の設計変更と動かし方の改善が区別され、共有化から疎化、線形化、除去、再利用、条件付き参照、深さ適応へと置き場所の設計が広がった。精度の層では、対象と時点の分離が削り方の選択を整理し、器と手法の区別が手元実行の見通しを守った。復号の層では、受容率という単一の物差しが草案の進化を貫き、予算表が足す操作と削る操作を同じ表に載せた。運用の層では、記憶管理と束ね方が構造の利益を実現する条件となり、受け渡しの経済学が差し引きの姿勢を教えた。振り分けの層では、閾値の運用と適合の監視が節約を持続させ、特化判断が生成の代替ではなく前段のさばき役として居場所を得た。実装点の層では、四つの設計が順位ではなく機構の選択の違いとして並置され、証拠の深さに応じた言葉が守られた。測定層では、条件をそろえる作法が数字の誤読を防いだ。各層の要点は独立した教訓であると同時に、隣の層への問いかけでもある。配分の問いは組成へ、組成の問いは振り分けへ、振り分けの問いは監視へ、監視の問いは測定へと連なる。この連なりをたどることが、層として読むということである。

手法を選ぶ場面での指針を、層の言葉で言い直す。学習の場面では、器の大きさではなく配分と組成を問う。過小学習の有無、混合比の根拠、更新範囲の限定が要点になる。構造の場面では、厳密さを保つ工夫と構造を変える設計を区別する。保持対象の変更か動かし方の改善か、疎化の適用層はどこか、除去と再利用と選択のどれを使うのが要点になる。精度の場面では、対象と時点を分ける。重みか活性化か、学習後か適応時か原生か、器か手法かを問い、較正の被覆を確認する。復号の場面では、受容率と予算表を持つ。草案の来歴、受容条件の統制、足す操作と削る操作の配置、学習費用と推論費用の分離が要点になる。運用の場面では、まず漏れを塞ぐ。記憶管理、束ね方、受け渡しの差し引き、生きた文書の読み方が要点になる。振り分けの場面では、閾値と適合を運用する。止める・上げる・見直すの決定、確信度の帯別運用、分布変化への備えが要点になる。実装点を読む場面では、順位ではなく機構の選択と証拠の深さを見る。測定層の場面では、条件をそろえる。どの指針も、対象の名指

しと条件の束ねと深さに応じた言葉に還元される。指針を逆向きに使えば、失敗の診断にもなる。効かなかったときは、対象の取り違え、条件の不一致、深さの超過のいずれかを疑う。この往復が、本巻の実践である。

10.1 失敗の診断学

効かなかったときの診断も、層の言葉で行う。第一に疑うのは、対象の取り違えである。重みの軽さとキャッシュの軽さ、学習の効率と推論の効率、厳密さの改善と構造の変更、受容率の向上と予算の節約を取り違えていないかを確認する。第二に疑うのは、条件の不一致である。束ね量や同時実行数や文脈長や装置の顔ぶれ、版と足回りと道具と手間が、想定と現場でずれていないかを確認する。第三に疑うのは、深さの超過である。要旨だけ読んだことを本文読解として扱っていないか、告知を事実として引用していないか、移植の存在を正確性の裏書きにしていないかを確認する。三つの疑いのいずれにも当てはまらない場合、残る可能性は分布の変化である。振り分け器の前提、較正の前提、料金の前提が動いていないかを確認し、監視と再調整の環を回す。診断は非難ではなく、次の問いへの接続である。

10.2 層の読み方の訓練

層の読み方は、繰り返して身につく。新しい手法に出会ったら、まず対象を名指しする訓練をする。何を減らすのか、どの欄の軽さなのかを一文で書く。次に条件を束ねる訓練をする。到着と遅延と同時実行と文脈長と装置と版と足回りと道具と手間のうち、欠けている条件を書き出す。次に深さを確認する訓練をする。本文まで読んだのか、要旨だけか、告知だけか、存在だけかを記録する。三つの訓練を一枚の記録用紙にまとめ、手法ごと、実装点ごと、数字ごとに残す。記録が溜まれば、自分の分布での閾値と適合の判断材料になる。読む訓練と測る訓練と直す訓練が一続きになることが、層の読み方の完成である。

記録用紙の雛形

対象の名指し——何を減らすのか、どの欄の軽さなのかを一文で書く。条件の束ね——到着と遅延と同時実行と文脈長と装置と版と足回りと道具と手間のうち、欠けている条件を書き出す。深さの

確認——本文まで読んだのか、要旨だけか、告知
だけか、存在だけかを記録する。境界の記録——
未解決の事項と提供元主張の隔離を書き留める。
見直しの予定——分布変化と料金改定と版切替
えの確認時期を決める。一枚にまとめて残すこ
とが、訓練の完成である。

記録の活用

溜まった記録は、自分の分布での判断材料にな
る。閾値の初期値、確信度の帯別の対応、較正集
合の代表例、料金試算の前提、版切り替えの影響
を、記録から引き出す。記録の活用は、振り返り
の習慣である。版ごとや四半期ごとに記録を眺
め、前提の変化を確かめ、見直しの予定を更新す
る。記録が活用されて初めて、訓練は完成する。

10.3 立場別の指針

研究の立場では、構成の違いに注目する。同じ効率
の言葉でも、対象と条件と深さが違えば別の主張であ
ることを確かめ、自分の構成での再適合を怠らない。
未解決の系譜には手を出さず、並行関係として記録す
る。運用の立場では、漏れを先に塞ぐ。記憶管理と束
ね方と受け渡しの差し引きを固めてから、構造の利益
を取り込む。生きた文書を読み、固定された構成での
検証に留め、監視と再調整の環を回す。意思決定の立
場では、総費用で見る。トークン単価ではなく仕事単
価で比べ、提供元主張を事実 zu 格上げせず、未解決を
未解決のまま予算に織り込む。三つの立場は、対象の
名指しと条件の束ねと深さに応じた言葉という同じ作
法を共有する。立場が違っても、物差しの使い方は同
じである。

場面	問うこと	確かめること	失敗の疑い
学習	配分と組成	過小学習・混合比・更新範囲	器の大きさへの短絡
構造	厳密と変更の区別	保持対象・適用層・四分法の 選択	対象の取り違い
精度	対象と時点の分離	較正の被覆と器・手法の区別	較正の流用
復号	受容率と予算表	草案来歴・受容条件・費用 分離	直列と予算の混同
運用	漏れの封鎖	管理・束ね・差し引き・文書 読み	構造と運用の混同
振り分け	閾値と適合の運用	決定・帯別運用・変化への 備え	分布変化の見落とし
実装点	機構と深さ	選択の違いと許される言葉	順位付けの誘惑
測定	条件の束ね	束ね・版・足回り・総費用	条件違いの並置

[1, 2, 3, 4, 21, 36, 44, 47, 58, 65, 71, 73, 81, 84, 90, 91, 98, 92, 105, 102]

本巻の読解上の境界

本巻は順位付けも推奨も行わない。概要水準の主張は概要水準のまま扱い、提供元測定は隔離する。未解
決事項を閉じないことが、本巻の読みの作法である。

Source-backed Technical Notes / 出典に裏付けられた技術要点

各節の技術的主張と読解上の境界を、一次資料の裏付けとともに表にまとめる。数値は条件付きの報告として扱い、横断の優劣には用いない。

第1節—— 隘路の契約

論点	一次資料で確認する技術要点	読解上の境界
規模則	損失が規模・データ・計算に滑らかに従う関係の形	係数値・実験条件には立ち入らない
配分の見直し	過小学習の指摘と均等配分の処方箋	目安を普遍定数として扱わない
段階の分離	反復単位の継続的束ねと前処理・復号の分離	処理細部・速度幅には踏み込まない

[1, 2, 3]

第2節—— 条件付き計算の系譜

論点	一次資料で確認する技術要点	読解上の境界
疎ゲートの起点	トークンごとの専門家選択と補助損失	数式・実験表の細部は範囲外
並列と分割	分散配置と容量係数・管理可能性	bench 横断の数値比較はしない
振り分けの分類	一人選択・側選択・粒度・共有	方式の優劣の断定はしない
稼働量の経済学	総量と稼働・常駐の分離	常駐・通信の費用を直視する
学習の土台	低精度の大規模学習と予測目的	速度の定量主張はしない

[4, 5, 6, 8, 9, 11]

第3節—— 注意と記憶

論点	一次資料で確認する技術要点	読解上の境界
共有化	単一頭・束共有と移行可能性	代償の数値表は範囲外
厳密な高速化	分割と逐次正規化の両立	証明・速度表は範囲外
疎化と局所	索引選択と窓基準の対比	名称の混同を避ける
線形系譜	門付き差分と交互配置	再現は未確定
保持と再利用	除去・共有・条件付き記憶・深さ	数値比較はしない

[19, 20, 21, 22, 27, 36]

第4節—— 精度と局所実行

論点	一次資料で確認する技術要点	読解上の境界
重み限定の圧縮	外れ値分離と再構成誤差の抑制	活性値同時・学習時と混同しない
同時量子化	平滑化と重要度保護	導出細部は範囲外
器の理解	表現・コンテナ形式としての共有	量子化手法と呼ばない
低ビット浮動小数点	混合精度と微小スケーリング	査読・断定はしない
適応と原生	付加器と一ビット方向	配備の証拠として扱わない
手元実行	不均質配置と移植報告	配置依存の観察に留める

[43, 44, 53, 47, 50, 54]

第5節——復号と予算

論点	一次資料で確認する技術要点	読解上の境界
投機的検証	草案と一括検証、分布保存	証明・速度表は範囲外
先読みの質	特徴量予測と動的木、学習の改善と再利用	数値の一般化は留保
予算の足し算	検証器探索と予算強制	訓練費と推論費を分ける
予算の引き算	上限制約の学習と過思考の知見	対象外への一般化はしない

[58, 60, 63, 65, 67, 69]

第6節——運用

論点	一次資料で確認する技術要点	読解上の境界
記憶管理	塊単位と連続の束ね	表設計の細部は範囲外
使い回し	木共有と外部層	粒度の違いは使い分け
分離	段階別の資源と受け渡し	転送費用を差し引いて評価
噛み合わせ	断片化と釣り合い	区切り方は条件依存
基盤	核・通信・低精度の三層	数値は引用しない

[71, 72, 73, 74, 75]

第7節——振り分けと特化

論点	一次資料で確認する技術要点	読解上の境界
引き継ぎ	安価な順の試行と打ち切り	当時の料金の報告
学習型振り分け	好みデータの勝敗予測	分布依存の条件付き
脆さ	分布ずれの当たり外れ	一般化しない
特化判断	並列標本と確信度	生成の代替ではない
主張の扱い	速度・費用の優位の提示	提供元主張、再現未確認

[81, 82, 83, 84, 86]

第8節——実装点の照合

論点	一次資料で確認する技術要点	読解上の境界
疎活性化の設計	総量と作動量の分離	手順書は生きた文書
共同設計	残差・表・最適化の三軸	自己報告の性能表は隔離
線形系の実行	集積所と実行経路	数値・性能は未消費
要旨限定	要旨のみの消費	本文言語では記述しない

[90, 91, 98, 92]

第9節——測定の作法

論点	一次資料で確認する技術要点	読解上の境界
速さの分離	出だし・間隔・全体処理量	条件欠きの比較はしない
実課題の系譜	報告と試験の手順、区分の分離	区分違いを混ぜない
版の特定	改訂・足回り・道具・手間	名だけの言及はしない
長さの等級	公称と有効のずれ	内訳の断定は避ける

[105, 106, 107, 109, 102]

章末境界の一覧

各節末の境界ボックスを集めた索引である。本文の詳述に戻る際の道標として使う。

節	境界の要点
第 1 節	係数値・配分目安の普遍化・処理細部・速度幅に立ち入らない。土俵違いの比較をしない
第 2 節	同時代の優劣比較をしない。数式・評価表・通信定量・収束定理に立ち入らない
第 3 節	未確定系譜を断定しない。呼称衝突に触れない。数値・実装・速度を引用しない
第 4 節	容器形式を量子化手法と呼ばない。提供元数値を隔離する。査読・再構築をしない
第 5 節	証明・速度表・アブレーション・訓練費・対象外一般化に立ち入らない。直列性と予算を一括りにしない
第 6 節	設計細部・条件付き評価・配置解析・費用表に踏み込まない。文書対応の転用をしない
第 7 節	料金を現在価格に当てはめない。脆さを一般化しない。特化型を代替として語らない
第 8 節	順位付けをしない。要旨限定を守る。ベンダー比率を隔離する。手順書を転用しない
第 9 節	個別数値を隔離する。現物の複製・再実行をしない。後継を推測で閉じない。順位を作らない
結び	順位付けも推奨もしない。概要水準を概要のまま扱う。未解決を閉じない

用語集

本巻で使う技術用語の定義を集めたものである。節をまたいで同じ言葉が同じ意味で使われていることを確かめるために置いた。

用語	本巻での意味
プリフィル	入力全体を一気に読み込み内部状態を構築する前処理の段階。並列に処理しやすく演算が律速になりやすい
デコード	トークンを一つずつ逐次的に生み出す復号の段階。重みと中間記憶の読み出しが律速になりやすい
遅延	一つの応答が返るまでの待ち時間。対話の応答性に関わる
スループット	単位時間あたりに処理できる要求量。同時接続の捌き量に関わる
総パラメータ	模型が保持する重みの全体規模。保存容量と読み込み対象を決める
有効パラメータ	一トークンの処理に実際に寄与する部分の規模。一トークンあたりの計算量の見通しを与える
作動量	実際に計算に寄与する部分の量。総量との対比で効率を測る物差し
KV キャッシュ	逐次生成に伴って蓄積されるキーと値の中間記憶。文脈長と同時実行数に応じて膨らむ変動費
条件付き計算	トークンごとに一部の専門家だけを稼働させる計算様式。容量と計算を切り離す
混合専門家	トークンごとに一部の専門家だけを稼働させる構成。容量と計算を切り離す
専門家	条件付き計算で選択される部分模型。細分化と共有化の対象になる
共有専門家	共通知識を受け持つ専門家。細分化された専門家と分担する
混合特殊化	専門家を細かく刻んで分担の質を上げる設計。通信との両立が要点
ゲーティング	どの専門家を働かせるかを定める学習された選択機構。偏りと補助損失が論点になる
補助損失	選択の偏りを穏やかにいさめる損失。均等強制ではなく加減が要点
容量係数	振り分け先の溢れを扱うための容量の決め方。運用の安定に直結する
振り分け	トークンや問いを専門家や模型に割り当てる操作。経路選択とも呼ぶ
振り分け器	問いや処理の行き先を決める予測器。分布依存の条件付き部品
混合比	学習データの領域別配合。予算配分の上流にある割当

用語	本巻での意味
疎な注意	見るべき場所だけを選ぶ注意機構。索引選択と局所窓の二つの顔を持つ
疎な索引	注目先を選び出す軽量の索引機構。細粒度の選択を担う
束共有	照会頭の群ごとに鍵と値を共有する方式。記憶削減と表現力の中間点
潜在共有	照会頭群で共有される潜在表現へ圧縮する方式。保持と再構成を分担する
線形注意	二次計算を漸化的な状態更新に置き換える注意系列。保持形式そのものを変える
残差ゲート	残差経路に設けた素子ごとの扉。表参照と連動して設計される
塊管理	鍵と値の履歴を小さな塊に分けて管理する運用。断片化を抑え再利用の単位をそろえる
接頭辞再利用	共通の前置きの計算結果を使い回す運用。木共有と塊照合の二つの流儀がある
分離配置	前処理と復号を別の資源溜まりに置く運用。受け渡し転送の費用を伴う
保持と退避	記憶を近くに置くことと遠くに預けること。呼び戻しの費用と引き換えになる
再利用	作った計算結果を使い回すこと。当たれば得をし外れても正しさを損なわない
チャンク化前処理	長い読み込みを小さな断片に分けて生成の合間に混載する運用
一括処理方針	切り幅と顔ぶれの組み立てで待ちの配分を決める運用方針
蒸留	教師の柔らかな出力を目標に小さな模型へ能力を移す手法。容量と品質を切り離す
低ランク適応	凍結重みに低ランク行列だけを挿入して微調整する手法。更新範囲を絞る
チェックポイント	学習済み重みと構成の検査点。同一性の固定に用いる
量子化	数値のビット幅を減らして記憶と計算を軽くする操作。対象と時点の区別が要る
コンテナ形式	テンソル配置と付随情報を定める器。量子化手法ではない
較正データ	丸めの基準を定める小さな参照集合。被覆の代表性が誤差を左右する
平滑化	活性化の振れを均して重み側へ難しさを移す操作。同時量子化の前提になる
重要度保護	顕著な重みを優先的に守る操作。少量の較正で強い低ビット化を可能にする
不均質実行	中央演算装置と画像演算装置に専門家を配置換えする運用。容量と計算を切り離す

用語	本巻での意味
受容率	草案が標的模型に受け入れられる割合。投機的復号の効率を決める物差し
ドラフト	標的模型の検証を受けるために先に作られる草案。小さな模型や先読み層が担う
投機的復号	草案と一括検証で直列性を緩和する方式。受容率が効率を決める
受容条件	分布を保ったまま加速を正当化する条件。無制約な近似と区別する
先読み	未来の複数トークンを予測する学習と草案生成。補助課題と復号再利用の両面を持つ
確信度ヘッド	振り分けの確からしさを見積もる付加的な頭。受容の機会を広げる
検証器	出来栄えを判定する付加的な模型。難易度適応の探索で用いる
予算強制	思考の打ち切りと延長で推論予算を操作する手法。短い思考の延ばしと長い思考の抑止を含む
過思考	長い推論が精度低下を招く現象。正答の反転を伴うことがある
推論努力量	推論に投じる思考の量の指定。数値と段階のつまみで操作する
カスケード	安価な模型から順に試し不足分だけ上位へ引き継ぐ設計。打ち切り判定が核心になる
較正	確信度に見合った頻度で当たりが出る集計的性質。個の正しさの保証ではない
縮退動作	適合劣化時に安全側へ倒す正規の動作。利用者への影響を抑える
出だしの待ち	応答の最初の文字が出るまでの時間。TTFT と呼ばれる
生成の間隔	生成が続く区間の一トークンあたりの時間。TPOT や ITL と呼ばれる
全体処理量	負荷をかけたときの単位時間あたりの出力。到着率と同時実行の条件と一体である
枠組み	評価の道具立てと採点手順。枠組み名は数字の一部をなす
足回り	評価の枠組みと実行環境。枠組み名は数字の一部をなす
版	課題集合や判定手順の改訂。名が同じでも版が違えば別の物差しである
証拠の深さ	本文消費・要旨限定・告知水準・移植報告の四段階。許される言葉の範囲を決める
本文消費	本文まで構造的に読み込めた範囲。構造と実装条件を語れるが数値は語れない
要旨限定	要旨のみを消費した範囲。本文言語では記述しない
告知水準	発表と文書の範囲。方向と構想の表明として読む
移植報告	実装存在の証拠の範囲。存在は語れるが正確性の断定には進まない

References

- [1]J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei. *Scaling Laws for Neural Language Models*. Primary paper used in this survey; scaling-form scope. Jan. 2020. URL: <https://arxiv.org/abs/2001.08361> (visited on 09/21/2026).
- [2]J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de Las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, J. W. Rae, O. Vinyals, and L. Sifre. *Training Compute-Optimal Large Language Models*. Primary paper used in this survey; headline allocation prescription. Mar. 2022. URL: <https://arxiv.org/abs/2203.15556> (visited on 09/21/2026).
- [3]G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun. *Orca: A Distributed Serving System for Transformer Generative Models*. Primary paper used in this survey; scheduling-design scope. Aug. 2022. URL: <https://www.usenix.org/conference/osdi22/presentation/yu> (visited on 09/21/2026).
- [4]N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*. Primary paper used in this survey; survey-level mechanism. Jan. 2017. URL: <https://arxiv.org/abs/1701.06538> (visited on 09/21/2026).
- [5]D. Lepikhin, H. Lee, Y. Xu, D. Chen, O. Firat, Y. Huang, M. Krikun, N. Shazeer, and Z. Chen. *GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding*. Primary paper used in this survey; survey-level mechanism. June 2020. URL: <https://arxiv.org/abs/2006.16668> (visited on 09/21/2026).
- [6]W. Fedus, B. Zoph, and N. Shazeer. *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity*. Primary paper used in this survey; survey-level mechanism. Jan. 2021. URL: <https://arxiv.org/abs/2101.03961> (visited on 09/21/2026).
- [7]Y. Zhou, T. Lei, H. Liu, N. Du, Y. Huang, V. Zhao, A. Dai, Z. Chen, Q. Le, and J. Laudon. *Mixture-of-Experts with Expert Choice Routing*. Primary paper used in this survey; survey-level mechanism. Feb. 2022. URL: <https://arxiv.org/abs/2202.09368> (visited on 09/21/2026).
- [8]DeepSeek-AI. *DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models*. Primary paper used in this survey; survey-level mechanism. Jan. 2024. URL: <https://arxiv.org/abs/2401.06066> (visited on 09/21/2026).
- [9]DeepSeek-AI. *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model*. Primary paper used in this survey; established configuration facts. May 2024. URL: <https://arxiv.org/abs/2405.04434> (visited on 09/21/2026).
- [10]A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. de las Casas, E. B. Hanna, F. Bressand, G. Lengyel, G. Bour, G. Lample, L. R. Lavaud, L. Saulnier, M.-A. Lachaux, P. Stock, S. Subramanian, S. Yang, S. Antoniak, T. L. Scao, T. Gervet, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed. *Mixtral of Experts*. Primary paper used in this survey; established configuration facts. Jan. 2024. URL: <https://arxiv.org/abs/2401.04088> (visited on 09/21/2026).
- [11]DeepSeek-AI. *DeepSeek-V3 Technical Report*. Primary paper used in this survey; training-substrate scope. Dec. 2024. URL: <https://arxiv.org/abs/2412.19437> (visited on 09/21/2026).
- [12]DeepSeek-AI. *DeepGEMM: clean and efficient FP8 GEMM kernels*. Library authority; role-level use, body out of scope. URL: <https://github.com/deepseek-ai/DeepGEMM> (visited on 09/21/2026).
- [13]M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro. *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism*. Primary paper used in this survey; placement-context scope. Sept. 2019. URL: <https://arxiv.org/abs/1909.08053> (visited on 09/21/2026).

- [14]S. Rajbhandari, J. Rasley, O. Ruwase, and Y. He. *ZeRO: Memory Optimizations Toward Training Trillion Parameter Models*. Primary paper used in this survey; placement-context scope. Oct. 2019. URL: <https://arxiv.org/abs/1910.02054> (visited on 09/21/2026).
- [15]Keller Jordan. *Muon optimizer reference implementation*. Implementation authority; convergence theory out of scope. URL: <https://github.com/KellerJordan/Muon> (visited on 09/21/2026).
- [16]S. M. Xie, H. Pham, X. Dong, N. Du, H. Liu, Y. Lu, P. Liang, Q. V. Le, T. Ma, and A. W. Dai. *DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining*. Primary paper used in this survey; composition-design scope. May 2023. URL: <https://arxiv.org/abs/2305.10429> (visited on 09/21/2026).
- [17]G. Hinton, O. Vinyals, and J. Dean. *Distilling the Knowledge in a Neural Network*. Primary paper used in this survey; lineage-pointer scope. Mar. 2015. URL: <https://arxiv.org/abs/1503.02531> (visited on 09/21/2026).
- [18]E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. Primary paper used in this survey; survey-level mechanism. June 2021. URL: <https://arxiv.org/abs/2106.09685> (visited on 09/21/2026).
- [19]N. Shazeer. *Fast Transformer Decoding: One Write-Head Is All You Need*. Primary paper used in this survey; established mechanism. Nov. 2019. URL: <https://arxiv.org/abs/1911.02150> (visited on 09/21/2026).
- [20]J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai. *GQA: Training Generalized Multi-Query Transformers from Multi-Head Checkpoints*. Primary paper used in this survey; established mechanism. May 2023. URL: <https://arxiv.org/abs/2305.13245> (visited on 09/21/2026).
- [21]T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. Primary paper used in this survey; exact-IO thesis scope. May 2022. URL: <https://arxiv.org/abs/2205.14135> (visited on 09/21/2026).
- [22]Yuan et al. *Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention*. Primary paper used in this survey; abstract scope, ablations outstanding. Feb. 2025. URL: <https://arxiv.org/abs/2502.11089> (visited on 09/21/2026).
- [23]A. Q. Jiang, A. Sablayrolles, A. Mensch, C. Bamford, D. S. Chaplot, D. de las Casas, F. Bressand, G. Lengyel, G. Lample, L. Saulnier, L. R. Lavaud, M.-A. Lachaux, P. Stock, T. L. Scao, T. Lavril, T. Wang, T. Lacroix, and W. E. Sayed. *Mistral 7B*. Primary paper used in this survey; sliding-window anchor scope. Oct. 2023. URL: <https://arxiv.org/abs/2310.06825> (visited on 09/21/2026).
- [24]DeepSeek. *Introducing DeepSeek-V3.2-Exp*. Launch announcement scope; indexer equations belong to the report. Sept. 2025. URL: <https://www.deepseek.com/en/news/v3-2-exp/> (visited on 09/21/2026).
- [25]DeepSeek-AI. *DeepSeek-V3.2 report*. Primary paper used in this survey; section-scope consumption. Dec. 2025. URL: <https://arxiv.org/abs/2512.02556> (visited on 09/21/2026).
- [26]S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim. *Parallelizing Linear Transformers with the Delta Rule over Matrix-Valued States*. Primary paper used in this survey; abstract scope. Dec. 2024. URL: <https://arxiv.org/abs/2412.06464> (visited on 09/21/2026).
- [27]Moonshot AI. *Kimi Linear: An Expressive, Efficient Attention Architecture*. Primary paper used in this survey; section-scope consumption. Oct. 2025. URL: <https://arxiv.org/abs/2510.26692> (visited on 09/21/2026).
- [28]Qwen Team. *Qwen3.8-Flash-Next README (Hugging Face)*. First-party model documentation; vendor scores quarantined to stated conditions. URL: <https://huggingface.co/Qwen/Qwen3.8-Flash-Next> (visited on 09/21/2026).
- [29]Qwen Team. *Qwen3.8-Flash-Next repository (N-gram versus Engram lineage assessment)*. Repository authority; lineage parallel-or-unresolved, never asserted. URL: <https://github.com/QwenLM/Qwen3.8-Flash-Next> (visited on 09/21/2026).

- [30]Flash Linear Attention contributors. *KDA kernel implementation (FLA) with vLLM path*. Implementation authority; numerics and performance out of scope. URL: <https://github.com/fla-org/flash-linear-attention/tree/main/fla/ops/kda> (visited on 09/21/2026).
- [31]Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen. *H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models*. Primary paper used in this survey; survey-level mechanism. June 2023. URL: <https://arxiv.org/abs/2306.14048> (visited on 09/21/2026).
- [32]G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis. *Efficient Streaming Language Models with Attention Sinks*. Primary paper used in this survey; survey-level mechanism. Sept. 2023. URL: <https://arxiv.org/abs/2309.17453> (visited on 09/21/2026).
- [33]D. Cai, T. Gan, H. Mo, Y. Wang, J. Liu, X. Liu, J. Li, G. Jiang, and H. Zhang. *PyramidKV: Dynamic KV Cache Compression based on Pyramidal Information Funneling*. Primary paper used in this survey; survey-level mechanism. June 2024. URL: <https://arxiv.org/abs/2406.02069> (visited on 09/21/2026).
- [34]vLLM contributors. *vLLM Automatic Prefix Caching (documentation)*. Documentation authority; release level, line-level detail outstanding. URL: https://docs.vllm.ai/en/latest/features/automatic_prefix_caching/ (visited on 09/21/2026).
- [35]LMCache contributors. *LMCache (external KV-cache layer)*. Implementation authority; role-level use. URL: <https://github.com/LMCache/LMCache> (visited on 09/21/2026).
- [36]Cheng et al. *Conditional Memory via Scalable Lookup / Engram*. Primary paper used in this survey; section-scope consumption. Jan. 2026. URL: <https://arxiv.org/abs/2601.07372> (visited on 09/21/2026).
- [37]D. Raposo, S. Ritter, B. Richards, T. Lillicrap, P. Humphreys, and A. Santoro. *Mixture-of-Depths: Dynamically allocating compute in transformer-based language models*. Primary paper used in this survey; survey-level mechanism. Apr. 2024. URL: <https://arxiv.org/abs/2404.02258> (visited on 09/21/2026).
- [38]Elhoushi et al. *LayerSkip: Enabling Early Exit Inference and Self-Speculative Decoding*. Primary paper used in this survey; survey-level mechanism. Apr. 2024. URL: <https://arxiv.org/abs/2404.16710> (visited on 09/21/2026).
- [39]J. Xin, R. Tang, J. Lee, Y. Yu, and J. Lin. *DeeBERT: Dynamic Early Exiting for Accelerating BERT Inference*. Primary paper used in this survey; precedent-role scope only. Apr. 2020. URL: <https://arxiv.org/abs/2004.12993> (visited on 09/21/2026).
- [40]DeepSeek-AI. *FlashMLA: efficient MLA decoding kernels*. Kernel authority; release level, benchmark tables out of scope. URL: <https://github.com/deepseek-ai/FlashMLA> (visited on 09/21/2026).
- [41]DeepSeek. *DeepSeek-V3.2-Exp launch and API documentation (DSA debut)*. Launch and API documentation scope; excerpt-level consumption. URL: <https://deepseek.com/en/news/v3-2-exp> (visited on 09/21/2026).
- [42]DeepSeek-AI. *Engram official implementation*. Implementation authority; role-level use. URL: <https://github.com/deepseek-ai/Engram> (visited on 09/21/2026).
- [43]T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer. *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale*. Primary paper used in this survey; survey-level mechanism. Aug. 2022. URL: <https://arxiv.org/abs/2208.07339> (visited on 09/21/2026).
- [44]E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh. *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers*. Primary paper used in this survey; survey-level mechanism. Oct. 2022. URL: <https://arxiv.org/abs/2210.17323> (visited on 09/21/2026).
- [45]G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han. *SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models*. Primary paper used in this survey; survey-level mechanism. Nov. 2022. URL: <https://arxiv.org/abs/2211.10438> (visited on 09/21/2026).

- [46]J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han. *AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration*. Primary paper used in this survey; method detail abstract scope. June 2023. URL: <https://arxiv.org/abs/2306.00978> (visited on 09/21/2026).
- [47]ggml contributors. *GGUF specification (ggml project)*. Specification authority. GGUF is a representation and container format, not a quantization algorithm. URL: <https://github.com/ggerganov/ggml/blob/master/docs/gguf.md> (visited on 09/21/2026).
- [48]llama.cpp contributors. *llama.cpp (GGML history, GGUF runtime, k-quants, CPU/GPU split)*. Implementation authority; line-level detail outstanding. URL: <https://github.com/ggerganov/llama.cpp> (visited on 09/21/2026).
- [49]Unsloth. *GLM-5.3-Flash local quantization authority (GGUF table, memory-fit guidance)*. Local packaging authority; GGUF availability and memory-fit scope only; vendor speed and quality figures quarantined. URL: <https://unsloth.ai/docs/models/glm-5.3-flash> (visited on 09/21/2026).
- [50]P. Micikevicius, D. Stosic, N. Burgess, M. Cornea, P. Dubey, R. Grisenthwaite, S. Ha, A. Heinecke, P. Judd, J. Kamalu, N. Mellempudi, S. Oberman, M. Shoeybi, M. Siu, and H. Wu. *FP8 Formats for Deep Learning*. Primary paper used in this survey; abstract scope. Sept. 2022. URL: <https://arxiv.org/abs/2209.05433> (visited on 09/21/2026).
- [51]Open Compute Project. *OCP Microscaling (MX) Specification v1.0*. Normative specification authority; element details at established level. URL: <https://www.opencompute.org/documents/ocp-microscaling-formats-mx-v1-0-spec-final-pdf> (visited on 09/21/2026).
- [52]vLLM contributors. *MXFP4/FP4-expert deployment evidence (V4 line and vLLM dispatch)*. Deployment documentation; release level. URL: https://docs.vllm.ai/en/latest/api/vllm/models/deepseek_v41/ (visited on 09/21/2026).
- [53]T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer. *QLoRA: Efficient Finetuning of Quantized LLMs*. Primary paper used in this survey; survey-level mechanism. May 2023. URL: <https://arxiv.org/abs/2305.14314> (visited on 09/21/2026).
- [54]H. Wang, S. Ma, L. Dong, S. Huang, H. Wang, L. Ma, F. Yang, R. Wang, Y. Wu, and F. Wei. *BitNet: Scaling 1-bit Transformers for Large Language Models*. Primary paper used in this survey; direction-level scope. Oct. 2023. URL: <https://arxiv.org/abs/2310.11453> (visited on 09/21/2026).
- [55]S. Ma, H. Wang, L. Ma, L. Wang, W. Wang, S. Huang, L. Dong, R. Wang, J. Xue, and F. Wei. *The Era of 1-bit LLMs: All Large Language Models are in 1.58 Bits*. Primary paper used in this survey; training-detail outstanding. Feb. 2024. URL: <https://arxiv.org/abs/2402.17764> (visited on 09/21/2026).
- [56]KTransformers contributors. *KTransformers (heterogeneous CPU/GPU MoE execution)*. Implementation authority; body outstanding. URL: <https://github.com/kvccache-ai/ktransformers> (visited on 09/21/2026).
- [57]llama.cpp contributors. *Qwen3.8-Flash-Next implementation port (merged PR 27742)*. Port report authority; correctness rests on PR-reported validation, no independent rebuild. URL: <https://github.com/ggml-org/llama.cpp/pull/27742> (visited on 09/21/2026).
- [58]Y. Leviathan, M. Kalman, and Y. Matias. *Fast Inference from Transformers via Speculative Decoding*. Primary paper used in this survey; mechanism and guarantee scope. Nov. 2023. URL: <https://arxiv.org/abs/2211.17192> (visited on 09/21/2026).
- [59]C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. *Accelerating Large Language Model Decoding with Speculative Sampling*. Primary paper used in this survey; companion origin scope. Feb. 2023. URL: <https://arxiv.org/abs/2302.01318> (visited on 09/21/2026).

- [60]Y. Li, F. Wei, C. Zhang, and H. Zhang. *EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty*. Primary paper used in this survey; speedups and guarantee scope consumed. Jan. 2024. URL: <https://arxiv.org/abs/2401.15077> (visited on 09/21/2026).
- [61]T. Cai, Y. Li, Z. Song, H. Fan, H. Xu, H. Wang, J. Wang, Y. Zhuang, D. Yang, H. Peng, Z. Huang, R. Girshick, Y. Li, K. Li, X. Chen, Z. Liu, and S. Xie. *Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads*. Primary paper used in this survey; precedent-bridge scope. Jan. 2024. URL: <https://arxiv.org/abs/2401.10774> (visited on 09/21/2026).
- [62]Y. Li, F. Wei, C. Zhang, and H. Zhang. *EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees*. Primary paper used in this survey; summary-level corroboration. June 2024. URL: <https://arxiv.org/abs/2406.16858> (visited on 09/21/2026).
- [63]F. Gloeckle, B. Y. Idrissi, B. Rozière, D. Lopez-Paz, and G. Synnaeve. *Better and Faster Large Language Models via Multi-token Prediction*. Primary paper used in this survey; mechanism and evaluation gains consumed. Apr. 2024. URL: <https://arxiv.org/abs/2404.19737> (visited on 09/21/2026).
- [64]vLLM contributors. *vLLM speculative decoding with Multi-Token Prediction (documentation)*. Documentation authority; release level. URL: https://docs.vllm.ai/en/latest/features/speculative_decoding/mtp (visited on 09/21/2026).
- [65]C. Snell, J. Lee, K. Xu, and A. Kumar. *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters*. Primary paper used in this survey; section-scope consumption. Aug. 2024. URL: <https://arxiv.org/abs/2408.03314> (visited on 09/21/2026).
- [66]N. Muennighoff, Z. Yang, W. Shi, X. L. Li, L. Fei-Fei, H. Hajishirzi, L. Zettlemoyer, P. Liang, E. Candès, and T. Hashimoto. *s1: Simple test-time scaling*. Primary paper used in this survey; section-scope consumption. Jan. 2025. URL: <https://arxiv.org/abs/2501.19393> (visited on 09/21/2026).
- [67]Hou et al. *ThinkPrune: Pruning Long Chain-of-Thought of LLMs via Reinforcement Learning*. Primary paper used in this survey; results table and protocol consumed. Apr. 2025. URL: <https://arxiv.org/abs/2504.01296> (visited on 09/21/2026).
- [68]DeepSeek-AI. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. Primary paper used in this survey; reasoning-context scope, distillation bounded. Jan. 2025. URL: <https://arxiv.org/abs/2501.12948> (visited on 09/21/2026).
- [69]Overthinking study authors. *When More Thinking Hurts: Overthinking in LLM Test-Time Compute Scaling*. Primary paper used in this survey; section-scope consumption. Apr. 2026. URL: <https://arxiv.org/abs/2604.10739> (visited on 09/21/2026).
- [70]DeepSeek-AI and Qwen Team. *Current-model effort-control binding (V4.1 and Qwen documentation)*. First-party model documentation; interface-summary scope. URL: <https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash> (visited on 09/21/2026).
- [71]W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica. *Efficient Memory Management for LLM Serving with PagedAttention*. Primary paper used in this survey; survey-level mechanism. Sept. 2023. URL: <https://arxiv.org/abs/2309.06180> (visited on 09/21/2026).
- [72]L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, H. Zhang, and J. E. Gonzalez. *SGLang: Efficient Execution of Structured Language Model Programs*. Primary paper used in this survey; survey-level mechanism. Dec. 2023. URL: <https://arxiv.org/abs/2312.07104> (visited on 09/21/2026).
- [73]P. Patel, E. Choukse, C. Zhang, Í. Goiri, A. Shah, S. Maleki, and R. Bianchini. *Splitwise: Efficient Generative LLM Inference Using Phase Splitting*. Primary paper used in this survey; survey-level mechanism. Nov. 2023. URL: <https://arxiv.org/abs/2311.18677> (visited on 09/21/2026).

- [74]R. Qin, Z. Li, Y. Ren, S. Zhu, M. Zhang, Y. Lou, Z. Wang, and C. Zhang. *Mooncake: A KVCache-centric Disaggregated Architecture for LLM Serving*. Primary paper used in this survey; abstract scope. July 2024. URL: <https://arxiv.org/abs/2407.00079> (visited on 09/21/2026).
- [75]A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee. *SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills*. Primary paper used in this survey; chunked-prefill lineage scope with 2024 follow-up bound additively. Aug. 2023. URL: <https://arxiv.org/abs/2308.16369> (visited on 09/21/2026).
- [76]DeepSeek-AI. *DeepEP (expert-parallel communication library)*. Library authority; communication-cost corroboration, body outstanding. URL: <https://github.com/deepseek-ai/DeepEP> (visited on 09/21/2026).
- [77]FlashInfer contributors. *FlashInfer (AI kernels for LLM serving)*. Kernel-stack authority; production-stack corroboration, body outstanding. URL: <https://github.com/flashinfer-ai/FlashInfer> (visited on 09/21/2026).
- [78]vLLM contributors and NVIDIA Dynamo contributors. *DSpark evidence bundle (vLLM deepseek-v41 docs and Dynamo recipe)*. Serving and runtime corroboration; release level. URL: https://docs.vllm.ai/en/latest/api/vllm/models/deepseek_v41/ (visited on 09/21/2026).
- [79]vLLM contributors. *vLLM DeepSeek-V4.1-Flash recipe (deployment authority)*. Deployment recipe authority; living documentation, pinned-config validation only. URL: <https://recipes.vllm.ai/deepseek-ai/DeepSeek-V4.1-Flash> (visited on 09/21/2026).
- [80]vLLM contributors. *vLLM deepseek-v41 model code documentation*. Documentation authority; runtime-implementation corroboration. URL: https://docs.vllm.ai/en/latest/api/vllm/models/deepseek_v41/ (visited on 09/21/2026).
- [81]L. Chen, M. Zaharia, and J. Zou. *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*. Primary paper used in this survey; cascade-design scope, prices stale by construction. May 2023. URL: <https://arxiv.org/abs/2305.05176> (visited on 09/21/2026).
- [82]I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica. *RouteLLM: Learning to Route LLMs with Preference Data*. Primary paper used in this survey; routing-design scope. June 2024. URL: <https://arxiv.org/abs/2406.18665> (visited on 09/21/2026).
- [83]Router-LLM fragility study authors. *Router-LLM fragility analysis (EACL 2026 Findings)*. Primary paper used in this survey; summary scope, no generalization beyond summary. URL: <https://aclanthology.org/2026.findings-acl.1199.pdf> (visited on 09/21/2026).
- [84]TypeSafe. *Introducing System One Models and Jev (TypeSafe launch)*. First-party launch authority; vendor claims bounded by thin independent reproduction. Sept. 2026. URL: <https://typesafe.ai/blog/introducing-system-one-models-and-jev> (visited on 09/21/2026).
- [85]TypeSafe. *TypeSafe docs: System One concepts and introduction*. First-party documentation; suitability and task-class bounds documented. URL: <https://docs.typesafe.ai/concepts/system-one> (visited on 09/21/2026).
- [86]TypeSafe. *Jev 1.13 official Models page*. First-party model documentation; latency, price, and throughput figures are vendor claims. URL: <https://docs.typesafe.ai/models> (visited on 09/21/2026).
- [87]Jev community. *Jev wiki: Jev 1.13 model-card facts*. Community snapshot; timeline corroboration only, explicitly not specification authority. URL: <https://jevai.wiki/> (visited on 09/21/2026).
- [88]OmniJev contributors. *awesome-jev community reproduction hub (OmniJev)*. Community hub; attempt-activity corroboration only. URL: <https://github.com/OmniJev/awesome-jev> (visited on 09/21/2026).
- [89]TypeSafe. *TypeSafe API reference*. First-party API documentation; contract details belong here, not in narrative claims. URL: <https://docs.typesafe.ai/api> (visited on 09/21/2026).

- [90]DeepSeek-AI. *DeepSeek-V4.1-Flash technical report*. Primary paper used in this survey; section-level consumption. Sept. 2026. URL: <https://arxiv.org/abs/2609.19969> (visited on 09/21/2026).
- [91]Qwen Team. *On the Design of Qwen3.8-Next Architecture*. Primary paper used in this survey; section-level consumption. Aug. 2026. URL: <https://arxiv.org/abs/2608.30320> (visited on 09/21/2026).
- [92]Z.ai. *GLM-5 technical report*. Primary but abstract-scope authority: report abstract consumed; full-body adoption details, RL infrastructure, and benchmark tables NOT consumed. Feb. 2026. URL: <https://arxiv.org/abs/2602.15763> (visited on 09/21/2026).
- [93]DeepSeek-AI. *DeepSeek-V4.1-Flash model card (Hugging Face)*. First-party model card; checkpoint-identity authority. URL: <https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash> (visited on 09/21/2026).
- [94]NVIDIA. *NVIDIA Dynamo DeepSeek-V4.1-Flash deployment recipe*. Deployment recipe authority; release level, body line-level detail outstanding. URL: <https://docs.nvidia.com/dynamo/dev/recipes/deepseek-v4-1-flash.md> (visited on 09/21/2026).
- [95]Qwen Team. *Qwen3.8-Flash-Next official blog*. First-party launch-claim authority; announcement scope. URL: <https://qwen.ai/blog?id=qwen3.8-flash-next> (visited on 09/21/2026).
- [96]Qwen Team. *Qwen3.8-Flash-Next GitHub (report and README source of truth)*. First-party repository authority; source-of-truth scope. URL: <https://github.com/QwenLM/Qwen3.8-Flash-Next> (visited on 09/21/2026).
- [97]NVIDIA. *Qwen3.8-Flash-Next on GB300 NVL72 (deployment blog)*. Deployment-existence evidence with prefix-hit-conditioned boundary; vendor-adjacent numbers quarantined. URL: <https://developer.nvidia.com/blog/experiment-with-qwen3-8-flash-next-on-nvidia-gb300-nvl72-for-agentic-coding> (visited on 09/21/2026).
- [98]Moonshot AI. *Kimi-Linear repository (KDA kernel, vLLM path, checkpoints)*. Implementation authority; no independent reproduction this pass. URL: <https://github.com/MoonshotAI/Kimi-Linear> (visited on 09/21/2026).
- [99]Z.ai. *GLM-5.3-Flash model card (Hugging Face)*. First-party model card; identity-fact scope. URL: <https://huggingface.co/zai-org/GLM-5.3-Flash> (visited on 09/21/2026).
- [100]Z.ai. *Z.ai developer docs: GLM-5.3-Flash overview*. First-party developer documentation; overview scope. URL: <https://docs.z.ai/guides/vlm/glm-5.3-flash> (visited on 09/21/2026).
- [101]Z.ai. *zai-org/GLM-5 GitHub (IndexShare, MTP, deployment matrix)*. First-party repository authority; deployment-matrix scope. URL: <https://github.com/zai-org/GLM-5> (visited on 09/21/2026).
- [102]MLCommons. *MLPerf Inference (LLM methodology)*. Methodology authority; auditable inference-measurement scope. URL: <https://mlcommons.org/benchmarks/inference/> (visited on 09/21/2026).
- [103]NVIDIA. *GenAI-Perf (Triton Inference Server tooling documentation)*. Tooling documentation; standalone product identity unresolved. URL: https://docs.nvidia.com/deeplearning/triton-inference-server/user-guide/docs/perf_analyzer/genai-perf/README.html (visited on 09/21/2026).
- [104]Ray contributors. *LLMPerf (Ray; caveated throughput harness)*. Harness authority; teaches incomparability, not ground truth. URL: <https://github.com/ray-project/llmperf> (visited on 09/21/2026).
- [105]C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* Primary paper used in this survey; Verified and Pro distinction anchor. Oct. 2023. URL: <https://arxiv.org/abs/2310.06770> (visited on 09/21/2026).
- [106]Y. Deng et al. *SWE-Bench Pro: Can AI Agents Solve Long-Horizon Software Engineering Tasks?* Primary paper used in this survey; validity-evolution case, section-level consumption outstanding. Sept. 2025. URL: <https://arxiv.org/abs/2509.16941> (visited on 09/21/2026).

- [107]A. Merrill et al. *Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces*. Primary paper used in this survey; agentic and tool-access case. Jan. 2026. URL: <https://arxiv.org/abs/2601.11868> (visited on 09/21/2026).
- [108]EleutherAI. *lm-evaluation-harness*. Harness authority; framework-name scope. URL: <https://github.com/EleutherAI/lm-evaluation-harness> (visited on 09/21/2026).
- [109]C.-P. Hsieh, S. Sun, S. Krman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg. *RULER: What's the Real Context Size of Your Long-Context Language Models?* Primary paper used in this survey; abstract scope. Apr. 2024. URL: <https://arxiv.org/abs/2404.06654> (visited on 09/21/2026).
- [110]Stanford CRFM. *HELM (holistic evaluation of language models)*. Harness authority; transparency-posture scope. URL: <https://github.com/stanford-crfm/helm> (visited on 09/21/2026).